

POLITECNICO DI TORINO

Master's Degree in Data Science and Engineering



**Politecnico
di Torino**

EPFL

Master's Degree Thesis

Input-Affine Spatiotemporal GNN for Fault Detection and Isolation in Closed-Loop IIoT Systems

Supervisors:

Prof. Marco MELLIA

Prof. Photios STAVROU

PhD Mengjie ZHAO

Candidate:

Mattia MOLINARI

October 2026

Abstract

Condition monitoring plays a crucial role in ensuring operational efficiency, safety, and reliability across modern industrial plants, where automated Fault Detection and Isolation (FDI) serves as a primary defense by recognizing anomalies and diagnosing root causes. To tackle this problem, the increasing availability of sensor telemetry has enabled the widespread adoption of data-driven approaches; however, traditional methods often fail to account for complex interdependencies between process units, leading to suboptimal performance. While graph-based frameworks effectively model the structural interconnectedness, existing architectures fail to decouple control actions from intrinsic dynamics and struggle in tightly regulated closed-loop environments. In such settings, feedback controllers actively compensate for abnormal behavior, thereby masking faults and hindering the ability to accurately identify their root causes. To date, the lack of a principled approach for incorporating control variables into graph-based models remains a considerable challenge, limiting the applicability of these methods in real-world industrial scenarios. Addressing this gap, this thesis introduces an input-affine Spatiotemporal Graph Neural Network framework for unsupervised FDI, which explicitly integrates control inputs through a dedicated linear channel. This formulation allows the modeled effects of control actions to be strictly separated from autonomous dynamics, enabling actuator faults to be accurately isolated and identified as parametric gain shifts. Evaluated on representative case studies, our approach improves dynamic modeling accuracy and significantly outperforms baseline methods in diagnosing root causes under closed-loop control.

Summary

Introduction

Modern industrial infrastructures have evolved into ultra-complex dynamical systems, characterized by several interconnected units operating in equilibrium. In these safety-critical environments, process monitoring is structured around Anomaly Detection (AD), i.e., the identification of out-of-distribution behaviors, and Fault Detection and Isolation (FDI), which localizes the root cause and physical origin of a disruption. While the pervasive integration of the Industrial Internet of Things has facilitated the acquisition of high-dimensional multivariate time series telemetry, traditional analytical methods and isolated data-driven baselines struggle to model these systems due to persistent challenges: multi-modal operating regimes, severe telemetry heterogeneity, noise amplification, dynamic and nonlinear interactions, and propagation delays.

Crucially, industrial processes are governed by closed-loop feedback controllers. Feedback loops actively adjust manipulated actuators to reject disturbances, which frequently suppresses symptoms, distorts causal directionality, and masks underlying faults. While Spatiotemporal Graph Neural Networks (STGNNs) have emerged as powerful paradigms by mapping process variables to nodes and dependencies to edges, contemporary architectures exhibit critical limitations. Because graph-based methods blend spatial message passing with temporal recurrence, they can inadvertently introduce non-causal relationships or smear feedbacks across the network when operating in closed-loop scenarios. Furthermore, existing approaches primarily optimize for global AD, whereas high-precision unlabeled FDI remains almost universally restricted to supervised classification architectures.

To overcome these deficiencies, this thesis develops an unsupervised framework tailored for closed-loop industrial processes, answering three foundational research questions: integrating spatial topology with temporal recurrence under closed-loop control (RQ1), achieving interpretable structural FDI without labeled failure catalogs (RQ2), and evaluating topological agnosticism when relying on physical schematics versus data-driven system identification (RQ3).

Proposed method

We introduce an Input-Affine Spatiotemporal Graph Neural Network (IA-STGNN), a novel two-stage unsupervised framework for predictive condition monitoring, robust AD, and parametric fault isolation. The industrial plant is formalized as a discrete temporal graph, with topological dependencies either inferred from physical schematics or learned via a separate data-driven model.

Healthy process modeling. The architecture integrates a spatiotemporal GRU to encode historical states, with intermediate incoming messages aggregated via a gated mechanism:

$$M_t^i = \sum_{j \in \mathcal{N}(i)} \sigma(\mathbf{W}_g \tilde{\mathbf{h}}_{t-1}^{ij} + \mathbf{b}_g) \tanh(\mathbf{W}_m \tilde{\mathbf{h}}_{t-1}^{ij} + \mathbf{b}_m), \quad \tilde{\mathbf{h}}_{t-1}^{ij} = [\mathbf{h}_{t-1}^i \parallel \mathbf{h}_{t-1}^j].$$

Hidden representations are then propagated through a neural input-affine dynamical system to forecast future manipulated and controlled variables:

$$\begin{aligned} \dot{x}_t^i &= a_\theta(\mathbf{h}_t^i) + b_\theta(\mathbf{h}_t^i)u_t^j, \\ \dot{u}_t^i &= c_\theta(\mathbf{h}_t^j). \end{aligned}$$

By routing control signals strictly through an explicit linear channel, feedback actions are structurally decoupled from autonomous degradation, preventing the network from utilizing actuator dynamics to cancel out underlying fault signatures. Additionally, the model exploits a regularized latent space of learned node embeddings to capture local heterogeneous sensor characteristics within a global paradigm. Multi-step forecasting is executed via an explicit forward Euler integration rollout, and the healthy training objective minimizes the MSE on integrated displacements.

AD and Sparse Parameter Isolation. During the first stage, raw forecasting residuals are converted into smoothed residual energy via an Exponentially Weighted Moving Average, where the smoothing factor is calibrated using the signal’s first-order autocorrelation. Scaled z-scores are checked against quantile thresholds, calibrated under healthy validation regimes with a fixed significance level p . Finally, alarms are declared via sliding-window majority voting. Once an alarm is triggered, the second stage reparameterizes the model with physical deviation parameters, accounting for additive node drift, edge coupling scaling, input gain shifts, and control law drift. A candidate mask restricts optimization to the top- k threshold-exceeding variables, and the system is solved via Ordinary Least Squares regularized with a Lasso penalty to yield a sparse, interpretable structural diagnostic vector.

Results

The framework was evaluated across the Tennessee Eastman Process (TEP) and Fluid Catalytic Cracking Unit (FCCU) case studies, benchmarked against a comprehensive set of statistical, recurrent, and graph-based architectures. On TEP, IA-STGNN achieved superior performance across nearly all metrics, outperforming baselines in accuracy, F1 score, and AUC. Most critically, it reduced the best baseline mean detection delay by more than 92%. Under insidious, low-amplitude fault regimes, IA-STGNN delivered an AUC roughly 50% higher and improved precision up to fourfold over alternative methods. On FCCU, baseline models exhibited severe recall degradation caused by tight multi-loop control interactions; in contrast, IA-STGNN achieved balanced detection with an F1 score 26% higher and a 31% reduction in detection latency. Topological agnosticism was confirmed by the competitive performance of data-driven connectivity, whereas extended horizon forecasting maintained consistent AD metrics while halving the detection delay.

In diagnostic isolation evaluations, the model correctly decomposed complex faults into physical mechanisms and localized failing subunits in the majority of scenarios. More precisely, we successfully distinguished valve degradation from feed disturbances, and actuator compensation from sensor drift.

Conclusion

The work validates IA-STGNN as a unified, unsupervised methodology for spatiotemporal, structural FDI in closed-loop industrial facilities. By formalizing dynamics as an input-affine system and enforcing edge independence via unnormalized message passing, the architecture resolves the confounding effects introduced by feedback controllers and recycle loops. The substantial reduction in detection delay provides the temporal margin required to catch localized faults before systemic contamination occurs. Empirical analysis highlights important directions for future research. A key limitation stems from purely magnitude-based ranking for fault signatures: because non-zero detection delays allow downstream high-energy consequences to propagate, secondary effects can occasionally overshadow principal causes. Additionally, uniform Lasso penalties can prove overly restrictive on subtle faults. Implementing adaptive, variance-normalized regularization, transitioning to nonlinear causal discovery algorithms, incorporating explicit variable-delay attention kernels, and exploring surrogate scoring metrics represent promising paths to advance quantitative, unsupervised industrial root-cause analysis.

Table of Contents

Summary	I
List of Figures	VII
List of Tables	IX
Acronyms	XI
1 Introduction	1
2 Background	5
2.1 Graph Theory	5
2.2 Graph Neural Networks	7
2.3 Spatiotemporal Graph Neural Networks	9
3 State of the art	11
3.1 Early designs	11
3.2 Recent advancements	13
3.3 Current limitations	13
4 Industrial process monitoring	15
4.1 Monitoring objectives	15
4.2 Monitoring challenges	16
4.3 Process representation	18
4.4 Problem formulation	20
5 Proposed method	23
5.1 Overcoming heterogeneity	24
5.2 Input-affine system formulation	25
5.3 Forecasting Model	26
5.4 Training Objective	29
5.5 Anomaly Detection	29
5.6 Fault Isolation	31
5.7 Evaluation metrics	32

6	Experimental design	35
6.1	Case studies	35
6.2	System identification	36
6.3	Variance bounding	38
6.4	Baseline models	39
7	Results	41
7.1	Tennessee Eastman Process	41
7.2	Fluid Catalytic Craking Unit	45
8	Conclusion	51
	Bibliography	55
A	Notable neural networks	65
A.1	RNN layers	65
A.2	GNN layers	65
B	Embedding regularization	67
B.1	Variational regularization	67
B.2	Clustering regularization	67
C	Neural Differential Equations	69
D	Case studies	71
D.1	Tennessee Eastman Process	71
D.2	Fluid Catalytic Craking Unit	74
E	Experiments	81
E.1	Hyperparameter Tuning	81
E.2	Tennessee Eastman Process	82
E.3	Fluid Catalytic Craking Unit	90

List of Figures

Figure 2.1	Directed graph with five nodes.	5
Figure 2.2	MP mechanism on the graph of Figure 2.1, w.r.t. the central node.	7
Figure 4.3	Control system with both open-loop and closed-loop configurations. More precisely u is an endogenous MV, x is the CV, and d is an exogenous disturbance.	19
Figure 4.4	Example of multivariate time series with physical/causal relationships. The process presents 2 exogenous MVs, each influencing only one of the 5 CVs.	22
Figure 5.5	Proposed IA-STGNN for unsupervised FDI. In blue, the healthy model inference steps are illustrated; in green, the calibration of the detection thresholds is shown; in red, the AD stages are depicted; and in orange, the post-alarm fault isolation stage is presented.	24
Figure 7.6	TEP diagnosis for fault 4.	44
Figure 7.7	TEP diagnosis for fault 14.	44
Figure 7.8	FCCU diagnosis for fault <code>deltaP_increase</code>	46
Figure 7.9	FCCU diagnosis for fault <code>Uaf_decrease</code>	47
Figure 7.10	FCCU diagnosis for fault <code>CAB_valveLeak</code>	48
Figure 7.11	FCCU diagnosis for fault <code>condEff_decrease</code>	49
Figure 7.12	FCCU diagnosis for fault <code>Fhn_sensorDrift</code>	49
Figure D.13	P&ID diagram of the TEP case study (Li et al. 2011).	71
Figure D.14	SNR values for the TEP case study. The horizontal line indicates the mean value of 0.9.	72
Figure D.15	R^2 values for the TEP case study. The horizontal line indicates the mean value of 46.61%.	73
Figure D.16	Training MSE for TEP through early stopping or max epochs. The dashed line indicates the lower bound given by explainable variance.	73
Figure D.17	P&ID diagram of the FCCU case study (Santander et al. 2022).	74

Figure D.18	Some of the anomalous variables impacted by the deltaP_increase fault within FCCU case study. Plots arranged hierarchically from root cause to effect.	75
Figure D.19	Some of the anomalous variables impacted by the UAf_decrease fault within FCCU case study. Plots arranged hierarchically from root cause to effect.	76
Figure D.20	Some of the anomalous variables impacted by the CAB_valveLeak fault within FCCU case study. Plots arranged hierarchically from root cause to effect.	76
Figure D.21	Some of the anomalous variables impacted by the condEff_decrease fault within FCCU case study. Plots arranged hierarchically from root cause to effect.	77
Figure D.22	Some of the anomalous variables impacted by the Fhn_sensorDrift fault within FCCU case study. Plots arranged hierarchically from root cause to effect.	77
Figure D.23	System identification of the FCCU case study via Lasso regualrized SVAR. The two highlighted squares indicate the FCC unit and the fractionator, respectively, showing how well the model captures the physical structure of the plant.	78
Figure D.24	SNR values for the FCCU case study. The horizontal line indicates the mean value of 5.47.	79
Figure D.25	R^2 values for the FCCU case study. The horizontal line indicates the mean value of 72.14%.	79
Figure D.26	Training MSE for FCCU through early stopping or max epochs. The dashed line indicates the lower bound given by explainable variance.	79
Figure E.27	TEP diagnosis for fault 1.	87
Figure E.28	TEP diagnosis for fault 2.	87
Figure E.29	TEP diagnosis for fault 4.	87
Figure E.30	TEP diagnosis for fault 5.	87
Figure E.31	TEP diagnosis for fault 6.	88
Figure E.32	TEP diagnosis for fault 7.	88
Figure E.33	TEP diagnosis for fault 8.	88
Figure E.34	TEP diagnosis for fault 10.	88
Figure E.35	TEP diagnosis for fault 11.	89
Figure E.36	TEP diagnosis for fault 12.	89
Figure E.37	TEP diagnosis for fault 13.	89
Figure E.38	TEP diagnosis for fault 14.	89

List of Tables

Table 6.1	Summary of baseline models used for comparison, including the task they are optimized for: forecasting (F), reconstruction (R), or both (F+R).	39
Table 7.2	TEP overall AD metrics across all faults. Best values in bold, second best underlined.	42
Table 7.3	FCCU overall AD metrics across all faults. IA-STGNN results are reported for experiments with either the physical (P) or system-identification (SVAR) connectivity matrix. Best values in bold, second best underlined.	46
Table 7.4	FCCU AD metrics across all faults for different prediction horizons (H). The connectivity matrix used is the physical one. Best values in bold, second best underlined.	46
Table D.5	Faulty scenarios with known causes of the TEP case study. . .	72
Table D.6	Closed-loop controllers of the FCCU case study.	74
Table D.7	Faulty scenarios of the FCCU case study.	75
Table E.8	Hyperparameter tuning for all models, with best values reported for each dataset. For IA-STGNN, the regularization strength of the FDI stage was calibrated on the validation partition, such that deviations remained within a reasonably small range from nominal values.	81
Table E.9	TEP AD metrics for fault 1. Best values in bold, second best underlined.	82
Table E.10	TEP AD metrics for fault 2. Best values in bold, second best underlined.	82
Table E.11	TEP AD metrics for fault 3. Best values in bold, second best underlined.	82
Table E.12	TEP AD metrics for fault 4. Best values in bold, second best underlined.	83
Table E.13	TEP AD metrics for fault 5. Best values in bold, second best underlined.	83
Table E.14	TEP AD metrics for fault 6. Best values in bold, second best underlined.	83

Table E.15	TEP AD metrics for fault 7. Best values in bold, second best underlined.	84
Table E.16	TEP AD metrics for fault 8. Best values in bold, second best underlined.	84
Table E.17	TEP AD metrics for fault 9. Best values in bold, second best underlined.	84
Table E.18	TEP AD metrics for fault 10. Best values in bold, second best underlined.	85
Table E.19	TEP AD metrics for fault 11. Best values in bold, second best underlined.	85
Table E.20	TEP AD metrics for fault 12. Best values in bold, second best underlined.	85
Table E.21	TEP AD metrics for fault 13. Best values in bold, second best underlined.	86
Table E.22	TEP AD metrics for fault 14. Best values in bold, second best underlined.	86
Table E.23	TEP AD metrics for fault 15. Best values in bold, second best underlined.	86
Table E.24	FCCU AD metrics for fault <code>CAB_valveLeak</code> . Best values in bold, second best underlined.	90
Table E.25	FCCU AD metrics for fault <code>Fhn_sensorDrift</code> . Best values in bold, second best underlined.	90
Table E.26	FCCU AD metrics for fault <code>UAf_decrease</code> . Best values in bold, second best underlined.	90
Table E.27	FCCU AD metrics for fault <code>condEff_decrease</code> . Best values in bold, second best underlined.	91
Table E.28	FCCU AD metrics for fault <code>deltaP_increase</code> . Best values in bold, second best underlined.	91

Acronyms

AD	Anomaly Detection
AUC	Area Under the Curve
CDE	Controlled Differential Equation
CNN	Convolutional Neural Network
CTTG	Continuous-Time Temporal Graphs
CV	Controlled variable
DCRNN	Diffusion Convolutional RNN
DL	Deep Learning
DTTG	Discrete-Time Temporal Graphs
DyEdgeGAT	Dynamic Edge via Graph Attention Network
EWMA	Exponentially Weighted Moving Average
FCCU	Fluid Catalytic Cracking Unit
FDI	Fault Detection and Isolation
FN	False Negatives
FP	False Positives
GAT	Graph Attention Network
GCN	Graph Convolutional Network
GCRNN	Graph Convolutional RNN
GDN	Graph Deviation Network
GIN	Graph Isomorphism Network
GNN	Graph Neural Network
GRU	Gated Recurrent Unit
HN	heavy naphtha
IA-STGNN	Input-Affine Spatiotemporal Graph Neural Network
IIoT	Industrial Internet of Things
IQR	Interquartile Range
LSTM	Long Short-Term Memory
ML	Machine Learning

MLP	Multi-Layer Perceptron
MP	Message-Passing
MSE	Mean Squared Error
MTAD-GAT	Multivariate Time-series Anomaly Detection via Graph Attention Network
MTS	Multivariate Time Series
MV	Manipulated variable
NCDE	Neural CDE
NODE	Neural ODE
ODE	Ordinary Differential Equation
P&ID	Piping and Instrumentation Diagram
PCA	Principal Component Analysis
PCMCi	Peter and Clark Momentary Conditional Independence
PID	Proportional-Integral-Derivative
RNN	Recurrent Neural Network
ROC	Receiver Operating Characteristic
SMA	Simple Moving Average
SNR	Signal-to-Noise Ratio
SP	Setpoint
SSM	State-Space Model
STGNN	Spatiotemporal Graph Neural Network
SVAR	Structural VAR
TEP	Tennessee Eastman Process
TN	True Negatives
TP	True Positives
VAR	Vector Autoregression

Chapter 1

Introduction

Dynamical systems describe the evolution of an entity state through time. Given the large spectrum of applications, their study has been of paramount importance in a variety of fields such as mathematics, physics, biology, chemistry, economics, and engineering (Gandolfo 1971; Zill and Cullen 2006; T. Jackson, A. Radunskaya 2015). In the last decades particularly, the growing complexity of both consumer-grade and bleeding-edge engineering devices has further motivated the relevance of dynamical systems as a fundamental tool to model, maintain, and control such technologies. More precisely, condition monitoring plays a crucial role in ensuring safety, reliability, and efficiency across modern industrial facilities, where Fault Detection and Isolation (FDI) serves as a primary defense by identifying abnormalities and diagnosing root causes (Chandola et al. 2009). This highly specialized activity is of great interest in the industrial sector, where failures can lead to catastrophic consequences, including environmental disasters, financial losses, and even human casualties. Related to this, Anomaly Detection (AD) is the more general exercise of recognizing deviations from expected behavior, without investigating whether they are indicative of an underlying issue or just an innocuous outlier.

Given the immense scale and high dimensionality of modern facilities, these tasks have become non-trivial and frequently render traditional analytical methods inadequate or entirely intractable. Supported by the Industrial Internet of Things (IIoT) diffusion of deployed sensors and measurements availability, data-driven approaches have been largely adopted as an alternative, with Machine Learning (ML) techniques especially showing promise in this area. However, classical models often ignore the interconnectedness between subsystems of a larger whole, which has been broadly recognized as a key factor to understand the underlying mechanisms within industrial settings (Kazemi et al. 2020). In fact, internal interactions often give rise to complex emergent phenomena that cannot be explained by analyzing components in

isolation. Consequently, modeling these systems requires accounting for an intricate web of spatial and temporal dependencies.

Recently, Graph Neural Networks (GNNs) have emerged as great competitors to conventional frameworks in a wide range of domains, such as financial market analysis, traffic flow optimization, weather forecasting, and infectious disease modeling (Jin et al. 2024). Given the natural representation of factory plants as graphs, where nodes correspond to process variables and edges represent physical or statistical relationships, these architectures are a promising candidate to model the complex interaction networks of industrial systems and perform FDI activities. However, the exploration in this context is still in its infancy, soliciting further research to fully exploit the potential of these techniques and properly address the unique challenges of the industrial domain (Jia et al. 2025). In fact, monitoring these facilities is an exceptionally arduous task, as they are subject to multi-modal operating regimes, dynamic and non-linear interdependencies, delayed propagation effects, noise artifacts, external disturbances, and heterogeneous physical reactions, all of which have been considerably overlooked in the literature. Additionally, industrial plants are inherently closed-loop systems, where feedback controllers actively compensate for faulty behaviors, possibly masking the underlying failure until the control authority saturates (Sun et al. 2019). This control action distorts the apparent causal directionality, with both standard and graph-based approaches struggling to determine whether an actuator’s intense adjustment is the root cause of a disturbance or merely a corrective reaction to an unobserved process fault. Moreover, graph-based techniques often ignore prior knowledge about the system’s physical structure, which can be of great value in guiding the learning process and enhancing interpretability. Strictly related to this, the proposed static graph topologies cannot naturally capture controller-induced phase shifts, nonlinear operating modality changes, or interdependencies evolving over time, leading to high false-alarm rates or delayed detections.

From a methodological perspective, a pronounced gap persists in the domain of graph-based unsupervised FDI for IIoT applications. Contemporary unsupervised architectures primarily address the AD task, predicting or reconstructing nominal operating profiles and triggering binary alarms when deviations exceed predetermined thresholds. When fault localization is attempted, these models predominantly rely on heuristic post-hoc attributions, such as ranking individual node residuals or visualizing raw attention weights. Conversely, high-precision fault isolation methodologies in the literature almost universally formulate the task as supervised multi-class classification, requiring extensive historical catalogs of labeled fault instances that are rarely available in

real-world operational environments. Consequently, there remains a critical need for unified, unsupervised graph frameworks capable of diagnosing faults without relying on labeled failure data.

In view of these challenges, this thesis investigates the integration of control variables into graph-based models, enabling the accurate identification of root causes in closed-loop industrial systems. More specifically, our work is guided by the following research questions.

(RQ1) System Modeling. How can spatial topology and temporal sequence modeling be integrated into a graph-based framework to account for dynamic interactions between process variables and lagged propagation effects, while minimizing AD delay and maintaining high discriminative precision in closed-loop industrial regimes?

(RQ2) Interpretable, Unlabeled FDI. How can an unsupervised architecture transition from global AD to interpretable structural fault isolation without access to historical labeled fault catalogs, while correctly disambiguating between root causes and secondary propagation effects?

(RQ3) Topological Relevance. What is the contribution of physical prior knowledge to the performance, and how can the proposed framework maintain high accuracy and robustness when such information is unavailable, requiring graph connectivity to be inferred purely from data-driven system identification?

We propose a novel framework that addresses the aforementioned challenges via the core contributions:

- **Input-Affine Spatiotemporal GNN** - The model explicitly incorporates manipulated variables into a spatiotemporal graph-based architecture through a dedicated linear channel, allowing the strict separation of control action effects from autonomous dynamics.
- **Unsupervised Sparse Parameter Divergence Estimation** - After reparameterization, a second-stage optimization enables unsupervised FDI, producing physically meaningful structural deviations from nominal behavior, such as sensor drifts, influence scaling, and input gain shifts.
- **Topological Agnosticism** - Through evaluation across both physical schematic-derived connectivity and data-driven linear system identification, we showcase the model’s effectiveness in handling complex industrial scenarios even in the absence of exact domain-specific engineering knowledge.

In summary, this work introduces a principled approach for incorporating control variables into graph-based models, and performing unsupervised FDI

in closed-loop industrial systems. In particular, we demonstrate that the input-affine formulation enables actuator failures to be accurately identified as parametric gain shifts, while also improving dynamic modeling accuracy. The proposed framework is evaluated on representative case studies, and significantly outperforms baseline methods in AD and fault isolation under closed-loop control, independently of the availability of prior knowledge about the system's physical structure.

Chapter 2

Background

Before delving into the specifics of industrial process monitoring, it is important to provide a clear definition of the fundamental concepts behind the techniques employed in this work. After a brief overview of graph theory, this chapter introduces the basic principles of standard and spatiotemporal GNNs, which are the core building blocks of the proposed methodology.

2.1 Graph Theory

Graphs are mathematical structures that consist of nodes and edges connecting them. Each node can represent an entity or a data point, while edges represent the relationships or interactions in-between. Formally, a graph can be defined as:

$$\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathbf{X}^V, \mathbf{X}^E), \quad (2.1)$$

where $\mathcal{V}$ is the set of nodes, $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ is the set of edges, while $\mathbf{X}^V \in \mathbb{R}^{|\mathcal{V}| \times d_v}$ and $\mathbf{X}^E \in \mathbb{R}^{|\mathcal{E}| \times d_e}$ are the possibly empty node and edge feature matrices, respectively. By denoting an edge going from node $i \in \mathcal{V}$ to node $j \in \mathcal{V}$ as $(i, j) \in \mathcal{E}$, we can distinguish between *directed* graphs, where edges have a specific direction, and *undirected* graphs, where relationships are symmetric, i.e., $(i, j) \in \mathcal{E} \implies (j, i) \in \mathcal{E}$. To monitor the connectivity, graphs are characterized by an adjacency matrix $\mathbf{A} \in \mathbb{R}^{|\mathcal{V}| \times |\mathcal{V}|}$, where $A_{ij} = 1$ if $(i, j) \in \mathcal{E}$, $A_{ij} = 0$ otherwise. Figure 2.1 shows a simple example to visualize these concepts.

Graphs are especially useful for depicting complex systems, where the relationships between entities are crucial for understanding the overall behavior, such as social networks, recommender systems, and causal inference. However, this static representation lacks the ability to capture the temporal evolution

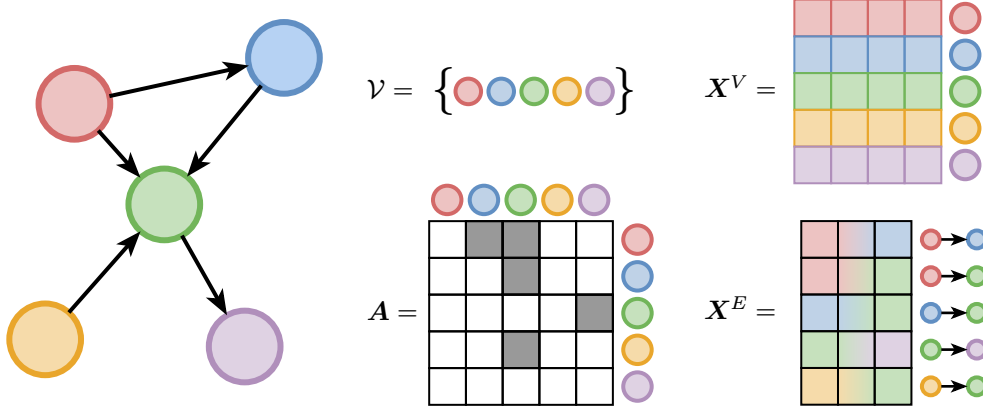


Figure 2.1: Directed graph with five nodes.

of the system, which is crucial in many real-world applications (Kazemi et al. 2020).

Temporal graphs, also known as dynamic, extend the original mathematical concept to a scenario where features and/or topology change over time, resulting in an evolving structure: nodes and edges can be added, removed, or modified. Formally, a temporal graph at instant t can be defined as $\mathcal{G}_t = (\mathcal{V}_t, \mathcal{E}_t, \mathbf{X}_t^V, \mathbf{X}_t^E)$, with structure sets and feature matrices possibly changing in size and content as time progresses. Adjacency matrices can be defined accordingly as $\mathbf{A}_t \in \mathbb{R}^{|\mathcal{V}| \times |\mathcal{V}|}$, with $\mathcal{V} = \bigcup_t \mathcal{V}_t$ being the set of all possible nodes. Notably, a distinction is usually made between Continuous-Time Temporal Graphs (CTTG) and Discrete-Time Temporal Graphs (DTTG), depending on the nature of the temporal dimension (Feng et al. 2026). The first is usually represented as a sequence of events. The discrete-time case, instead, can be differentiated into *snapshotted* and *aggregated* representations: the former consists of a sequence of static graphs, each representing the state of the system at a specific time step; on the other hand, the latter aggregates information over a time window, resulting in a single graph that captures the temporal evolution within (Gao and Ribeiro 2023).

The definition of temporal graph encompasses a wide range of real-world scenarios. For example, in a traffic network, nodes can represent intersections or road segments, while edges represent the flow of traffic between them. The structure and features of the graph can change over time due to various factors, such as traffic congestion, accidents, or road closures (Jiang et al. 2023).

2.2 Graph Neural Networks

Graph Neural Networks (GNNs) (Gori et al. 2005; Scarselli et al. 2009) are a class of ML techniques designed to process data represented as static graphs. The defining feature¹ is the neural Message-Passing (MP) mechanism, which allows the model to learn and propagate information across the graph structure, thus embedding a topological inductive bias into the processing. Specifically, the following notion of node neighborhood is derived from the adjacency matrix:

$$\mathcal{N}(i) = \{j \in \mathcal{V} : (j, i) \in \mathcal{E}\}. \quad (2.2)$$

This definition separates the receiver (or target) node i from its senders (or sources) j , which basically have an edge pointing directly towards it. Nodes undergoing a MP layer exchange information with their neighbors and update their representation accordingly. This process typically involves three main steps: message generation, aggregation, and node update. Firstly, each node generates a message based on its current state and, optionally, the states of its neighbors. Then, the messages from nodes in a common neighborhood are aggregated using a permutation invariant function. Finally, each node updates its representation based on the aggregated messages and its previous state. Considering node i 's initial hidden representation $\mathbf{h}^{i,0}$ and its neighbors j associated with edge features $\mathbf{e}^{ji}$, at iteration l these steps can be formulated as follows:

$$\begin{aligned} \mathbf{m}^{i,l} &= \text{AGGR}_{j \in \mathcal{N}(i)} \{ \text{MSG}^l(\mathbf{h}^{i,l}, \mathbf{h}^{j,l}, \mathbf{e}^{ji}) \}, \\ \mathbf{h}^{i,l+1} &= \text{UPD}^l(\mathbf{h}^{i,l}, \mathbf{m}^{i,l}), \end{aligned} \quad (2.3)$$

where $\text{UPD}^l(\cdot)$ and $\text{MSG}^l(\cdot)$ are arbitrary differentiable functions, e.g., MLPs. After L layers, the final node representations $\mathbf{h}^{i,L}$ incorporate information from their L -hop neighborhood, thus capturing the local structure and features of the graph. Figure 2.2 illustrates an example of MP mechanism.

The MP design is inherently flexible, and can be implemented using various architectures and techniques, such as attention mechanisms, recurrent units, and convolutional operations (Appendix A.2). We can further distinguish between *isotropic* message generation, when the function depends on the source node only, as opposed to *anisotropic*, where both sender and receiver influence this step (Dwivedi et al. 2023). In consequence, the MP mechanism

¹This applies to spatial GNNs, in contrast with spectral GNNs, which are outside the scope of this work.

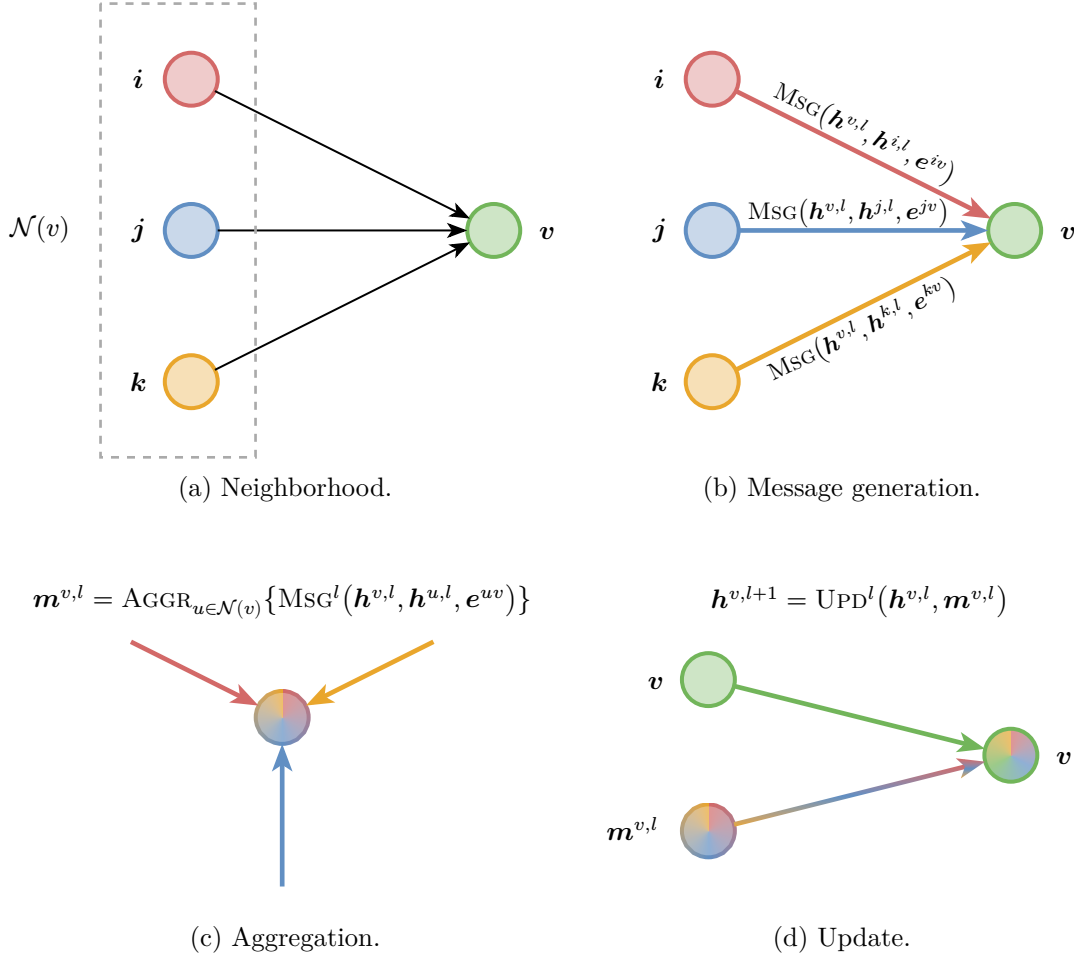


Figure 2.2: MP mechanism on the graph of Figure 2.1, w.r.t. the central node.

adaptability enables GNNs to capture nonlinear relationships and dependencies between nodes, making them particularly effective for tasks such as node classification, link prediction, and graph generation (Zheng et al. 2024).

The competitive performance of GNNs, together with their explainability and interpretability, has encouraged their widespread adoption in various domains, including social networks, transportation systems, supply chains, and many more (Longa et al. 2023). Nonetheless, GNNs neglect of the temporal dimension has led to the emergence of Dynamic GNNs (Rossi et al. 2020), operating on event-based and continuous-time dynamics, and Spatiotemporal GNNs (Corradini et al. 2025), accounting simultaneously for transitions and spatial interaction, thus extending applications to time-varying data and dynamic graphs.

2.3 Spatiotemporal Graph Neural Networks

Spatiotemporal Graph Neural Networks (STGNNs) are a class of GNNs suited to capture the evolving patterns and dependencies across both space and time, making them especially useful for handling temporal graphs. The MP mechanism is adapted to account for the temporal dimension: previous notation can be easily extended to the spatiotemporal setting by considering node representations $\mathbf{h}_t^{i,l}$ at time step t and iteration l , and defining the neighborhood $\mathcal{N}_t(i) = \{j \in \mathcal{V}_t : (j, i) \in \mathcal{E}_t\}$ accordingly. The new generic formulation is as follows:

$$\begin{aligned} \mathbf{m}_{\leq t}^{i,l} &= \text{AGGR}_{j \in \mathcal{N}_t(i)} \left\{ \text{MSG}^l \left(\mathbf{h}_{\leq t}^{i,l}, \mathbf{h}_{\leq t}^{j,l}, \mathbf{e}_{\leq t}^{ji} \right) \right\}, \\ \mathbf{h}_t^{i,l+1} &= \text{UPD}^l \left(\mathbf{h}_{\leq t}^{i,l}, \mathbf{m}_{\leq t}^{i,l} \right), \end{aligned} \quad (2.4)$$

with information aggregation from both spatial and temporal dimensions.

The flexibility of STGNNs allows for various implementations and adaptations to specific application scenarios, making them a powerful tool for modeling complex spatiotemporal data. However, the choice of the spatiotemporal MP layer is strictly dependent on the specific data characteristics and task requirements, as it can significantly impact the model’s performance and interpretability, thus requiring careful consideration and experimentation. Specifically, various designs have been proposed to implement this mechanism: in literature, the most common categorization identifies the *time-and-graph*, *time-then-graph*, and *graph-then-time* frameworks (Gao and Ribeiro 2023; Cini et al. 2025).

Time-and-graph. This setup implies that spatiotemporal processing cannot be factorized in separate steps, as the model learns a joint representation by simultaneously evaluating these information. This is typically achieved by interleaving spatial and temporal operations within the same layer or block, allowing for a more integrated learning of inter-node and intra-node dependencies. Formally, the time-and-graph representation of node $i \in \mathcal{V}_t$ can be abstracted as:

$$\mathbf{H}_t^i = \text{RNN} \left([\text{GNN}_{\text{in}}^L(\mathcal{G}_t, \mathbf{A}_t)]_i, [\text{GNN}_{\text{rc}}^L(\mathbf{H}_{t-1}, \mathbf{A}_t)]_i \right), \quad (2.5)$$

where the L -layer modules GNN_{in}^L and GNN_{rc}^L respectively encode each snapshot inputs and historical representations. The node states at the final time step are then used for downstream tasks.

Graph-then-time. This design factorizes the spatiotemporal MP into a spatial operation followed by a temporal one. In the first step, each node’s representation is updated based on its neighbors’ information at the current time step. Then, the final node representations are processed through a recurrent unit to capture the temporal evolution of each node’s state over time. This approach allows for a more explicit modeling of spatial dependencies before incorporating temporal dynamics, which can be beneficial in scenarios where inter-node relationships are more complex or critical. Formally:

$$\mathbf{H}_t^i = \text{RNN}\left([\text{GNN}_{\text{in}}^L(\mathcal{G}_t, \mathbf{A}_t)]_i, \mathbf{H}_{t-1}^i\right). \quad (2.6)$$

This notation highlights how graph-then-time models can be seen as a subset of the time-and-graph class, where GNN_{rc}^L of Equation (2.5) directly outputs previous node representations.

Time-then-graph. This configuration is the opposite of the previous one, where the temporal operation is applied first to capture a node’s historical evolution independently of its neighbors, thus providing final static representations on which the spatial MP layer later operates. This approach can be formalized as:

$$\mathbf{Z}_i = [\text{GNN}^L(\text{RNN}_{\text{node}}(\mathbf{X}_{\leq T}^V), \text{RNN}_{\text{edge}}(\mathbf{X}_{\leq T}^E)), \mathbf{A}_T]_i, \quad (2.7)$$

with node and edge sets being constant across time, and T representing the final time step.

Chapter 3

State of the art

In this chapter, we propose a brief review of the most recent literature on IIoT-oriented STGNNs to provide a more exhaustive understanding of the current landscape of their research, limitations, and potential for future advancements.

3.1 Early designs

The most common design in the early stages of STGNNs development was the time-and-graph approach. Specifically, initial implementations revolve around the generalization of classical Recurrent Neural Networks (RNNs), with standard fully connected layers in gate operations being replaced by graph convolutions (Appendix A.2). However, this family of so-called Graph Convolutional RNNs (GCRNNs) (Seo et al. 2018) is inherently limited by the flaws of the parent architectures, such as over-smoothing and over-squashing issues.

These vulnerabilities were partly mitigated by replacing the underlying graph operations. For instance, Diffusion Convolutional RNN (DCRNN) (Li et al. 2018) addresses previous challenges with the introduction of a diffusion convolutional layer that uses bidirectional random walks on a directed graph to model the spatial dependencies. Concurrently, temporal dynamics are captured by an encoder-decoder structure, implemented by GRUs (Appendix A.1) with diffusion convolutions in substitution for matrix multiplications. The resulting sequence-to-sequence architecture (Sutskever et al. 2014) is able to capture spatiotemporal dependencies among time series and can be applied to various spatiotemporal forecasting problems. Nonetheless, this framework relies on static topologies, restraining its applicability to the traffic forecasting tasks it was originally designed for.

Multivariate Time-series Anomaly Detection via Graph Attention Network (MTAD-GAT) (Zhao et al. 2020) is the first model to tackle the problem of AD in industrial scenarios. By considering univariate time series as features, the model leverages two parallel GATs (Appendix A.2) to learn inter-features correlations and intra-features temporal dependencies at the same time. Additionally, training is performed via forecasting-based and reconstruction-based components, acting on a joint optimization objective. Thanks to the attention scores, this proposal is able to provide a certain level of interpretability, while being very capable in detecting anomalies. However, prior knowledge on feature correlations is ignored and the graph structure is assumed to be fully connected, leaving a margin for improvement.

Sparsity exploitation in this context has been introduced by Graph Deviation Network (GDN) (Deng and Hooi 2021), which falls under the graph-then-time paradigm. Each measurement is assumed to be dependent on a subset of other sensors, possibly filtered by prior domain knowledge; based on these candidate dependencies, the model computes the normalized pairwise products between sensor embeddings, and selects the k most relevant ones to build a sparse unweighted adjacency matrix. This method relies on the idea that learnable embeddings can capture the unique characteristics of individual sensors, highlighting the similarities and differences of the physical variables they represent. The core MP operation is a GAT applied to the sparse graph at each time step to capture the spatial dependencies, while a fully connected layer is used for forecasting. Finally, anomaly scores are calculated for each sensor and for each time step, allowing the identification of signal-specific anomalies via a thresholding mechanism. Regardless of its contributions, the model only captures spatial relations in the direct neighborhood of each node, and greatly disregards temporal dependencies within each univariate time series (Zheng et al. 2024).

Although these methods capture spatiotemporal dynamics within industrial telemetry streams, they generally assume that relationships between system variables remain static within a defined observation time period, thus neglecting the evolution of functional interdependencies and ignoring a potential indicator of process faults (Chen et al. 2023). Additionally, identifying whether these shifts are due to faults or novel unseen operating modes persists as a non-trivial task.

3.2 Recent advancements

A pivotal contribution to the graph-based architectures is represented by the formulation of the time-then-graph algorithm (Gao and Ribeiro 2023), which soon gained popularity thanks to its efficiency and scalability. The shift from a temporal to a static representation of the graph results in a significant reduction of the computational complexity, and has been a key enabler for the development of more expressive STGNNs. This approach has been widely adopted in subsequent works, and has led to the development of more sophisticated models, but it is not exempt from drawbacks.

Inspired by this paradigm, Dynamic Edge via Graph Attention Network (DyEdgeGAT) (Zhao and Fink 2024) addresses previous restrictions by considering operating conditions as contextual information during node dynamics extraction, and modeling pairwise relationship evolution via a dynamic edge construction mechanism. Furthermore, a temporal topology-informed anomaly scoring is proposed to identify both signal-specific and relationship-specific anomalies, thus improving the interpretability of the model. Despite the promising results, DyEdgeGAT presents several limitations. First, the conditioning context is homogeneous across all variables, and heavily relies on the availability of system-independent variables, which may not be always present in real-world scenarios. Second, the dynamic edge mechanism of this time-then-graph framework works under a full connectivity assumption and operates strictly on concurrent information, neglecting sparsity and delayed effects propagation.

3.3 Current limitations

In other domains, time-lagged relational dependencies in STGNNs have been actively investigated through diverse structural mechanisms. These include learnable delay kernels for latency in dynamic spiking networks (Wang et al. 2025), causal-lag-aware attention modules for hydrological runoff forecasting (Yuan and Yan 2026), message traverse layers that route information along multi-hop delayed paths in traffic flow prediction (Wu et al. 2024), and delay-aware graph neural stochastic differential equations for continuous financial modeling (You et al. 2026). In industrial process settings, however, exploration remains comparatively constrained, largely relying on static mutual information (Ji et al. 2021) or basic pairwise time-lagged correlation graphs (Feng et al. 2025).

Concurrently, closed-loop systems have been largely overlooked in the literature. While heterogeneous architecture designs have been proposed to address the challenges of capturing distinct, multi-modal analytical interactions between interconnected process units (Chen et al. 2023), and the issue of learning spurious correlations from confounder signals has been addressed via causal intervention techniques (Liu et al. 2024), existing work treats these relationships primarily as statistical bypasses rather than dynamic feedback controllers actively compensating for physical faults. A promising direction for closed-loop industrial systems can be found in recent adaptations of State-Space Models (SSMs) into graph learning, such as embedding linear state-space recurrences directly into message-passing frameworks (Ceni et al. 2025) or jointly learning continuous latent state transitions and dynamic relational structures end-to-end (Zambon et al. 2026). These explicit state-transition formulations naturally mirror physical dynamical systems, while their efficient sequence-recurrence capabilities allow them to retain long-range temporal context and track subtle, low-frequency state drifts that feedback controllers displace over extended horizons. However, their application to feedback-based fault diagnosis remains constrained by restrictive assumptions, as they largely rely on exogenous inputs and open-loop data-generating processes.

Finally, most existing GNNs frameworks target global AD rather than providing principled mechanisms for fault isolation and root-cause analysis. This scarcity of graph-based methods tailored specifically for unsupervised FDI in IIoT arises from both operational realities and methodological constraints. The industrial sector persistently relies on supervised classifiers to bypass the challenges of unlabeled data, where complex sensor interdependencies and low signal-to-noise ratios hinder precise subunit localization (Łuczak et al. 2023). Methodologically, the lack of explicit process topologies often forces models into unguided, joint optimization of graph structure and anomaly scoring, which degrades diagnostic accuracy and impairs component-level attribution (Xu et al. 2023; Zhao et al. 2024).

Chapter 4

Industrial process monitoring

In the modern era, industrial processing systems serve as the core foundational infrastructure driving global manufacturing, energy generation, chemical synthesis, and critical utility management. As these facilities scale up to meet global economic demands, industrial plants have evolved into massive and ultra-complex environments where thousands of heterogeneous units co-exist. These components operate in a state of highly interdependent equilibrium, where localized events can rapidly propagate across the entire system layout.

Within this context, the integration of cyber-physical systems, Artificial Intelligence, Big Data Analytics, and cloud computing into operations is driving the fourth industrial revolution, commonly referred to as Industry 4.0. More specifically, the widespread adoption of the IIoT has transformed these environments by embedding high-density sensor networks across every layer of production: this pervasive deployment generates an unprecedented amount of high-frequency Multivariate Time Series (MTS) data, providing continuous telemetry on any physical variable of interest.

Given the major value of these facilities, the ability to effectively monitor their health and performance is of great relevance. In this chapter, we initially provide a detailed overview of the objectives and challenges associated with industrial process monitoring. Then, we introduce a formal representation of a factory plant, and finally define the monitoring problem as a spatiotemporal forecasting task on a dynamic graph.

4.1 Monitoring objectives

The primary goal of industrial systems supervision is condition monitoring, which translates into the ability to anticipate the evolution of process variables and detect any deviations from expected behavior, while exposing underlying

faults. In this sense, real-time forecasting provides a prognostic window into the future trajectory of the process state, granting operators the foresight to preemptively mitigate structural drifts.

In this scenario, FDI focuses on the identification of unexpected behaviors and their root causes: it is a specialized task that goes beyond simply recognizing anomalous trends, as it reveals the presence of a fault and distinguishes its exact nature and origin. This enables targeted, efficient, and effective maintenance and repair actions, reducing downtime and minimizing the risk of recurring problems.

Concurrently, AD serves as a more general line of defense, recognizing subtle, out-of-distribution deviations and cross-sensor relational discrepancies before they can escalate into catastrophic equipment failures, hazardous environmental leaks, or multi-million-dollar unscheduled shutdowns. Anomalies can be classified as either: (i) point anomalies, which are individual data points that deviate significantly from the expected behavior; (ii) contextual anomalies, which are data points that may be normal in one context but anomalous in another (Song et al. 2007). This distinction is crucial for accurately identifying and addressing potential issues in complex industrial systems.

Both FDI and AD can be broadly categorized into two main approaches: (i) supervised, which relies on labeled data, where the model is trained to recognize known anomalies or faults based on historical examples, and results in an effective approach when sufficient training data is available, but it may struggle to detect novel or unforeseen scenarios; (ii) unsupervised, which does not require labels and instead focuses on learning the normal behavior of the system, by establishing a baseline of normal operations and identifying deviations that may indicate potential issues.

4.2 Monitoring challenges

Despite the abundance of data, effectively monitoring modern industrial facilities remains an exceptionally difficult task, highlighting the critical importance of robust methodologies. More precisely, we identify the key factors that contribute to the complexity of these systems:

- Multi-modal operating conditions - The system can operate under various modes (e.g., varying production capabilities, changing feedstock rates), which usually lead to different baseline behaviors for the same sensor

readings, drastically reducing the effectiveness of threshold-based alarms and static statistical models.

- **Data heterogeneity** - The sensor network is composed of heterogeneous units, each measuring distinct physical variables with possibly unique characteristics, such as sampling rates, dynamic ranges, and noise levels. In fact, industrial telemetry is inherently corrupted by high-frequency measurement noise, and distinguishing a genuine anomalous trend from a noisy spike or a benign operational disturbance requires sophisticated filtering that risks introducing phase lags.
- **Dynamic and nonlinear relationships** - The underlying processes can evolve over time due to operating environment transitions, equipment aging, or maintenance activities. This drift can cause the relationships between sensor readings to change, making it difficult for static models to maintain accuracy. At the same time, the complex interdependencies between sensors can lead to situations where individual readings appear normal, but their relationships with other units indicate an underlying issue.
- **Closed-loop feedbacks** - Industrial systems are often equipped with control loops that automatically adjust the process state to maintain desired performance. These feedback mechanisms can introduce additional complexity, as they can mask the effects of faults or anomalies, obscuring the true nature of the problem and making it harder to detect and diagnose.
- **Time-lags** - The effects of certain events in the system may not be immediately reflected in the entire sensor network, but rather propagate with a delay. Depending on their origin, these time-lags can be either: (i) physical, when the effect of a change in one part of the system takes time to manifest in another part; (ii) causal, if the relationship between two variables is not instantaneous but rather mediated by other factors or processes.

Supported by data availability, these interconnected challenges are being solved by applying Deep Learning (DL) techniques, due to their ability to learn complex patterns from large datasets. However, the intricate interdependencies of these systems are usually overlooked in traditional approaches, which often treat sensor streams as isolated channels and fail to capture the underlying interactions between them. This disregard of the process topology leads to suboptimal performance in monitoring tasks, and the need to properly account for system relationships has motivated the application of graph-based methods directly to MTS data. Rather than treating signals as independent features, the industrial process is modelled as a dynamic topological graph, where each sensor or actuator acts as a node, and the physical, causal, or learned functional dependencies between them serve as edges. Given these

premises, the ability to capture spatial and temporal factors makes STGNNs particularly competitive with respect to grid-based models (e.g., CNNs) and purely sequential models (e.g., LSTMs, Transformers). Nonetheless, the closed-loop shape of industrial plants still poses a significant challenge, as it can lead to situations where the internal state is not fully observable, and the relationships between variables are not straightforward to model. Blending spatial MP with temporal recurrence can inadvertently introduce non-causal dependencies or smear directional feedbacks across the network. Moreover, the presence of system-independent variables can further complicate the learning task, as they can introduce additional sources of variability and uncertainty that are not directly controlled by the process.

As the roles of system variables and their connections are of significant concern in this context, it is mandatory to introduce some terminology derived from control theory to better understand the process modeling.

4.3 Process representation

Factory plants are dynamic systems, with the internal state evolving over time according to past observations and external influences. More precisely, it is possible to model the process by characterizing the type of variables involved, their relationships, and the underlying dynamics governing their evolution. Based on their role in the system, we can broadly classify variables into two categories (Mankiw 2016):

- *Endogenous* - These are the system-dependent variables, which are directly influenced by the process dynamics and control actions. They represent the internal state and are the target of the monitoring objectives.
- *Exogenous* - These are the system-independent variables, which influence the process behavior without being directly controlled by it. They can include environmental factors, external inputs, disturbances, or other relevant measurements that may impact the system's performance.

A further distinction between variables can be made based on their role in control loops, which are one of the most important aspects of industrial processes, as they are used to adjust the internal state as needed. In accordance with control theory terminology, we label as follows:

- **Controlled variables (CVs)** - These are the elements of the process that are actively monitored and regulated to maintain desired performance. They are typically the primary variables of interest in the system, and their values

are compared to predefined Setpoints (SPs) to determine the necessary control actions.

- Manipulated variables (MVs) - These are the values that can be adjusted to influence the system's behavior. They are typically the actuators or control signals that affect the process dynamics and help maintain the appropriate internal state. If the system is operating in a closed-loop configuration, these variables are manipulated based on the feedback from CVs to maintain stability, thus making them endogenous. Otherwise, in an open-loop configuration, they are set based on predefined control strategies or external inputs, resulting in their classification as exogenous.

To have a better understanding of these concepts, Figure 4.3 provides a process example to better illustrate these concept. For instance, in a reactor cooling process, the cooling water flow is the endogenous MV. This is adjusted to maintain the reactor temperature, i.e., the CV, at the desired level via a feedback mechanism. The state is also subject to an exogenous input, such as the air temperature, which can affect the dynamics.

With these premises, the control system can be formulated as a set of nonlinear time-variant equations that describe the evolution of the internal state and the interactions between CVs and MVs (Khalil 2002; Zagorowska et al. 2020):

$$\begin{aligned}\dot{\mathbf{x}}_t &= f(t, \mathbf{x}_t, \mathbf{u}_t), \\ \mathbf{u}_t &= g(t, \mathbf{x}_t).\end{aligned}\tag{4.1}$$

For example, in Proportional-Integral-Derivative (PID) controllers, the control action is computed as a weighted sum of the current, past, and predicted future errors e between the CVs and their corresponding SPs, which is then used to adjust the endogenous MVs accordingly:

$$\mathbf{u}_t = K_p e_t + K_i \int_0^t e_\tau d\tau + K_d \frac{de_t}{dt},\tag{4.2}$$

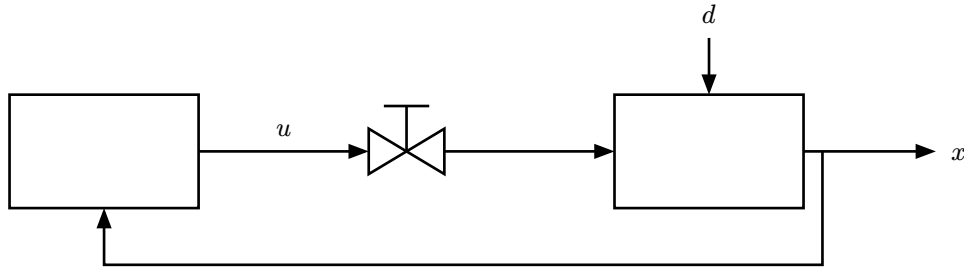


Figure 4.3: Control system with both open-loop and closed-loop configurations. More precisely u is an endogenous MV, x is the CV, and d is an exogenous disturbance.

with K_p, K_i, K_d being the proportional, integral, and derivative coefficients, respectively. Equation (4.1) encompasses a wide range of systems, including the linear state-space representations:

$$\begin{aligned}\dot{\mathbf{x}}_t &= \mathbf{A}_t \mathbf{x}_t + \mathbf{B}_t \mathbf{u}_t, \\ \mathbf{u}_t &= \mathbf{C}_t \mathbf{x}_t,\end{aligned}\tag{4.3}$$

where the matrices capture the dynamics and the relationships between variables. In detail, $\mathbf{A}_t$ describes the inter-CVs dependencies, $\mathbf{B}_t$ the effects of MVs on the internal state, and $\mathbf{C}_t$ the feedback loop.

Now that process variables have been appropriately defined, we can continue with the formalization of the learning problem, which establishes the foundation for the subsequent chapters and the proposed methodology.

4.4 Problem formulation

Consider a discrete MTS dataset of regularly and synchronously sampled measurements $\mathbf{X} \in \mathbb{R}^{N \times T \times d_x}$, where N is the number of sensors, T of time steps, and d_x of features per sensor. This matrix collects the temporal evolution of the CVs. Given the process at time $t \in [0, T)$, we denote the i -th sensor reading at time t as x_t^i , and the full endogenous state as $\mathbf{x}_t = [x_t^1, \dots, x_t^N]^T \in \mathbb{R}^N$. Concurrently, we assume $\mathbf{U} \in \mathbb{R}^{M \times T \times d_u}$ collecting MVs is available. For each of these variables, it is known whether it is endogenous or exogenous. It is worth noticing that we will restrict our analysis to the case of fully observed systems, where all sensor readings are homogeneous, available, reliable, and univariate, i.e., $d_x = 1$ and $d_u = 1$. This is not always the case in real-world scenarios, where missing data, sensor failures, or other sources of uncertainty further complicate measurement.

The exact dependencies between sensors (e.g., physical links, control loops, or any functional relationship) are known a priori only if the P&ID (Piping and Instrumentation Diagram) and domain-specific knowledge are fully available and accurate: not only is this rarely the case, but it risks resulting in a very sparse representation of the system structure, which is not necessarily optimal for current objectives. On the other hand, defining relationships based on statistical correlations or learned functional dependencies extends the notion of system topology and allows deriving the connectivity from the data, thus removing the need for prior information; however, it also might introduce a very dense representation, which can be computationally expensive and may

lead to overfitting if not properly regularized. We avoid limiting the proposal to a fixed relational structure, and encode inter-CVs relationships via $\mathbf{A} \in \mathbb{R}^{N \times N}$, where $A_{ij} = 1$ implies either direct physical connections or predictive causality *à la Granger* (Granger 1969) from sensor j to i depending on the scenario. Analogously, the MVs influence on the system can be described by $\mathbf{B} \in \mathbb{R}^{N \times M}$, capturing their individual contribution to the internal states, while closed-loop feedbacks are encompassed in $\mathbf{C} \in \mathbb{R}^{M \times N}$. As the previous matrix, these can be either derived or learned. Therefore, the model is delegated the dual responsibility of accounting for missing valuable connections and pruning spurious ones, which is a non-trivial task. Additionally, these static adjacency matrices capture only the presence but not the strength or evolution of dependencies: this is based on the idea that both theoretical and data-driven approaches can only partially provide reliable representations of the underlying system topology, which is usually not fully observable. This allows for a flexible modeling but also constitutes a strong limitation and potential source of error, further complicating the learning task.

With this configuration, the process at time t can be modeled as a temporal graph $\mathcal{G} = \langle \mathbf{x}_t, \mathbf{u}_t, \mathbf{A}, \mathbf{B}, \mathbf{C} \rangle$, where each variable is represented as a node, and their interactions as directed edges. Figure 4.4 provides an illustration of this concept.

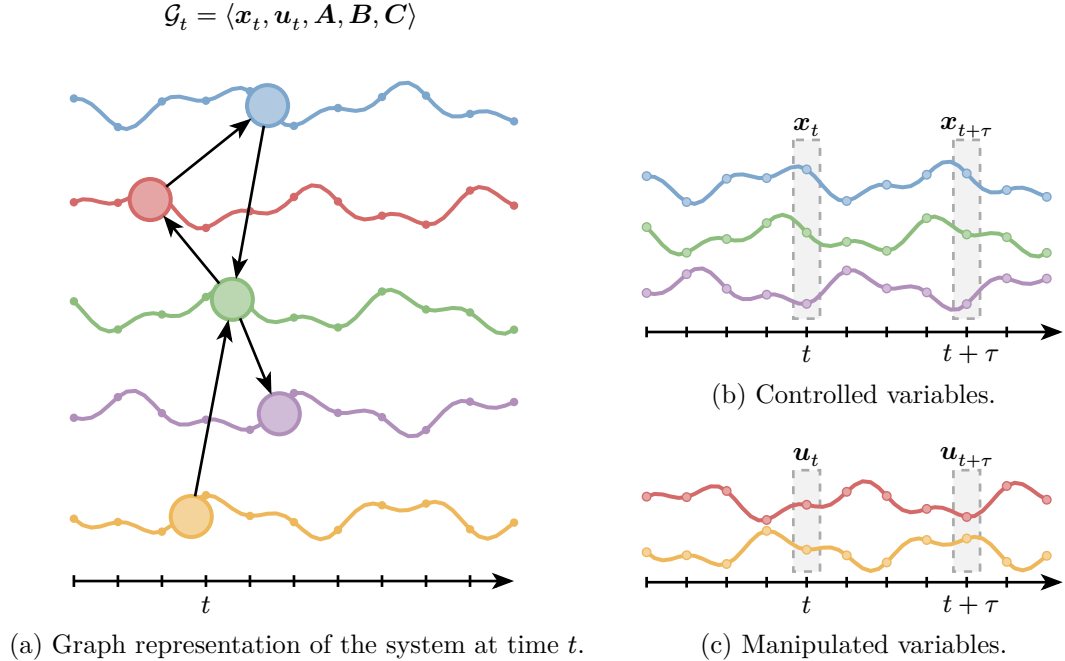


Figure 4.4: Example of multivariate time series with physical/causal relationships. The process presents 2 exogenous MVs, each influencing only one of the 5 CVs.

The objective of the monitoring problem is to learn a function $\mathcal{F}$ that maps historical internal states and exogenous factors to the future target endogenous variables:

$$\begin{aligned} x_{t+H}^i &= \mathcal{F}_{\theta}^{\text{CV}}(\mathbf{X}_{\leq t}, \mathbf{U}_{\leq t}, \mathbf{A}, \mathbf{B}) \quad \forall i, \\ u_{t+H}^i &= \mathcal{F}_{\theta}^{\text{MV}}(\mathbf{X}_{\leq t}, \mathbf{C}) \quad \forall i : \mathbf{u}^i \text{ is endogenous,} \end{aligned} \tag{4.4}$$

where H is the prediction horizon, and θ are the learnable parameters of the model. This function should be able to capture the complex spatiotemporal dependencies in the system, enabling accurate forecasting, effective AD, and reliable fault diagnosis.

Chapter 5

Proposed method

This chapter provides a detailed description of the proposed framework, including its formulation, components, and implementation details. The architecture, called Input-Affine Spatiotemporal Graph Neural Network (IA-STGNN), is optimized for forecasting on healthy MTS data from industrial processes, then performs unsupervised FDI via a two-stage approach, as illustrated in Figure 5.5.

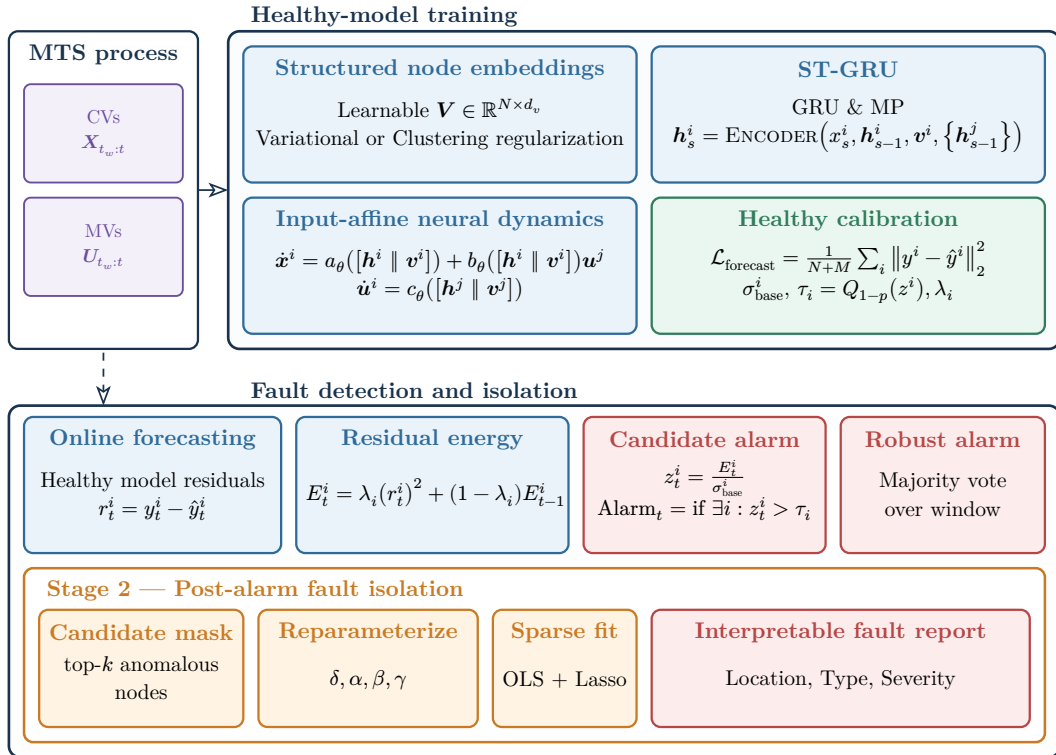


Figure 5.5: Proposed IA-STGNN for unsupervised FDI. In blue, the healthy model inference steps are illustrated; in green, the calibration of the detection thresholds is shown; in red, the AD stages are depicted; and in orange, the post-alarm fault isolation stage is presented.

From a high-level perspective, the architecture leverages structured node embeddings to account for local effects (Section 5.1), and it is designed to learn a global representation of the process dynamics through an input-affine formulation (Section 5.2). Precisely, node-specific latent representations are captured via a spatiotemporal GRU, which accounts for both intra-node and inter-node dependencies, and the evolution of the system is modeled by predicting the integrated displacements (Section 5.3). The training objective is the minimization of prediction error on healthy data (Section 5.4). After appropriate calibration on a validation partition, a threshold-based AD mechanism is employed to identify deviations from the expected behavior (Section 5.5), and a novel optimization on the reparameterized model is performed to isolate faults (Section 5.6).

5.1 Overcoming heterogeneity

When approaching the generic task of MTS forecasting, the safest route is to assume that each element of the set is generated by a different, independent, stochastic data-generating processes; thus, every variable should be modeled individually. In other words, predictions should be addressed as separate regression problems, where each time series' future is based on its own observed past. The resulting model estimates parameters independently for each variable: in consequence, it suffers from several issues, including sample size limitations, overfitting, complexity, and scalability. In literature, this univariate approach is referred to as *local*, in opposition to a *global* architecture, which shares parameters across all variables (Januschowski et al. 2020).

The application of this latter method instead overcomes previous limitations in terms of robustness, generalization, transferability, and efficiency. However, it also introduces the strong assumption that all time series are generated by the same underlying process, or they are at least related, which is not completely true in most real-world scenarios. This restriction has prevented the wide adoption of global frameworks. Nevertheless, recent formal advancements and competitive empirical performance have demonstrated that global models are not more restrictive than local architectures for time series forecasting in heterogeneous scenarios, i.e., when no assumption about similarity is set. On the contrary, it has been shown that each forecast that can be expressed by a local algorithm can also be expressed by a global algorithm (Montero-Manso and Hyndman 2021).

These models are not completely immune to the limitation of the globality assumption, and can gain from the integration of local information. Specifically, individual effects can be accounted for by introducing local components: that is, sub-modules that are specific to each variable, and that can be learned independently from the global model (e.g., node-specific encoders or decoders). Despite its popularity, this hybrid approach risks increasing complexity and specialization, thus negating the benefits of global solutions. A valid alternative to the integration of local information relies on the interpretation of learned node embeddings as a mechanism to amortize the consideration of local effects, rather than treating them as simple positional encodings. Consequently, the need for additional variable-specific processing layers is avoided and computational infeasibility is mitigated. Furthermore, the idea of giving structure to the embedding space to provide an effective regularization has recently surfaced in literature (Cini et al. 2023; Butera et al. 2024), thus allowing the emergence of similarities between time series.

In this broader context, STGNNs fall within the global class, as they leverage graph structures and relational inductive biases to learn a single, unified model. Consequently, we denote the learned node embeddings matrix $\mathbf{V} \in \mathbb{R}^{N \times d_v}$, which can be easily fed to the model at our discretion. Following the original work, the embedding field is structured via two regularization techniques: *variational* and *clustering* (Appendix B).

5.2 Input-affine system formulation

Given the multiple challenges that arise in predictive monitoring (Section 4.2), special attention is given to the robustness capabilities of the proposed framework. Since industrial processes are fundamentally governed by conservation laws and continuous time differential equations, we align the forecasting objective to infer the underlying dynamics, rather than simply learning a mapping from past to future observations. Specifically, we model the evolution of the system’s state variables by predicting their cumulative changes. This approach acts as an inductive bias, constraining the model to learn a more physically consistent representation of the process. Therefore, the model is well suited to handle steady-state equilibrium, out-of-distribution SPs during transitional phases, and robust single and multiple-step-ahead extrapolation, while retaining less sensitivity to noise and outliers.

Furthermore, we require a methodology capable of operating reliably in closed-loop systems, where automated feedback adjusts actuator positions,

inadvertently masking process faults. Consequently, we inject controls exclusively through an explicit channel during state integration, ensuring that these signals cannot be utilized by the network to artificially cancel out fault signatures, strictly decoupling the automated actions from the system’s true degradation and preserving the reliability of the diagnostic residuals.

Following these premises, an appropriate formulation of the system is necessary. Inspired by control theory, we model the process as an input-affine system, which is a class of dynamical nonlinear systems where the evolution of the state variables is linearly influenced by control inputs (Elkin 1998; Clelland 2009):

$$\begin{aligned}\dot{\mathbf{x}}_t &= \mathbf{a}(\mathbf{x}_t) + \mathbf{b}(\mathbf{x}_t)\mathbf{u}_t, \\ \dot{\mathbf{u}}_t &= \mathbf{c}(\mathbf{x}_t),\end{aligned}\tag{5.1}$$

where $\mathbf{a}$, $\mathbf{b}$, and $\mathbf{c}$ are nonlinear functions. This generic design is coherent with industrial processes, as they represent the standard paradigms for a vast majority of real-world control-applications (Henson and Seborg 1996). More precisely, a dynamic extension is applied, treating endogenous MVs as augmented states, as required by Section 4.4, and assigning the control law to the derivative of $\mathbf{u}$. This notation reflects the continuous and smooth behavior of actuators.

With the high-level formulation established, we can now provide a more detailed description of the several components that constitute the proposed framework.

5.3 Forecasting Model

The model assumes healthy operating condition data for training and validation. During testing on unseen scenarios, the architecture is expected to correctly forecast the evolution of the process, while allowing for the detection of anomalies and identification of faults. Equation (5.1) is reformulated to operate on node hidden states $\mathbf{H} \in \mathbb{R}^{N \times d_h}$ and learnable embeddings, predicting first-order differences of the system variables:

$$\begin{aligned}\dot{x}_t^i &= a_\theta([\mathbf{h}_t^i \parallel \mathbf{v}^i]) + b_\theta([\mathbf{h}_t^i \parallel \mathbf{v}^i])u_t^j \quad \forall j : B_{ji} = 1, \\ \dot{u}_t^i &= c_\theta([\mathbf{h}_t^j \parallel \mathbf{v}^j]) \quad \forall j : C_{ji} = 1.\end{aligned}\tag{5.2}$$

Note that the feedback edge constraint implicitly limits prediction of MVs to endogenous variables only. The functions are implemented as shared two-layer

MLPs with tanh activation functions, and they are responsible for modeling the local drift (a_θ), the input gain (b_θ), and the control law (c_θ) of the system, respectively. The concatenation of the node embedding $\mathbf{v}^i$ to the hidden state $\mathbf{h}_t^i$ allows the model to account for local effects, as explained in Section 5.1, while shared parameters maintain computational efficiency and generalization capabilities.

The encoding process is the core of the model, as it is responsible for capturing both intra-node and inter-node dependencies, thus accounting for dynamic, nonlinear, and delayed relationships. Operating via a sliding time-window $[t_w, t]$ of size W , the model learns:

$$\mathbf{h}_s^i = \text{ENCODER}\left(x_s^i, \mathbf{h}_{s-1}^i, \mathbf{v}^i, \{\mathbf{h}_{s-1}^j\}_{j \in \mathcal{N}(i)}\right) \in \mathbb{R}^{d_h} \quad \forall s \in [t_w, t], \quad (5.3)$$

with $\mathbf{h}_{t_w-1}^i$ set to $\mathbf{0}$. Specifically, this component is implemented as a spatiotemporal GRU, where spatial MP and temporal recurrence unfold iteratively for each time step. Specifically, messages are generated and aggregated as:

$$\begin{aligned} \tilde{\mathbf{h}}_{s-1}^i &= [\mathbf{h}_{s-1}^i \parallel \mathbf{v}^i] \in \mathbb{R}^{d_h+d_v}, \\ g_s^{ji} &= \sigma(\mathbf{W}_g [\tilde{\mathbf{h}}_{s-1}^i \parallel \tilde{\mathbf{h}}_{s-1}^j] + \mathbf{b}_g) \in (0, 1), \\ m_s^{ji} &= \tanh(\mathbf{W}_m [\tilde{\mathbf{h}}_{s-1}^i \parallel \tilde{\mathbf{h}}_{s-1}^j] + \mathbf{b}_m) \in (-1, 1), \\ M_s^i &= \sum_{j \in \mathcal{N}(i)} g_s^{ji} m_s^{ji}, \end{aligned} \quad (5.4)$$

where m_s^{ji} represents the message between two nodes, while g_s^{ji} weights the interaction appropriately, focusing on the most relevant information and disregarding misleading signals. The design of this latter operation as an unnormalized sigmoid gate is motivated by the need to modulate physical coupling strengths independently. In fact, conventional softmax attention enforces a strict sum-to-one constraint across a node's incoming connections: in a physical system undergoing degradation, the attenuation of a faulty edge's influence causes softmax to artificially inflate the weights of healthy siblings to preserve this normalization. This redistribution masks localized degradation and confounds causal attribution. By contrast, our formulation treats each edge as an autonomous transmission channel governed by g_s^{ji} , allowing a degraded connection to diminish in magnitude without corrupting the representation of adjacent couplings. This independence is crucial for fault diagnosis, as it preserves localized residual signatures and enables

unambiguous discrimination between node-level faults and structural edge-coupling failures. Once the aggregated message is computed, node updates are performed via the following modified GRU gate operations:

$$\begin{aligned}
 \mathbf{z}_s^i &= \sigma(\mathbf{W}_z^x x_s^i + \mathbf{W}_z^h \mathbf{h}_{s-1}^i + \mathbf{W}_z^m M_s^i + \mathbf{b}_z), \\
 \mathbf{r}_s^i &= \sigma(\mathbf{W}_r^x x_s^i + \mathbf{W}_r^h \mathbf{h}_{s-1}^i + \mathbf{W}_r^m M_s^i + \mathbf{b}_r), \\
 \mathbf{n}_s^i &= \tanh(\mathbf{W}_n^x x_s^i + \mathbf{r}_s^i \odot \mathbf{W}_n^h \mathbf{h}_{s-1}^i + \mathbf{W}_n^m M_s^i + \mathbf{b}_n), \\
 \mathbf{h}_s^i &= (\mathbf{1} - \mathbf{z}_s^i) \odot \mathbf{n}_s^i + \mathbf{z}_s^i \odot \mathbf{h}_{s-1}^i.
 \end{aligned} \tag{5.5}$$

While the formulation of Equation (5.4) performs aggregations on a single-hop neighborhood, incoming messages depend on previous hidden states. Therefore, as the recurrence unfolds, the effective spatial receptive field expands to the s -hop neighborhood, also capturing long-range dependencies. In this scenario, latent representations are delegated the task of capturing the individual node dynamics alongside neighbor interactions, where the recurrent GRU gates naturally account for temporal delays and transmission lags.

To extend the framework for multi-step forecasting over a future horizon H , the model transitions from the teacher-forced paradigm to an autonomous closed-loop rollout. For each future time-point $t_h \in [t + 1, \dots, t + H]$, the discrete time derivatives are computed from the learned vector fields and integrated via an explicit forward Euler scheme:

$$\begin{aligned}
 \hat{x}_{t_h}^i &= \hat{x}_{t_h-1}^i + a_\theta(\tilde{\mathbf{h}}_{t_h-1}^i) + b_\theta(\tilde{\mathbf{h}}_{t_h-1}^i) \hat{u}_{t_h-1}^j, \\
 \hat{u}_{t_h}^i &= \hat{u}_{t_h-1}^i + c_\theta(\tilde{\mathbf{h}}_{t_h-1}^j),
 \end{aligned} \tag{5.6}$$

where the recurrence is initialized with the last observations $\hat{x}_t^i = x_t^i$ and $\hat{u}_t^i = u_t^i$. At each step of the rollout, the predicted states are autoregressively fed back into the spatiotemporal recurrent unit to advance the latent representations. Equivalently, the integrated trajectory from the end of the observation window t to any horizon step t_h can be expressed as the sum of intermediate increments:

$$\begin{aligned}
 \Delta \hat{x}_{t_h}^i &= \hat{x}_{t_h}^i - x_t^i = \sum_{s=t+1}^{t_h} [a_\theta(\tilde{\mathbf{h}}_{s-1}^i) + b_\theta(\tilde{\mathbf{h}}_{s-1}^i) \hat{u}_{s-1}^j], \\
 \Delta \hat{u}_{t_h}^i &= \hat{u}_{t_h}^i - u_t^i = \sum_{s=t+1}^{t_h} c_\theta(\tilde{\mathbf{h}}_{s-1}^j).
 \end{aligned} \tag{5.7}$$

This approach can be interpreted as the discrete approximation of a neural differential equation (Appendix C), possibly controlled by exogenous factors through the input-affine channel.

5.4 Training Objective

The objective of the training phase is to minimize the prediction error on the healthy data, thus learning a representation of the process dynamics that is consistent with normal operating conditions. The loss function is defined as the Mean Squared Error (MSE) between the predicted and observed values, averaged over all nodes:

$$\mathcal{L} = \frac{1}{N + M} \sum \|y_t^i - \hat{y}_t^i\|_2^2, \quad (5.8)$$

with $\hat{y}_t^i$ and y_t^i being the predicted and observed values of the i -th variable integrated displacement at time t , respectively. This objective can be extended to include the additional regularization terms for the embedding space, as discussed in Appendix B.

5.5 Anomaly Detection

Once predictions on data with unknown fault status are available, the following step is to identify whether the observed behavior is consistent with the expected one. This is achieved via a threshold-based AD mechanism, which computes a residual score based on inferred and actual values, then compares it to an automatically calibrated value from normal operating conditions. In detail, raw residuals are calculated as:

$$r_t^i = y_t^i - \hat{y}_t^i. \quad (5.9)$$

After converting these residuals into energy values, thus accounting for the magnitude of the deviation, we apply a smoothing filter that increases the importance of recent observations and decays the influence of older ones, with the aim of improving robustness and avoiding false alarms due to noise. Specifically, we employ an Exponentially Weighted Moving Average (EWMA) with a node-specific smoothing factor λ_i :

$$E_t = \lambda_i (r_t^i)^2 + (1 - \lambda_i) E_{t-1}. \quad (5.10)$$

The smoothing factor is calibrated via the validation partition, more precisely it is a function of the autocorrelation of the original signals, which expresses the degree of temporal dependency in the data and the propensity of the process to remain in a given state. Formally, we define:

$$\lambda_i = \lambda_{\min} + (1 - \rho_1^i)(\lambda_{\max} - \lambda_{\min}), \quad (5.11)$$

where ρ_1^i is the first-order autocorrelation of the i -th variable, while $\lambda_{\min}$ and $\lambda_{\max}$ are bounds that define the range of possible values. Next, scaled z-scores are computed from EWMA values using baseline standard deviations calibrated with the validation operations:

$$\begin{aligned} \sigma_{\text{base}}^i &= \text{std}(E_t^i) \quad \forall t \text{ is healthy,} \\ z_t^i &= \frac{E_t^i}{\sigma_{\text{base}}^i}. \end{aligned} \quad (5.12)$$

By omitting the mean subtraction, this score represents the ratio of the current error energy to the typical fluctuations observed during normal conditions. Node-specific thresholds are then obtained using empirical quantiles of the validation z-scores:

$$\tau_i = Q_{1-p}(z_t^i) \quad \forall t \text{ is healthy,} \quad (5.13)$$

where p is a user-defined significance level, and Q_{1-p} is the $(1 - p)$ -th quantile of the fault-free distribution. From a formal hypothesis testing perspective, this p -value represents the probability of a false positive, i.e., the likelihood of incorrectly classifying a normal observation as anomalous. However, in contrast to the classical theoretical method, no assumption is made about the underlying distribution of the data, and the threshold is directly derived from the computed z-scores, thus providing a more robust and reliable estimate of the critical value. Finally, a binary decision is made by comparing the z-score to the respective calibrated thresholds:

$$\text{Alarm}_t = \begin{cases} 1 & \text{if } \exists i : z_t^i > \tau_i. \\ 0 & \text{otherwise} \end{cases} \quad (5.14)$$

To make detection more robust, we apply a majority voting scheme over a sliding window of the same size W used for forecasting, and raise an alarm only if the majority of the observations within the window are classified as anomalous.

5.6 Fault Isolation

After an anomaly is detected, the main objective is to identify the specific origin of the fault. More precisely, we freeze all the neural network weights, and reparameterize the architecture using fault-specific attributes accounting for various types of deviation from the nominal behavior, namely:

- Node drift δ_i - Additive deviation from the expected evolution of the i -th variable. It represents the most common type of fault, and it is usually associated with point anomalies.
- Edge scaling α_{ji} - Multiplicative deviation in the relationships between the j -th and i -th variables. It usually corresponds to contextual anomalies.
- Input gain β_i - Multiplicative deviation in the influence of the i -th action on the system dynamics. Depending on the specific input nature, i.e., endogenous or exogenous, it can be associated with either an actuator malfunction or a sensitivity change to external conditions.
- Control law gain γ_i - Additive deviation in the control strategy of the i -th actuator. It usually corresponds to an unintentional change in the control action.

The reparameterized model is then expressed as:

$$\begin{aligned} \dot{x}_t^i &= \text{MLP}_a(\mathbf{h}_t^i) + \delta_i + \beta_i \text{MLP}_b(\mathbf{h}_t^i) \mathbf{u}_t^j \quad \forall j : B_{ji} = 1, \\ \dot{\mathbf{u}}_t^i &= \text{MLP}_c(\mathbf{h}_t^j) + \gamma_i \quad \forall j : C_{ji} = 1, \end{aligned} \quad (5.15)$$

with the message generation modified accordingly:

$$M_s^i = \sum_{j \in \mathcal{N}(i)} \alpha_{ji} g_s^{ji} m_s^{ji} \quad \forall s \in [t_w, t]. \quad (5.16)$$

Considering the parameter roles in the system, nominal values during healthy operations are 0 for δ_i and γ_i , 1 for β_i and α_{ji} .

This novel linear model is then optimized via ordinary least squares on post-alarm observations, with the objective to minimize the prediction error on the faulty data, while keeping the original STGNN fixed. Specifically, a sliding window approach is employed, starting from the first anomalous observation and moving forward in time until significant deviations are detected, or until the end of the available data is reached. The problem is constrained by pre-selecting candidate variables that are likely to be responsible for the fault, reducing the search space to the smaller subset of parameters associated with them. First, a candidate mask is created using the node-specific z -scores, considering only the variables that crossed their calibrated threshold during the

current time window, thus stopping supposedly unrelated scalars from moving in a coordinated way that still reduces the residuals. Then, these candidates are ordered by the respective z -score to calibrated threshold ratio, and further pruned with top- k selection to prioritize the most anomalous ones. Since even a small number of nodes can still lead to a large amount of associated parameters, Lasso regularization (L_1) is applied to promote sparsity in the solution, thus favoring a more interpretable and parsimonious model. The final output of this stage provides the estimated fault-specific parameters, and their deviation from the nominal values can be used to quantify the location and severity of the fault, possibly uncovering its origin or current effect depending on the delay between its occurrence and the alarm time.

5.7 Evaluation metrics

Given a test instance, the model provides for each time step an anomaly score (i.e., the highest z -score across all variables), and the corresponding binary alarm decision. The latter is compared to the ground truth to compute True Positives (TP), False Positives (FP), True Negatives (TN), and False Negatives (FN), while the former can be used to generate threshold-independent measures. We evaluate AD performance using the following classical metrics.

Accuracy. It is the proportion of all classifications that are correctly identified:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}} \quad (5.17)$$

Recall. It is the proportion of effective positives that are correctly identified:

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad (5.18)$$

Precision. It is the proportion of predicted positives that are correctly identified:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \quad (5.19)$$

F1 score. It is the harmonic mean of recall and precision:

$$\text{F1} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (5.20)$$

Area Under the Curve (AUC). It is the area under the Receiver Operating Characteristic (ROC) curve, which plots the true positive rate against the false positive rate at various threshold settings.

Best Detection Delay (Delay*). It is the average time between the occurrence of a fault and its detection by the model, measured using the threshold that maximizes the F1 score.

Chapter 6

Experimental design

This chapter describes the experimental setup and the evaluation methodology employed to assess the performance of the proposed approach. First, we describe the case studies analyzed in this work, then we present the system identification methodology to learn the adjacency matrix from data, and we describe the variance bounding technique to estimate the theoretical predictive limits of our model. Finally, we introduce the baseline methods and evaluation metrics used for comparison.

6.1 Case studies

We consider two representative benchmark datasets of the chemical process industry.

6.1.1 Tennessee Eastman Process

Tennessee Eastman Process (TEP) (Rieth et al. 2017; 2018) is a widely used benchmark dataset for process monitoring and fault detection, which simulates the production plant of the Eastman Chemical Company (Downs and Vogel 1993). The system includes 5 main units (reactor, condenser, separator, stripper, compressor), processing 8 chemical components (named from A to H). The P&ID diagram of the process (Figure D.13) provides a comprehensive overview of the plant’s layout, allowing the realization of a physical adjacency matrix. Data is generated by simulating the plant under different normal operating conditions and various fault conditions, providing the opportunity to evaluate the performance of a model in both supervised and unsupervised settings. The case study contains 52 variables, including 41 sensor measurements (**XMEAS**) and 11 manipulated variables (**XMV**). There are 20 different faulty scenarios, of which 15 are known (Table D.5).

In our experiments, we use:

- Training - 4 healthy partitions of 500 samples each.
- Validation - 1 healthy partition of 500 samples.
- Testing - 5 healthy partitions of 500 samples each and 15 different known faulty partitions of 960 samples each. In these latter instances, the faults start at sample 160. Consequently, faulty samples account for 83.33% of the partition.

6.1.2 Fluid Catalytic Cracking Unit

Fluid Catalytic Cracking Unit (FCCU) (Santander et al. 2022; Kumar 2025) is a recently released benchmark dataset that simulates the operation of a hydrocarbon chemical plant. The system includes 2 main units, namely the FCCU (reactor plus regenerator), and the fractionator, processing a fresh feed into various byproducts. The P&ID diagram of the process (Figure D.17) provides a comprehensive overview of the plant’s layout, allowing the realization of a physical adjacency matrix. Furthermore, we employ the system identification methodology described in Section 6.2 to derive a learned adjacency matrix directly from the data. The case study contains 46 variables, including 3 exogenous inputs and 10 manipulated variables (Table D.6). There are 5 different faulty scenarios, all of which are known (Table D.7).

In our experiments, we use:

- Training - 1 healthy partition of 1612 samples with various operating conditions.
- Validation - 1 healthy partition of 404 samples with stable operating conditions.
- Testing - 5 different faulty partitions of 864 samples each. In these latter instances, the faults start at sample 576. In consequence, the percentage of faulty samples is 33.33%.

6.2 System identification

Given the nature of the problem, we explore a linear system identification approach, as this class of methods is widely used in industrial applications due to their simplicity, interpretability, and effectiveness in many scenarios. More precisely, we explore SVAR.

Vector Autoregression (VAR) is a statistical model that captures the relationships between variables in a MTS. Based on the lag order p , each quantity in the system is formulated as a linear function of its own past values, the

historical values of all other variables, and a stochastic error term. Formally, the VAR(p) model applied to a $\mathbf{Y} \in \mathbb{R}^{d \times T}$ at time t is defined as:

$$\mathbf{y}_t = c + \sum_{l=1}^p \mathbf{A}_l \mathbf{y}_{t-l} + \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, \Sigma_\varepsilon) \quad (6.1)$$

Structural VAR (SVAR) extends this concept by allowing contemporaneous relationships between variables:

$$\mathbf{A}_0 \mathbf{y}_t = c + \sum_{l=1}^p \mathbf{A}_l \mathbf{y}_{t-l} + \varepsilon_t \quad (6.2)$$

Given our configuration, we drop the intercept term c , and denote:

$$\mathbf{y}_t = \begin{pmatrix} \mathbf{x}_t \\ \mathbf{u}_t \end{pmatrix} \quad (6.3)$$

with $\mathbf{x}_t \in \mathbb{R}^N$ and $\mathbf{u}_t \in \mathbb{R}^M$ as the vectors of CVs and MVs at time t , respectively, such that $d = N + M$. We limit contemporaneous relationships to MV:CV pairs only, and set $\mathbf{A}_0$ as a block matrix:

$$\mathbf{A}_0 = \begin{pmatrix} \mathbf{I}_N & 0 \\ \mathbf{A}_{0,u \rightarrow x} & \mathbf{I}_M \end{pmatrix} \quad (6.4)$$

where $\mathbf{A}_{0,u \rightarrow x} \in \mathbb{R}^{N \times M}$ is the only sub-matrix containing unknown, non-zero parameters to be estimated. Because $\mathbf{A}_0$ is lower block-triangular, its determinant $\det(\mathbf{A}_0) = 1$, making the system causally invertible and identifiable.

To encourage graph sparsity and avoid spurious connections, we apply a Lasso regularization, with strength appropriately tuned based on the number of edges we desire to retain. Since the contemporaneous matrix is structured, we can rewrite the SVAR in a reduced form that isolates the target variables for a node-by-node estimation:

$$y_t^i = - \sum_{j \neq i} a_0^{ji} y_t^j + \sum_{l=1}^p \sum_{j=1}^d a_l^{ji} y_{t-l}^j + \varepsilon_t^i, \quad i = 1, \dots, N \quad (6.5)$$

Subject to the constraints of $\mathbf{A}_0$, a_0^{ji} can be non-zero only if y^j is a MV and y^i is a CV. Additionally, we force all the incoming coefficients of exogenous MVs to be null, as they are not influenced by the system dynamics. The optimization problem for node i is formulated as:

$$\min_{\mathbf{a}_0^i, \dots, \mathbf{a}_p^i} \frac{1}{2T} \sum_{t=p}^T (y_t^i - \hat{y}_t^i)^2 + \lambda \sum_{l=0}^p \sum_{j=1}^d |a_l^{ji}| \quad (6.6)$$

where $\hat{y}_t^i$ is the predicted value of y_t^i given the current parameters, and λ is a hyperparameter that controls the strength of the regularization.

One key aspect of SVAR is that it requires the data to be covariance-stationary, meaning that the statistical properties of the time series do not change significantly over time. For this reason, we apply a first-order differencing to the original signals, which is a common technique to achieve stationarity in time series analysis.

6.3 Variance bounding

To evaluate the theoretical predictive limits of our model, the methodology incorporates a rigorous analysis of signal degradation and variance bounding. Derivative forecasting in tightly regulated systems introduces specific mathematical constraints that must be quantified to prevent erroneous evaluation. For instance, when modeling the instantaneous rate of change between consecutive time steps, the dataset is highly susceptible to noise amplification, as numerical differentiation inherently acts as a high-pass filter. Let a raw sensor reading at time t be defined as $y_t = x_t + \varepsilon_t$, where x_t is the true physical state and ε_t represents independent, identically distributed sensor noise with variance σ_ε^2 . Its first difference yields:

$$\Delta y_t = y_t - y_{t-1} = (x_t - x_{t-1}) + (\varepsilon_t - \varepsilon_{t-1}) \quad (6.7)$$

Since random noise terms are independent across time steps, their variances are additive. Therefore, the noise variance of the differenced observation is explicitly doubled. Conversely, in a tightly controlled quasi steady-state phase of a process, the true physical state undergoes negligible change between consecutive measurements, meaning the variance of the true differenced signal approaches zero. Predicting this differenced observation heavily risks overfitting to the random walk of the amplified noise rather than learning actual system dynamics.

To distinguish between true predictive capability and overfitting, we calculated the Signal-to-Noise Ratio (SNR) specifically for the differenced data. The linear differenced SNR (SNR_Δ) is defined as the ratio of the true differenced signal variance to the differenced noise variance:

$$\text{SNR}_\Delta = \frac{\sigma_{\Delta x}^2}{2\sigma_\varepsilon^2} \quad (6.8)$$

The computed SNR_Δ establishes a mathematical ceiling for model performance. The maximum amount of variance a model can theoretically explain without overfitting to the amplified noise, represented as the upper bound of the coefficient of determination, is given by the equation:

$$R_{\max}^2 = \frac{\text{SNR}_\Delta}{\text{SNR}_\Delta + 1} \quad (6.9)$$

By calculating SNR_Δ directly on the differenced dataset, we established a specific $R_{\max}^2$ threshold for each variable. For a standardized target variable evaluated using MSE, comparing the model’s empirical explained variance against this theoretical $R_{\max}^2$ limit provides a definitive metric to confirm whether the model has optimally captured the available physical signal or breached the threshold of sensor noise. In other words, if the model’s training loss falls below the $1 - R_{\max}^2$ threshold, it indicates that the model is overfitting to the noise.

6.4 Baseline models

To establish a benchmark for performance evaluation, we evaluate a comprehensive set of baseline models, including both simple and state-of-the-art approaches. Table 6.1 provides an overview of the methods considered.

These architectures encompass a range of modeling paradigms, from traditional statistical techniques to advanced deep learning architectures, each optimized for specific tasks such as forecasting, reconstruction, or both. Each

Model	Description	Type
VAR	Statistical model	F
PCA	Geometric model	F
LSTM	Recurrent model	F
LSTM-AE	Recurrent model	R
MTAD-GAT	Time-and-graph model	F+R
GDN	Graph-then-time model	F
DyEdgeGAT	Time-then-graph model	R

Table 6.1: Summary of baseline models used for comparison, including the task they are optimized for: forecasting (F), reconstruction (R), or both (F+R).

of them implements a unique approach to capturing the underlying dynamics of the system: in particular, the selection of the graph-based methods is motivated by their employment of attention mechanisms to capture relationships and their evolution. Most importantly, some of these baselines elaborate new strategies for AD, providing diverse perspectives on the problem. For instance:

- PCA relies on t -squared and Q statistics (Hotelling 1931; Ku et al. 1995).
- MTAD-GAT proposes a combined score based on both reconstruction and forecasting errors.
- DyEdgeGAT introduces a novel temporal topology-based anomaly score, which leverages the encoded interaction strengths between nodes.
- GDN employs residual-based deviation scores, normalized by node-specific median and Interquartile Range (IQR), aggregated via maximum pooling, and smoothed through a Simple Moving Average (SMA).

We adapted the rest of the baselines to follow this latter approach, instead, due to its simplicity and robustness.

All experiments were conducted across three independent runs with different random initializations. We trained all models using the Adam optimizer with a learning rate of $1e-3$ for up to 300 epochs, implementing early stopping after epoch 150 with a 50-step patience window. To account for dataset scale and complexity, we used batch sizes of 128 for the TEP case study and 256 for FCCU, while window sizes were set to 10 and 20, respectively. Horizon lengths were set to 1 for all models by default. Concerning our implementation, the p -value of Equation (5.13) was set to 0.1, the EWMA smoothing bounds were set to 0.01 and 0.2, whereas the k value during the fault isolation stage was set to 5. Hyperparameter tuning is tailored to each dataset, optimizing for the best MSE on the validation partition (Appendix E.1).

Chapter 7

Results

In this chapter, we present the experimental evaluation of the proposed model alongside baseline methods across the introduced case studies. We approach each scenario individually, providing a comprehensive analysis of overall and fault-specific AD performance, as well as the effectiveness of the fault isolation stage.

While the AD phase is evaluated against multiple statistical, recurrent, and graph-based baselines, a direct quantitative comparison for the fault isolation stage is constrained by fundamental architectural differences. Conventional deep learning and multivariate baselines localize faults primarily through residual reconstruction errors or localized attention weights. These mechanisms yield continuous attribution scores across sensors rather than decomposing the fault into distinct, physically meaningful structural categories. Consequently, the evaluation of the proposed isolation stage is benchmarked directly against the ground-truth physical fault mechanisms documented in the TEP and FCCU dataset definitions, supplemented by established literature findings when applicable. For each relevant fault instance, we measure the deviation of second-stage parameters from their nominal values, ranking the primary contributors by absolute magnitude.

7.1 Tennessee Eastman Process

The results of the overall AD performance on the TEP case study are summarized in Table 7.2. The models perform consistently in terms of accuracy, precision, and recall, with the sole exception of DyEdgeGAT, which exhibits significant drops on some metrics. Comparing with the more promising outcomes obtained by other graph-based methods, this behavior is likely indicative of the limited ability of time-then-graph approaches in complex

industrial loops: due to pronounced feedback delays and recycle streams, this decoupled architecture updates spatial graphs using stale temporal states, losing transient phase alignment and failing to track systems where spatial graph topology depends dynamically on temporal transit delays.

Concurrently, VAR remains competitive with respect to many ML methods. However, its marked decline in AUC indicates poor discriminative capability across varying decision thresholds; the high recall suggests that the model is not able to effectively distinguish between normal and anomalous samples, but rather over-predicts the positive class, leading to a high number of false positives. This stems primarily from VAR’s linear structure: TEP’s exothermic reaction kinetics and nonlinear controls induce large residuals under nominal dynamic SP changes, triggering excessive false alarms at lower detection thresholds.

Recurrent models produce satisfactory results, despite the inability to explicitly account for interdependencies. This demonstrates that incorporating connectivity graphs provides an advantage only when appropriately aligned with temporal dependencies, as realized in our framework. In fact, IA-STGNN shows superior performance across nearly all evaluation metrics. Notably, our framework achieves a significant reduction in detection delay, reducing the lag from several hundred samples in baseline models to fewer than 11 samples. This early detection capability is vital for isolating primary root causes before downstream secondary effects propagate through the plant.

Detailed single-fault metrics (Appendix E.2.1), further demonstrate the framework robustness and effectiveness in handling even tedious fault scenarios. For instance, faults 3, 9, and 15 are widely recognized as the most difficult and challenging to detect, as they are characterized by a slow and gradual deviation or random fluctuations in the process variables, blending into normal opera-

	Accuracy	Precision	Recall	F1	AUC	Delay*
VAR	<u>0.84 ± 0.00</u>	0.84 ± 0.00	1.00 ± 0.00	<u>0.91 ± 0.00</u>	0.56 ± 0.24	550.5 ± 0.0
PCA	0.76	0.97	0.69	0.81	0.87	<u>150.8</u>
LSTM	0.78 ± 0.01	0.96 ± 0.00	0.73 ± 0.01	0.83 ± 0.01	<u>0.87 ± 0.00</u>	185.4 ± 1.3
LSTM-AE	0.78 ± 0.00	0.96 ± 0.01	0.73 ± 0.01	0.83 ± 0.00	0.87 ± 0.00	167.5 ± 15.6
MTAD-GAT	0.77 ± 0.01	0.94 ± 0.01	0.73 ± 0.01	0.82 ± 0.01	0.86 ± 0.01	153.6 ± 1.5
GDN	0.71 ± 0.01	0.93 ± 0.00	0.66 ± 0.01	0.77 ± 0.01	0.82 ± 0.01	200.7 ± 13.6
DyEdgeGAT	0.52 ± 0.04	<u>0.97 ± 0.01</u>	0.47 ± 0.05	0.63 ± 0.05	0.75 ± 0.02	190.0 ± 2.4
IA-STGNN	0.90 ± 0.11	0.98 ± 0.00	<u>0.89 ± 0.13</u>	0.93 ± 0.08	0.90 ± 0.06	10.9 ± 0.4

Table 7.2: TEP overall AD metrics across all faults. Best values in bold, second best underlined.

tional noise and making them less distinguishable. While baselines greatly struggle in term of AUC, presenting almost random classification performance, the IA-STGNN model is able to achieve a significant improvement, stabilizing this value around 0.78, which is a remarkable result considering the complexity of these faults. Moreover, precision completely drops for all comparative models to less than 0.18, whereas our framework maintains the almost perfect value of 0.98, mirroring performance across more pronounced fouling instances. This highlights the ability of our model to effectively identify the anomalous samples, even in the presence of subtle faults, while minimizing the number of false positives.

From a diagnostic perspective, however, the model fails to identify the effects of these latter faults to the systems, as the second-stage phase does not provide any meaningful parameters diverging from nominal values. Given the nature of these faults, it plausible that their effects are too subtle to be captured by the isolation stage, and parameters are constrained to optimal values by the Lasso regularization. In other words, we observe how high AD sensitivity does not automatically translate to isolability. Nonetheless, the unpromising diagnostics on these specific faults align with existing findings in literature (Labba-Khaniki et al. 2024), even under supervised settings.

Moving our attention to less challenging cases, a notable example is the couple of faults 4 and 11, both characterized by change in the reactor cooling water inlet temperature, but with different magnitudes and rates of change (one is a sudden step change, the other is a random). Since this setting involves a closed-loop control system, it makes the isolation of these faults particularly relevant for our study. For the first instance, IA-STGNN prioritizes precision over recall, reflecting a more prudent alarm policy. For the second case, instead, our proposal achieves the best performance across the majority of metrics, showing the ability to effectively detect the anomalous behavior even when it is strongly masked by background noise and the controller actions. The fault isolation stage (Figure 7.6) pinpoints as root cause the gain increase of the MV representing the reactor cooling water flow (**XMV (10)**), with respect to reactor temperature (**XMEAS (9)**) and reactor cooling water outlet temperature (**XMEAS (21)**). This unexpected sensitivity increment to the cooling dynamics is indicative of an actuator fault; more precisely, the elevated reactor temperature confirms diminished heat removal efficiency, which is consistent with the underlying origin of the fault. Secondary deviations are perfectly within the expected consequences of the faults, such as the evolution of the interaction between reactor internal state and its feed rate (**XMEAS (6)**).

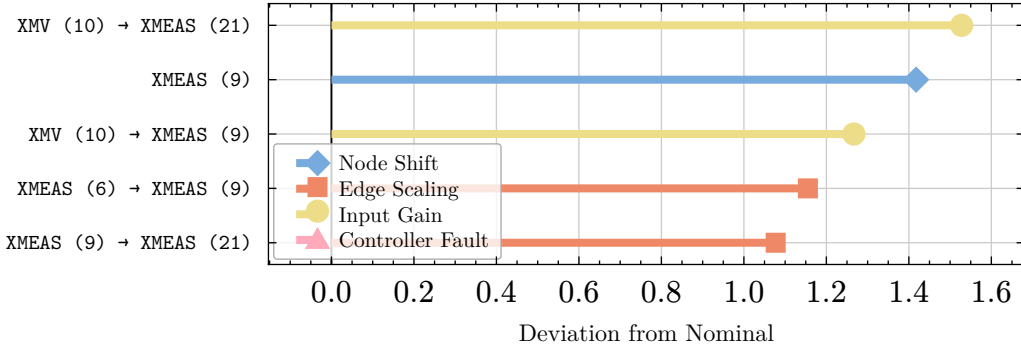


Figure 7.6: TEP diagnosis for fault 4.

A related case is represented by fault 14, involving a sticking valve malfunction within the same cooling subcomponent. Again, our model improves upon baseline AD performance. Additionally, fault diagnosis (Figure 7.7) provides similar top priority deviations to the previous scenarios, but secondary effects allow for a different understanding of the issue origin: the input gain increase, paired with concurrent drop in reactor temperature and pressure, signals over-cooling, overall indicative of a valve stuck open rather than a cooling feed temperature disturbance. This comparison exposes why multi-variable structural dependency modeling is essential to resolve diagnostic ambiguities between actuator malfunctions and process disturbance faults that share primary symptoms. In other words, we highlight the importance of an exhaustive analysis, providing more detailed information with respect to the sole reliance on anomaly scores, which may lead to misinterpretation.

Other evaluated scenarios (Appendix E.2.2) show overall superior performance, as well as consistency with documented findings (Chen et al. 2018; Zhu et al. 2022). In a few cases, the exact distinction between the effects of the fault and its origin are not remains unclear, which is likely due to the non-null delay in the AD phase, which causes the second-stage to be executed when the fault has already started and it propagates through the system.

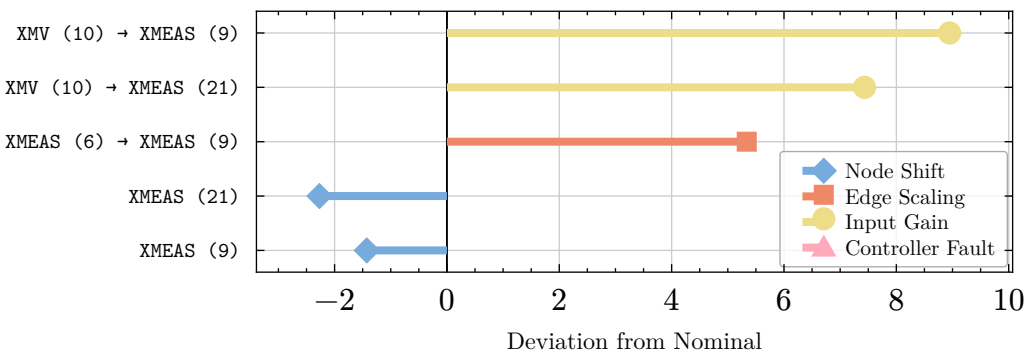


Figure 7.7: TEP diagnosis for fault 14.

7.2 Fluid Catalytic Craking Unit

Table 7.3 summarizes the overall AD results on the FCCU case study. While most baselines struggle with accuracy, recall, and F1 score, the IA-STGNN attains a significant improvements across all metrics. It demonstrates promising performance both on physical connectivity obtained from the P&ID diagram and on system-identification adjacency matrices derived from data. This complex process topology, dominated by tight multi-loop coupling and numerous closed-loop control subunits, put a considerable strain on baseline models, which exhibit recall lower than 0.5 and F1 score under 0.65 despite high precision. This pattern indicates overly conservative decision boundaries that produce an excessive number of false negatives. Single-fault evaluations generally corroborate this aggregate findings (Appendix E.3.1). Nonetheless, detection delays are mixed, with our model achieving the best and third best values.

Comparing connectivity schemes demonstrates that physics-based relationships capture underlying dynamics more effectively, leading to superior overall detection metrics. Nevertheless, the system-identification matrix still provides valuable performance, confirming the robustness of our framework when nominal P&ID information is unavailable or incomplete. Additionally, the linear nature of the system identification method likely limits its ability to fully capture the complex relationships between the variables, imposing an upper bound on AD performance. In fact, despite the overall satisfactory inference (Appendix D.2.2), several control loops are not correctly mapped, introducing a non-negligible structural bias into the graph.

	Accuracy	Precision	Recall	F1	AUC	Delay*
VAR	0.64 ± 0.01	0.90 ± 0.02	0.50 ± 0.01	0.64 ± 0.01	0.72 ± 0.00	457.8 ± 10.3
PCA	0.59	0.91	0.39	0.55	0.70	85.6
LSTM	0.60 ± 0.01	0.97 ± 0.02	0.39 ± 0.02	0.56 ± 0.02	0.69 ± 0.02	88.1 ± 8.6
LSTM-AE	0.64 ± 0.00	<u>0.99 ± 0.01</u>	0.45 ± 0.00	0.62 ± 0.00	<u>0.75 ± 0.01</u>	<u>41.0 ± 2.0</u>
MTAD-GAT	0.60 ± 0.03	1.00 ± 0.00	0.38 ± 0.05	0.54 ± 0.05	0.72 ± 0.00	73.7 ± 9.9
GDN	0.58 ± 0.02	0.99 ± 0.01	0.36 ± 0.03	0.53 ± 0.03	0.70 ± 0.02	98.1 ± 33.1
DyEdgeGAT	0.41 ± 0.05	0.67 ± 0.08	0.17 ± 0.09	0.26 ± 0.12	0.47 ± 0.07	515.5 ± 50.2
IA-STGNN _P	0.79 ± 0.15	0.84 ± 0.06	0.81 ± 0.23	0.81 ± 0.17	0.78 ± 0.12	28.3 ± 6.4
IA-STGNN _{SVAR}	<u>0.72 ± 0.19</u>	0.81 ± 0.08	<u>0.77 ± 0.11</u>	<u>0.76 ± 0.15</u>	0.73 ± 0.09	44.9 ± 3.2

Table 7.3: FCCU overall AD metrics across all faults. IA-STGNN results are reported for experiments with either the physical (P) or system-identification (SVAR) connectivity matrix.

Best values in bold, second best underlined.

	H	Accuracy	Precision	Recall	F1	AUC	Delay*
IA-STGNN	1	<u>0.79 ± 0.15</u>	0.84 ± 0.06	0.81 ± 0.23	0.81 ± 0.17	0.78 ± 0.12	28.2 ± 6.4
IA-STGNN	2	0.78 ± 0.13	0.81 ± 0.05	<u>0.85 ± 0.20</u>	<u>0.82 ± 0.13</u>	0.76 ± 0.10	29.4 ± 5.1
IA-STGNN	5	0.79 ± 0.15	<u>0.83 ± 0.06</u>	0.82 ± 0.23	0.81 ± 0.17	<u>0.77 ± 0.12</u>	<u>20.2 ± 6.1</u>
IA-STGNN	10	0.80 ± 0.08	0.81 ± 0.03	0.89 ± 0.13	0.84 ± 0.09	0.75 ± 0.06	12.8 ± 7.3

Table 7.4: FCCU AD metrics across all faults for different prediction horizons (H). The connectivity matrix used is the physical one. Best values in bold, second best underlined.

Evaluating our model on different forecasting horizons (Table 7.4) reveals stable results across all metrics, with only minor variations, confirming the robustness of our approach and the consistency of the learned dynamics. Notably, detection delay becomes less pronounced as the horizon length increases. At shorter horizons, the model likely lean on current temporal states and recent history, whereas extended horizons compel the architecture to exploit learned long-term dependencies, leading to more accurate and timely alarms. The potential risk of over-reliance on short-term patterns should not be underestimated, as it may lead to delayed or missed detections of anomalies that manifest over longer time scales, such as slow drifts or gradual changes. Nonetheless, a lower detection delay is particularly relevant for the isolation stage, as it allows to uncover the fault earlier, possibly providing more reliable information for the subsequent analysis of the root cause. Additionally, the ability of forecasting distant future samples is a valuable feature for industrial applications, as it favors proactive maintenance and early intervention, potentially preventing costly downtime and improving overall operational efficiency.

Isolation evaluations on FCCU further highlight the diagnostic precision of the proposed framework. The fault `deltaP_increase` represents an abrupt increase in the reactor pressure, which is otherwise stable during healthy operation. The model immediately identifies the root cause of the fault (Figure 7.8), assigning highest priority to the reactor pressure shift and the corresponding

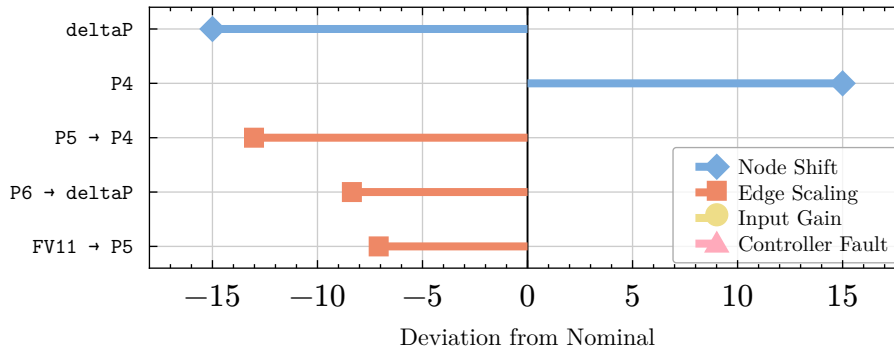


Figure 7.8: FCCU diagnosis for fault `deltaP_increase`.

drop in regenerator-reactor differential pressure. Secondary identified deviations align with the expected effects of the fault, such as plant-wide pneumatic imbalances described by edge scalings. However, the responses on catalyst circulation rate and level receive lower priority rankings due to their smaller relative parameter magnitudes.

In scenario **UAf_decrease**, a heat-transfer coefficient reduction in the reactor preheater requires higher furnace temperatures (T3) and larger fuel consumption (F5) to sustain nominal throughput. The estimated parameter deviations (Figure 7.9) accurately decouple the underlying equipment fouling from the controller's regulatory actions. A dominant negative edge scaling marks the breakdown in thermal exchange efficiency between fuel combustion and stack temperature caused by heat-exchanger resistance. In response to dropping feed temperatures, the controller aggressively increases fuel flow, manifested as a significant positive input gain shift on the fuel control valve. Because this excess thermal energy cannot be absorbed by the process fluid, it vents out the stack, producing the observed positive node shift in furnace temperature. Meanwhile, the sensitivity drop with respect to external atmospheric temperature (T1) is a direct consequence of the fault. Overall, the architecture successfully pinpoints the preheater as the primary fault location: since heat-transfer coefficients are unmeasured states, the model correctly infers the unmeasured fouling phenomenon through its footprint on the governing thermodynamics.

The instance **CAB_valveLeak** represents a pure actuator fault, characterized by an effective gain loss on the Combustion Air Blower (V6). The model recognized this degradation (Figure 7.10); however, downstream effects rank higher in absolute magnitude, with larger deviations representing disruptions withing the regenerator section. Air loss alters aerodynamic resistance and inlet mass balance, disrupting the the nominal relationships governing regenerator dynamics, such as pressure (P6), temperature (T_reg), flow of air

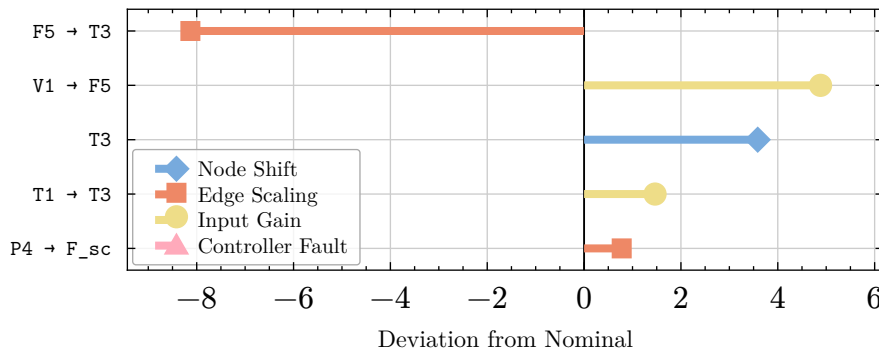


Figure 7.9: FCCU diagnosis for fault **UAf_decrease**.

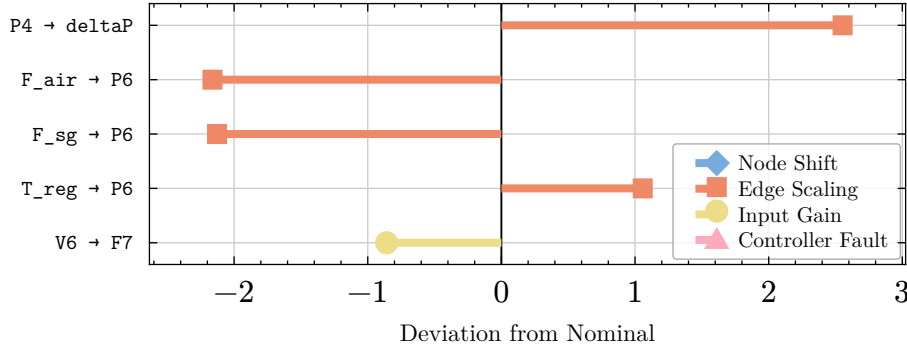
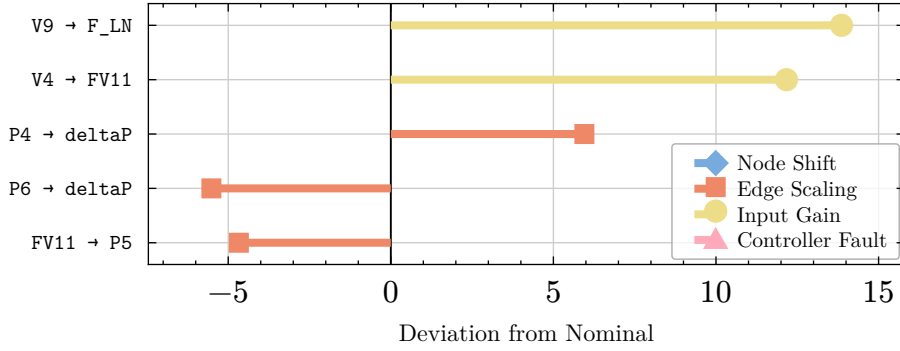


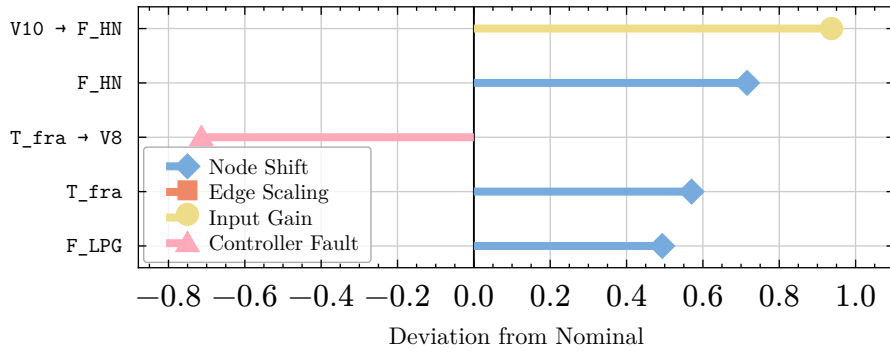
Figure 7.10: FCCU diagnosis for fault CAB_valveLeak.

(F_{air}), flow of stack gas (F_{sg}), and the differential pressure (deltaP). Although the model prioritizes these effects over the fault origin, this diagnosis should not be attributed to the suboptimal delay in the detection phase. In fact, the actuator gain decreases to its lower bound, approaching a near-zero parameter: in other words, the leakage is so severe that the control valve has lost control authority and must open fully to sustain flow. Consequently, the model correctly identifies the fault in its entirety, but the intensity-oriented ordering initially leads to a misinterpretation of the root cause. This case exposes a core limitation of purely magnitude-based ranking: additive biases and multiplicative gains operate on different scales, meaning that an actuator gain change may represent the true root cause despite having a smaller absolute value than downstream state deviations.

The case `condEff_decrease` involves decreased condenser efficiency, corresponding to less condensation of the gas products into liquid. This fouling inflates liquefied petroleum gas production while suppressing light naphtha yielding, disrupting the respective flow rates (F_{LPG} and F_{LN}), control valves (V4 and V9), and vapor draw through the Wet Gas Compressor (FV11). Predicted deviations (Figure 7.11) isolates the condensation shortfall while tracing noticeable propagation into the main pressure balances. The negative edge scaling between the overhead pressure (P5) and compressor flow accurately identifies the breakdown in overhead thermodynamic equilibrium, where reduced heat removal decouples vapor flow rate from column pressure. To stabilize the resulting vapor surge and protect fractionation, the overhead controllers adjust valves positions, reflected in the elevated positive input gains. The intermediate edge scalings across vessel pressures represent propagated pneumatic effects: because fractionator overhead pressure serves as the downstream boundary condition for the reactor, the architecture distributes part of the fault signature across the upstream reactor-regenerator pressure network. Otherwise, we achieve satisfactory results in terms of localization.

Figure 7.11: FCCU diagnosis for fault `condEff_decrease`.

Finally, `Fhn_sensorDrift` introduces a slow calibration drift in the measurement of heavy naphtha (HN) flow. Since this point-like bias accumulates gradually, early detection is critical to avoid confounding the sensor fault with subsequent regulatory action. The model flags the anomalous channel (Figure 7.12), but assigns higher priority to the compensatory actuator gain adjustment involving the controller. This action is a direct response to the false reading, and triggers a cascade of secondary effects across the fractionator: because withholding heavy naphtha alters internal column traffic, temperature and overhead vapor splits shift. Consequently, this prompts control action captured by the controller fault, and hydraulic and thermal symptoms represented by mild positive node shifts. The diagnostic ambiguity highlights how residual detection delays can allow controller compensation to obscure the initial measurement failure. Even though AD metrics are lower for this subtle drift than for other faults, the framework reliably localizes the affected subunit.

Figure 7.12: FCCU diagnosis for fault `Fhn_sensorDrift`.

Chapter 8

Conclusion

In this work, we presented an interpretable Input-Affine Spatiotemporal Graph Neural Network (IA-STGNN) framework specifically tailored for unified AD and structural root-cause fault isolation within continuous, highly nonlinear industrial processes. By systematically intertwining spatial dependency graphs with temporal sequence modeling, our architecture resolves the structural deficiencies prevalent in standard deep multivariate baselines and decoupled temporal-graph networks. Rather than treating plant sensor networks as unstructured multi-channel signals or static topological projections, our formulation tracks dynamics across transit delays, accurately mapping time-dependent physical interactions without losing transient temporal alignment.

Empirical validations conducted across representative, tightly coupled chemical engineering benchmarks demonstrate both the statistical rigor and the practical viability of the proposed framework. Most prominently, IA-STGNN delivers a decisive reduction in early detection latency when compared against state-of-the-art multivariate and spatial baselines. Within industrial facilities, where material recycles and thermal feedbacks rapidly distribute local perturbations across adjacent unit operations, minimal detection delay serves as the fundamental prerequisite for unmasking primary root causes prior to systemic contamination. In the most challenging fault configurations, defined by low-amplitude parameter drifts and persistent stochastic fluctuations, baseline architectures suffer from severe precision collapse, sliding toward near-random classification regimes. In contrast, IA-STGNN preserves high precision and sustained discriminative capacity under unfavorable signal-to-noise ratios, suppressing false alarms provoked by nominal operational transitions and nonlinear control actions.

Comparative connectivity analyses establish that, while domain-informed spatial topologies extracted directly from P&IDs yield peak detection fidelity,

purely empirical adjacency graphs inferred via data-driven system identification offer an operationally competitive alternative. This parity confirms the model’s practical utility in legacy processing environments where detailed engineering schematics are missing, incomplete, or inaccessible. Nevertheless, because standard linear system identification struggles with the severe complexities and recirculating flows inherent to industrial plants, the derived topologies can introduce unmodeled interactions or structural oversights. Future research should leverage advanced nonlinear causal discovery methodologies, such as constraint-based dynamic discovery and PCMCi algorithms (Spirtes and Glymour 1991; Runge et al. 2019; Runge 2020). Through the second-stage parameter divergence optimization, our framework accurately localizes failing subunits and isolates principal operational disruptions. The induced Lasso sparsity constraint successfully separates primary manipulated variable drifts and actuator degradations from downstream hydrodynamic repercussions in most cases.

Despite these technical achievements, the empirical findings define several clear directions for methodological refinement. Because an instantaneous zero-sample detection latency is physically unrealizable in distributed industrial processes, residual detection delays occasionally allow high-energy downstream consequences to eclipse the primary initiation point. Under purely magnitude-based ranking schemes, high-amplitude state fluctuations can overshadow subtle actuator parameter changes. To counteract this limitation, future iterations could incorporate directed causal flow tracking to weight estimated parameter deviations by their upstream topological depth rather than raw absolute magnitude, untangling closed-loop control masks through spatial priority ranking.

Furthermore, while the initial detection module reliably flags minimal deviations, the uniform penalization imposed by static Lasso regularization can prove overly restrictive when applied across disparate fault severities, occasionally suppressing subtle structural shifts below the identification threshold. Introducing adaptive, variance-normalized regularization schedules could provide the mathematical flexibility required to isolate weak drifts without introducing spurious edge activations. Extending this principle further, end-to-end formulations that merge AD and sparse parameter decomposition into a unified multi-task objective could bypass the latency overhead and boundary error inherent to sequential architectures.

Finally, while the temporal modeling layers capture variable transit times implicitly through internal recurrent dynamics, incorporating explicit variable-

delay attention kernels or parameterized time-lag channels would expose temporal latency distributions across specific piping segments, improving propagation effects modeling and providing operators with actionable estimates of their velocities.

Lastly, bridging the methodological divide between our parametric structural isolation framework and the heuristic, reconstruction-based attribution profiles used by black-box baselines remains a critical benchmarking challenge. Developing surrogate diagnostic scoring metrics will be essential to establish objective, cross-paradigm quantitative evaluations for industrial root-cause analysis.

Bibliography

- Brody, Shaked, Uri Alon, and Eran Yahav. 2022. “How Attentive Are Graph Attention Networks?.” no. arXiv:2105.14491. ArXiv, January. <https://doi.org/10.48550/arXiv.2105.14491>.
- Butera, Luca, Giovanni De Felice, Andrea Cini, and Cesare Alippi. 2024. “On the Regularization of Learnable Embeddings for Time Series Forecasting.” *Transactions on Machine Learning Research*, October.
- Ceni, Andrea, Alessio Gravina, Carola-Bibiane Schönlieb, Claudio Gallicchio, Davide Bacciu, and Moshe Eliasof. 2025. “Message-Passing State-Space Models: Improving Graph Learning with Modern Sequence Modeling.” In “UniReps: 3rd Edition of the Workshop on Unifying Representations in Neural Models.” Special issue, *UniReps: 3rd Edition of the Workshop on Unifying Representations in Neural Models*, November.
- Chandola, Varun, Arindam Banerjee, and Vipin Kumar. 2009. “Anomaly Detection: A Survey.” *ACM Computing Surveys (CSUR)* 41 (3): 15:1–15:58. <https://doi.org/10.1145/1541880.1541882>.
- Chen et al. 2023. “Interaction-Aware Graph Neural Networks for Fault Diagnosis of Complex Industrial Processes.” *IEEE Transactions on Neural Networks and Learning Systems* 34 (9): 6015–28. <https://doi.org/10.1109/TNNLS.2021.3132376>.
- Chen et al. 2018. “Root Cause Diagnosis of Process Faults Using Conditional Granger Causality Analysis and Maximum Spanning Tree.” *IFAC-Papersonline*, 10th IFAC Symposium on Advanced Control of Chemical Processes ADCHEM 2018, vol. 51 (18): 381–86. <https://doi.org/10.1016/j.ifacol.2018.09.330>.
- Chen et al. 2023. “Multivariate Time Series Anomaly Detection via Dynamic Graph Forecasting.” no. arXiv:2302.02051. ArXiv, February. <https://doi.org/10.48550/arXiv.2302.02051>.
- Chen et al. 2018. “Neural Ordinary Differential Equations.” In “Advances in Neural Information Processing Systems.” Special issue, *Advances in Neural Information Processing Systems* 31.
- Cho, Kyunghyun, Bart van Merriënboer, Dzmitry Bahdanau, and Yoshua Bengio. 2014. “On the Properties of Neural Machine Translation: Encoder–Decoder Approaches.” In *Proceedings of SSST-8, Eighth Workshop on*

- Syntax, Semantics and Structure in Statistical Translation*, edited by Dekai Wu, Marine Carpuat, Xavier Carreras, and Eva Maria Vecchi, *Proceedings of SSST-8, Eighth Workshop on Syntax, Semantics and Structure in Statistical Translation*. Association for Computational Linguistics. <https://doi.org/10.3115/v1/W14-4012>.
- Cini, Andrea, Ivan Marisca, Daniele Zambon, and Cesare Alippi. 2023. “Taming Local Effects in Graph-based Spatiotemporal Forecasting.” no. arXiv:2302.04071. ArXiv, November. <https://doi.org/10.48550/arXiv.2302.04071>.
- Cini, Andrea, Ivan Marisca, Daniele Zambon, and Cesare Alippi. 2025. “Graph Deep Learning for Time Series Forecasting.” *ACM Comput. Surv.* 57 (12): 321:1–321:34. <https://doi.org/10.1145/3742784>.
- Clelland, Jeanne N. 2009. “Geometry of Control-Affine Systems.” *Symmetry, Integrability and Geometry: Methods and Applications*, ahead of print, October. <https://doi.org/10.3842/SIGMA.2009.095>.
- Corradini, Flavio, Flavio Gerosa, Marco Gori, Carlo Lucheroni, and Martina Zannotti. 2025. *A Systematic Literature Review of Spatio-Temporal Graph Neural Network Models for Time Series Forecasting and Classification*. <https://doi.org/10.1016/j.neunet.2025.108269>.
- Deng, Ailin, and Bryan Hooi. 2021. “Graph Neural Network-Based Anomaly Detection in Multivariate Time Series.” no. arXiv:2106.06947. ArXiv, June. <https://doi.org/10.48550/arXiv.2106.06947>.
- Downs, J. J., and E. F. Vogel. 1993. “A Plant-Wide Industrial Process Control Problem.” *Computers & Chemical Engineering*, Industrial Challenge Problems in Process Control, vol. 17 (3): 245–55. [https://doi.org/10.1016/0098-1354\(93\)80018-I](https://doi.org/10.1016/0098-1354(93)80018-I).
- Dwivedi, Vijay Prakash, Chaitanya K. Joshi, Anh Tuan Luu, Thomas Laurent, Yoshua Bengio, and Xavier Bresson. 2023. “Benchmarking Graph Neural Networks.” *Journal of Machine Learning Research* 24 (43).
- Elkin, V. I. 1998. “Affine Control Systems: Their Equivalence, Classification, Quotient Systems, And Subsystems.” *Journal of Mathematical Sciences* 88 (5): 675–721. <https://doi.org/10.1007/BF02364666>.
- Feng, Xing, Hongru Li, and Yinghua Yang. 2025. “Time-Lagged Relation Graph Neural Network for Multivariate Time Series Forecasting.” *Engineering Applications of Artificial Intelligence* 139 (January): 109530. <https://doi.org/10.1016/j.engappai.2024.109530>.

- Feng, ZhengZhao, Rui Wang, TianXing Wang, Mingli Song, Sai Wu, and Shuibing He. 2026. "A Comprehensive Survey of Dynamic Graph Neural Networks: Models, Frameworks, Benchmarks, Experiments and Challenges." *IEEE Transactions on Knowledge and Data Engineering* 38 (1): 26–46. <https://doi.org/10.1109/TKDE.2025.3621291>.
- Gandolfo, G. 1971. *Economic Dynamics: Methods and Models*. Elsevier.
- Gao, Jianfei, and Bruno Ribeiro. 2023. "On the Equivalence Between Temporal and Static Graph Representations for Observational Predictions." no. arXiv:2103.07016. ArXiv, March. <https://doi.org/10.48550/arXiv.2103.07016>.
- Gori, M., G. Monfardini, and F. Scarselli. 2005. "A New Model for Learning in Graph Domains." In "Proceedings. 2005 IEEE International Joint Conference on Neural Networks, 2005.." Special issue, *Proceedings. 2005 IEEE International Joint Conference on Neural Networks, 2005*. 2 (July): 729–734 vol. 2. <https://doi.org/10.1109/IJCNN.2005.1555942>.
- Granger, C. W. J. 1969. "Investigating Causal Relations by Econometric Models and Cross-spectral Methods." *Econometrica* 37 (3): 424–38. <https://doi.org/10.2307/1912791>.
- Henson, M., and D. Seborg. 1996. "Nonlinear Process Control." March.
- Hotelling, Harold. 1931. "The Generalization of Student's Ratio." *The Annals of Mathematical Statistics* 2 (3): 360–78. <https://doi.org/10.1214/aoms/1177732979>.
- Hu, Weihua, Bowen Liu, Joseph Gomes, et al. 2020. "Strategies for Pre-training Graph Neural Networks." no. arXiv:1905.12265. ArXiv, February. <https://doi.org/10.48550/arXiv.1905.12265>.
- T. Jackson, A. Radunskaya. 2015. *Applications of Dynamical Systems in Biology and Medicine*. Vol. 158, pt. 158. The IMA Volumes in Mathematics and Its Applications. Springer. <https://doi.org/10.1007/978-1-4939-2782-1>.
- Jang, Eric, Shixiang Gu, and Ben Poole. 2017. "Categorical Reparameterization with Gumbel-Softmax." no. arXiv:1611.01144. ArXiv, August. <https://doi.org/10.48550/arXiv.1611.01144>.
- Januschowski, Tim, Jan Gasthaus, Yuyang Wang, et al. 2020. "Criteria for Classifying Forecasting Methods." *International Journal of Forecasting* 36 (1): 167–77. <https://doi.org/10.1016/j.ijforecast.2019.05.008>.

- Ji, Cheng, Fangyuan Ma, Jianhong Wang, Jingde Wang, and Wei Sun. 2021. “Real-Time Industrial Process Fault Diagnosis Based on Time Delayed Mutual Information Analysis.” *Processes* 9 (6): 1027. <https://doi.org/10.3390/pr9061027>.
- Jia, Mingwei, Yuan Yao, and Yi Liu. 2025. “Review on Graph Neural Networks for Process Soft Sensor Development, Fault Diagnosis, And Process Monitoring.” *Industrial & Engineering Chemistry Research* 64 (17): 8543–64. <https://doi.org/10.1021/acs.iecr.5c00283>.
- Jiang, Jiawei, Chengkai Han, Wayne Xin Zhao, and Jingyuan Wang. 2023. “PDFFormer: Propagation Delay-Aware Dynamic Long-Range Transformer for Traffic Flow Prediction.” In “Proceedings of the Thirty-Seventh AAAI Conference on Artificial Intelligence and Thirty-Fifth Conference on Innovative Applications of Artificial Intelligence and Thirteenth Symposium on Educational Advances in Artificial Intelligence.” Special issue, *Proceedings of the Thirty-Seventh AAAI Conference on Artificial Intelligence and Thirty-Fifth Conference on Innovative Applications of Artificial Intelligence and Thirteenth Symposium on Educational Advances in Artificial Intelligence*, AAAI'23/IAAI'23/EAAI'23, vol. 37 (February): 4365–73. <https://doi.org/10.1609/aaai.v37i4.25556>.
- Jin, Ming, Huan Yee Koh, Qingsong Wen, et al. 2024. “A Survey on Graph Neural Networks for Time Series: Forecasting, Classification, Imputation, And Anomaly Detection.” *IEEE Transactions on Pattern Analysis and Machine Intelligence* 46 (12): 10466–85. <https://doi.org/10.1109/TPAMI.2024.3443141>.
- Kazemi, Seyed Mehran, Rishab Goel, Kshitij Jain, et al. 2020. “Representation Learning for Dynamic Graphs: A Survey.” no. arXiv:1905.11485. ArXiv, April. <https://doi.org/10.48550/arXiv.1905.11485>.
- Khalil, Hassan K. 2002. *Nonlinear Systems*. Prentice Hall.
- Kidger, Patrick, James Morrill, James Foster, and Terry Lyons. 2020. “Neural Controlled Differential Equations for Irregular Time Series.” In “Advances in Neural Information Processing Systems.” Special issue, *Advances in Neural Information Processing Systems* 33 : 6696–707.
- Kipf, Thomas N., and Max Welling. 2017. “Semi-Supervised Classification with Graph Convolutional Networks.” no. arXiv:1609.02907. ArXiv, February. <https://doi.org/10.48550/arXiv.1609.02907>.

- Ku, Wenfu, Robert H. Storer, and Christos Georgakis. 1995. "Disturbance Detection and Isolation by Dynamic Principal Component Analysis." *Chemometrics and Intelligent Laboratory Systems*, InCINC '94 Selected Papers from the First International Chemometrics Internet Conference, vol. 30 (1): 179–96. [https://doi.org/10.1016/0169-7439\(95\)00076-3](https://doi.org/10.1016/0169-7439(95)00076-3).
- Kullback, S., and R. A. Leibler. 1951. "On Information and Sufficiency." *The Annals of Mathematical Statistics* 22 (1): 79–86. <https://doi.org/10.1214/aoms/1177729694>.
- Kumar, Ankur. 2025. "ML-PSE/Fluid-Catalytic-Cracking-Unit-Dataset-for-Process-Monitoring-Evaluation." December.
- Labbaġ-Khaniki, Mohammad Ali, Mohammad Manthouri, and Hanieh Ajami. 2024. "Twin Transformer Using Gated Dynamic Learnable Attention Mechanism for Fault Detection and Diagnosis in the Tennessee Eastman Process." no. arXiv:2403.10842. ArXiv, November. <https://doi.org/10.48550/arXiv.2403.10842>.
- Li, Gang, Carlos F. Alcala, S. Joe Qin, and Donghua Zhou. 2011. "Generalized Reconstruction-Based Contributions for Output-Relevant Fault Diagnosis With Application to the Tennessee Eastman Process." *IEEE Transactions on Control Systems Technology* 19 (5): 1114–27. <https://doi.org/10.1109/TCST.2010.2071415>.
- Li, Yaguang, Rose Yu, Cyrus Shahabi, and Yan Liu. 2018. "Diffusion Convolutional Recurrent Neural Network: Data-Driven Traffic Forecasting." no. arXiv:1707.01926. ArXiv, February. <https://doi.org/10.48550/arXiv.1707.01926>.
- Liu, Ruonan, Quanhu Zhang, Di Lin, Weidong Zhang, and Steven X. Ding. 2024. "Causal Intervention Graph Neural Network for Fault Diagnosis of Complex Industrial Processes." *Reliability Engineering & System Safety* 251 (November): 110328. <https://doi.org/10.1016/j.ress.2024.110328>.
- Longa, Antonio, Veronica Lachi, Gabriele Santin, et al. 2023. "Graph Neural Networks for Temporal Graphs: State of the Art, Open Challenges, And Opportunities." *Transactions on Machine Learning Research*, May.
- Łuczak, Dominik, Stefan Brock, and Krzysztof Siembab. 2023. "Cloud Based Fault Diagnosis by Convolutional Neural Network as Time-Frequency RGB Image Recognition of Industrial Machine Vibration with Internet of Things Connectivity." *Sensors* 23 (7): 3755. <https://doi.org/10.3390/s23073755>.

- Maddison, Chris J., Andriy Mnih, and Yee Whye Teh. 2017. “The Concrete Distribution: A Continuous Relaxation of Discrete Random Variables.” no. arXiv:1611.00712. ArXiv, March. <https://doi.org/10.48550/arXiv.1611.00712>.
- Mankiw, N. Gregory. 2016. *Macroeconomics*. Worth Pub.
- Montero-Manso, Pablo, and Rob J. Hyndman. 2021. “Principles and Algorithms for Forecasting Groups of Time Series: Locality and Globality.” *International Journal of Forecasting* 37 (4): 1632–53. <https://doi.org/10.1016/j.ijforecast.2021.03.004>.
- Rieth, Cory A., Ben D. Amsel, Randy Tran, and Maia B. Cook. 2017. “Additional Tennessee Eastman Process Simulation Data for Anomaly Detection Evaluation.” Harvard Dataverse, July. <https://doi.org/10.7910/DVN/6C3JR1>.
- Rieth, Cory A., Ben D. Amsel, Randy Tran, and Maia B. Cook. 2018. “Issues and Advances in Anomaly Detection Evaluation for Joint Human-Automated Systems.” In *Advances in Human Factors in Robots and Unmanned Systems*, edited by Jessie Chen, *Advances in Human Factors in Robots and Unmanned Systems*. Springer International Publishing. https://doi.org/10.1007/978-3-319-60384-1_6.
- Rossi, Emanuele, Ben Chamberlain, Fabrizio Frasca, Davide Eynard, Federico Monti, and Michael Bronstein. 2020. “Temporal Graph Networks for Deep Learning on Dynamic Graphs.” no. arXiv:2006.10637. ArXiv, October. <https://doi.org/10.48550/arXiv.2006.10637>.
- Runge, Jakob. 2020. “Discovering Contemporaneous and Lagged Causal Relations in Autocorrelated Nonlinear Time Series Datasets.” In “Proceedings of the 36th Conference on Uncertainty in Artificial Intelligence (UAI).” Special issue, *Proceedings of the 36th Conference on Uncertainty in Artificial Intelligence (UAI)*, August, 1388–97.
- Runge, Jakob, Peer Nowack, Marlene Kretschmer, Seth Flaxman, and Dino Sejdinovic. 2019. “Detecting and Quantifying Causal Associations in Large Nonlinear Time Series Datasets.” *Science Advances* 5 (11): eaau4996. <https://doi.org/10.1126/sciadv.aau4996>.
- Santander, Omar, Vidyashankar Kuppuraj, Christopher A. Harrison, and Michael Baldea. 2022. “An Open Source Fluid Catalytic Cracker - Fractionator Model to Support the Development and Benchmarking of Process Control, Machine Learning and Operation Strategies.” *Computers*

- Chemical Engineering* 164 (August): 107900. <https://doi.org/10.1016/j.compchemeng.2022.107900>.
- Scarselli, F., M. Gori, Ah Chung Tsoi, M. Hagenbuchner, and G. Monfardini. 2009. “The Graph Neural Network Model.” *IEEE Transactions on Neural Networks* 20 (1): 61–80. <https://doi.org/10.1109/TNN.2008.2005605>.
- Seo, Youngjoo, Michaël Defferrard, Pierre Vandergheynst, and Xavier Bresson. 2018. “Structured Sequence Modeling with Graph Convolutional Recurrent Networks.” In *Neural Information Processing*, edited by Long Cheng, Andrew Chi Sing Leung, and Seiichi Ozawa, *Neural Information Processing*. Springer International Publishing. https://doi.org/10.1007/978-3-030-04167-0_33.
- Song, Xiuyao, Mingxi Wu, Christopher Jermaine, and Sanjay Ranka. 2007. “Conditional Anomaly Detection.” *IEEE Transactions on Knowledge and Data Engineering* 19 (5): 631–45. <https://doi.org/10.1109/TKDE.2007.1009>.
- Spirtes, Peter, and Clark Glymour. 1991. “An Algorithm for Fast Recovery of Sparse Causal Graphs.” *Social Science Computer Review* 9 (1): 62–72. <https://doi.org/10.1177/089443939100900106>.
- Sun, Bowen, Jiongqi Wang, Zhangming He, Haiyin Zhou, and Fengshou Gu. 2019. “Fault Identification for a Closed-Loop Control System Based on an Improved Deep Neural Network.” *Sensors* 19 (9): 2131. <https://doi.org/10.3390/s19092131>.
- Sutskever, Ilya, Oriol Vinyals, and Quoc V. Le. 2014. “Sequence to Sequence Learning with Neural Networks.” In “Advances in Neural Information Processing Systems.” Special issue, *Advances in Neural Information Processing Systems* 27.
- Veličković, Petar, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. “Graph Attention Networks.” no. arXiv:1710.10903. ArXiv, February. <https://doi.org/10.48550/arXiv.1710.10903>.
- Wang, Zhiqiang, Jianghao Wen, and Jianqing Liang. 2025. “Delay-DSGN: A Dynamic Spiking Graph Neural Network with Delay Mechanisms for Evolving Graph.” In “Forty-Second International Conference on Machine Learning.” Special issue, *Forty-Second International Conference on Machine Learning*, June.

- Wu, Zonghan, Da Zheng, Shirui Pan, Quan Gan, Guodong Long, and George Karypis. 2024. “TraverseNet: Unifying Space and Time in Message Passing for Traffic Forecasting.” *IEEE Transactions on Neural Networks and Learning Systems* 35 (2): 2003–13. <https://doi.org/10.1109/TNNLS.2022.3186103>.
- Xu, Kang, Yuan Li, Yixuan Li, Liyan Xu, Ruiyao Li, and Zhenjiang Dong. 2023. “Masked Graph Neural Networks for Unsupervised Anomaly Detection in Multivariate Time Series.” *Sensors* 23 (17): 7552. <https://doi.org/10.3390/s23177552>.
- Xu, Keyulu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. 2019. “How Powerful Are Graph Neural Networks?.” no. arXiv:1810.00826. ArXiv, February. <https://doi.org/10.48550/arXiv.1810.00826>.
- You, Mingjie, Dawei Cheng, Meilin Zhang, Peng Zhu, and Yuqi Liang. 2026. “Delay-Aware Graph Neural Stochastic Differential Equations for Financial Time Series Modeling and Forecasting.” In “Proceedings of the ACM Web Conference 2026.” Special issue, *Proceedings of the ACM Web Conference 2026* (New York, NY, USA), WWW '26, April, 7967–78. <https://doi.org/10.1145/3774904.3792829>.
- Yuan, Weiheng, and Hua Yan. 2026. “Enhancing Runoff Prediction with Causal Lag-Aware Attention and Multi-Scale Fusion in Transformer Models.” *Journal of Hydrology* 664 (January): 134369. <https://doi.org/10.1016/j.jhydrol.2025.134369>.
- Zagorowska, Marta, Ouyang Wu, James R. Ottewill, Marcus Reble, and Nina F. Thornhill. 2020. “A Survey of Models of Degradation for Control Applications.” *Annual Reviews in Control* 50 : 150–73. <https://doi.org/10.1016/j.arcontrol.2020.08.002>.
- Zambon, Daniele, Andrea Cini, and Cesare Alippi. 2026. “Graph State-Space Models and Latent Relational Inference.” no. arXiv:2301.01741. ArXiv, April. <https://doi.org/10.48550/arXiv.2301.01741>.
- Zhao, Hang, Yujing Wang, Juanyong Duan, et al. 2020. “Multivariate Time-series Anomaly Detection via Graph Attention Network.” no. arXiv:2009.02040. ArXiv, September. <https://doi.org/10.48550/arXiv.2009.02040>.
- Zhao, Mengjie, and Olga Fink. 2024. “DyEdgeGAT: Dynamic Edge via Graph Attention for Early Fault Detection in IIoT Systems.” *IEEE Internet*

- of Things Journal* 11 (13): 22950–65. <https://doi.org/10.1109/JIOT.2024.3381002>.
- Zhao, Mengmeng, Haipeng Peng, Lixiang Li, and Yeqing Ren. 2024. “Multivariate Time Series Anomaly Detection Based on Spatial-Temporal Network and Transformer in Industrial Internet of Things.” *Computers, Materials & Continua* 80 (2): 2815–37. <https://doi.org/10.32604/cmc.2024.053765>.
- Zheng et al. 2024. “A Survey of Dynamic Graph Neural Networks.” *Frontiers of Computer Science* 19 (6): 196323. <https://doi.org/10.1007/s11704-024-3853-2>.
- Zheng et al. 2024. “Correlation-Aware Spatial-Temporal Graph Learning for Multivariate Time-series Anomaly Detection.” *IEEE Transactions on Neural Networks and Learning Systems* 35 (9): 11802–16. <https://doi.org/10.1109/TNNLS.2023.3325667>.
- Zhu, Jinlin, Muyun Jiang, and Zhong Liu. 2022. “Fault Detection and Diagnosis in Industrial Processes with Variational Autoencoder: A Comprehensive Study.” *Sensors* 22 (1): 227. <https://doi.org/10.3390/s22010227>.
- Zill, Dennis G., and Michael R. Cullen. 2006. *Advanced Engineering Mathematics*. Jones & Bartlett Learning.

Appendix A

Notable neural networks

A.1 RNN layers

Gated Recurrent Unit (GRU). (Cho et al. 2014)

$$\begin{aligned}
\mathbf{r}_t &= \sigma(W_r \mathbf{x}_t + U_r \mathbf{h}_{t-1} + \mathbf{b}_r) \\
\mathbf{z}_t &= \sigma(W_z \mathbf{x}_t + U_z \mathbf{h}_{t-1} + \mathbf{b}_z) \\
\mathbf{n}_t &= \tanh(W_h \mathbf{x}_t + U_h(\mathbf{r}_t \odot \mathbf{h}_{t-1}) + \mathbf{b}_h) \\
\mathbf{h}_t &= (1 - \mathbf{z}_t) \odot \mathbf{n}_t + \mathbf{z}_t \odot \mathbf{h}_{t-1}
\end{aligned} \tag{A.1}$$

A.2 GNN layers

Graph Convolutional Network (GCN). (Kipf and Welling 2017)

$$\mathbf{Z} = \hat{\mathbf{D}}^{-\frac{1}{2}} \hat{\mathbf{A}} \hat{\mathbf{D}}^{-\frac{1}{2}} \mathbf{X} \Theta \tag{A.2}$$

with $\hat{\mathbf{A}} = \mathbf{A} + \mathbf{I}$ and $\hat{D}_{ii} = \sum_{j=0} \hat{A}_{ij}$.

$$\mathbf{z}^i = \Theta^\top \sum_{j \in \mathcal{N}(i) \cup \{i\}} \frac{e^{j_i}}{\sqrt{\hat{d}_j \hat{d}_i}} \mathbf{x}^j \tag{A.3}$$

with $\hat{d}_i = 1 + \sum_{j \in \mathcal{N}(i)} e^{j_i}$.

Graph Attention Network (GAT). (Veličković et al. 2018; Brody et al. 2022)

$$\mathbf{z}^i = \alpha^{ii} \Theta_s \mathbf{x}^i + \sum_{j \in \mathcal{N}(i)} \alpha^{ji} \Theta_t \mathbf{x}^j \tag{A.4}$$

$$\alpha^{ij} = \frac{\exp(\text{LeakyReLU}(\mathbf{a}_s^\top \Theta_s \mathbf{x}^i + \mathbf{a}_t^\top \Theta_t \mathbf{x}^j))}{\sum_{k \in \mathcal{N}(i) \cup \{i\}} \exp(\text{LeakyReLU}(\mathbf{a}_s^\top \Theta_s \mathbf{x}^i + \mathbf{a}_t^\top \Theta_t \mathbf{x}^k))} \tag{A.5}$$

Graph Isomorphism Network (GIN). (Xu et al. 2019; Hu et al. 2020)

$$\mathbf{z}^i = h_{\Theta} \left((1 + \varepsilon) \mathbf{x}^i + \sum_{j \in \mathcal{N}(i)} \mathbf{x}^j \right) \quad (\text{A.6})$$

Appendix B

Embedding regularization

B.1 Variational regularization

Embeddings vectors are regularized by treating them as parameters of an approximate posterior distribution conditioned on the training data. More precisely, each endogenous embedding is a latent sample drawn from a node-specific multivariate Gaussian

$$\mathbf{v}^i \sim q^i(\mathbf{v}^i \mid \mathcal{D}) = \mathcal{N}(\boldsymbol{\mu}_i, \text{diag}(\boldsymbol{\sigma}_i)) \quad (\text{B.7})$$

with mean and standard deviation being the learnable local parameters. Starting from a standard Gaussian prior and exploiting the reparametrization trick, distributions are fitted to minimize the loss function:

$$\mathcal{L}_{\text{reg}} = \mathbb{E}_{\mathbf{V} \sim Q}[\mathcal{L}] + \beta D_{\text{KL}}(Q \mid \mathcal{N}(\mathbf{0}, \mathbb{I})) \quad (\text{B.8})$$

where D_{KL} is the Kullback-Leibler divergence (Kullback and Leibler 1951), while β controls the regularization strength. This approach results in a smooth latent space, where it is easier to interpolate between representations.

B.2 Clustering regularization

Embeddings vectors are regularized by encouraging them to form clusters in the latent space. Specifically, the module is provided with learnable centroids matrix $\mathbf{C} \in \mathbb{R}^{K \times d_v}$, where $K \ll N$, and cluster assignment matrix $\mathbf{S} \in \mathbb{R}^{N \times K}$, which encodes the probability of each node belonging to each cluster. Considering these scores as logits of a categorical distribution and exploiting the Gumbel-Softmax trick (Jang et al. 2017; Maddison et al. 2017), they are learned by minimizing the loss:

$$\mathcal{L}_{\text{reg}} = \mathcal{L} + \beta \mathbb{E}_M[\|\mathbf{V} - \mathbf{M}\mathbf{C}\|_2], \quad p(M_{ij} = 1) = \frac{\exp(S_{ij}/\tau)}{\sum_{k=1}^K \exp(S_{ik}/\tau)} \quad (\text{B.9})$$

where $M \in \{0, 1\}^{N \times K}$ is a one-hot matrix sampled from the categorical distribution, β controls the regularization strength, and τ is a temperature parameter.

Appendix C

Neural Differential Equations

Differential equations describe the relationship between a function and its derivatives, which is very common in mathematical models and scientific laws. In consequence, they are a versatile tool for modeling dynamic systems, and they have been widely used in various disciplines. We refer to Ordinary Differential Equation (ODE) when the system is deterministic and it depends on a single independent variable. Limiting our scope to a separable first-order scenario, we can formulate an ODE and its solution as

$$\begin{aligned} \mathrm{d}z_t &= F(z_t, t) \mathrm{d}t, \\ z_t &= z_\tau + \int_\tau^t F(z_s, s) \mathrm{d}s. \end{aligned} \tag{C.10}$$

In ML context, ODEs have been adapted to model complex dynamics and capture long-term dependencies in sequential data: with the term of Neural ODEs (NODEs) (Chen et al. 2018), we refer to a family of DL methods that maps a variable of interest to a continuous hidden representation, describes its evolution via the parametrized derivative, and then projects back to the output space. Formally:

$$\mathbf{h}_t = \mathbf{h}_\tau + \int_\tau^t F(\mathbf{h}_s, s, \theta) \mathrm{d}s, \tag{C.11}$$

where $\mathbf{h}$ represents the hidden state, and F is a neural network.

A significant limitation of ODEs is that they are designed to model autonomous systems, meaning that the evolution is deterministically determined by the initial conditions. The consequent extension of this framework has lead to the formulation of Controlled Differential Equations (CDEs), where the direction can be subsequently adjusted by an external factor $X(t)$:

$$\begin{aligned} \mathrm{d}z_t &= F(z_t, t) \mathrm{d}X_t, \\ z_t &= z_\tau + \int_\tau^t F(z_s, s) \mathrm{d}X_s. \end{aligned} \tag{C.12}$$

Analogously, Neural CDEs (NCDEs) (Kidger et al. 2020) are defined as:

$$\boldsymbol{h}_t = \boldsymbol{h}_\tau + \int_\tau^t F(\boldsymbol{h}_s, s, \theta) \mathrm{d}X_s. \tag{C.13}$$

Appendix D

Case studies

D.1 Tennessee Eastman Process

D.1.1 Overview

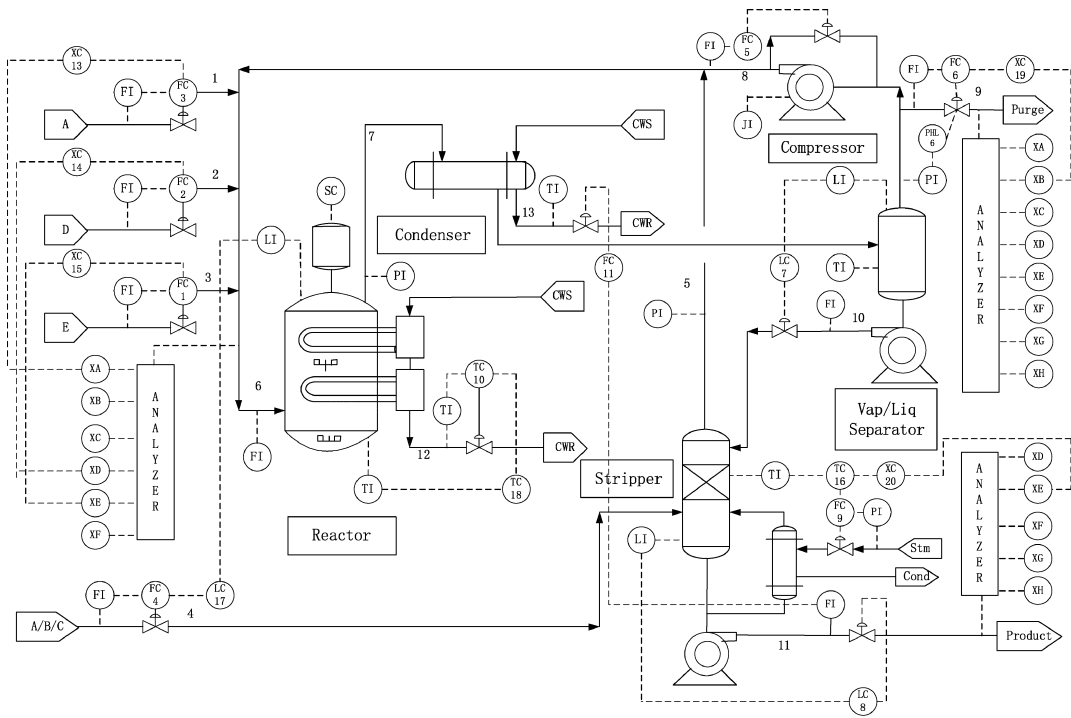


Figure D.13: P&ID diagram of the TEP case study (Li et al. 2011).

Fault ID	Description	Type
1	A/C Feed ratio, B Composition constant	Step
2	B Composition, A/C Ratio constant	Step
3	D Feed temperature	Step
4	Reactor cooling water inlet temperature	Step
5	Condenser cooling water inlet temperature	Step
6	A Feed loss	Step
7	C Header pressure loss – reduced availability	Step
8	A, B, C Feed composition	Random variation
9	D Feed temperature	Random variation
10	C Feed temperature	Random variation
11	Reactor cooling water inlet temperature	Random variation
12	Condenser cooling water inlet temperature	Random variation
13	Reaction kinetics	Slow drift
14	Reactor cooling water valve	Sticking
15	Condenser cooling water valve	Sticking

Table D.5: Faulty scenarios with known causes of the TEP case study.

D.1.2 Maximum explainable variance

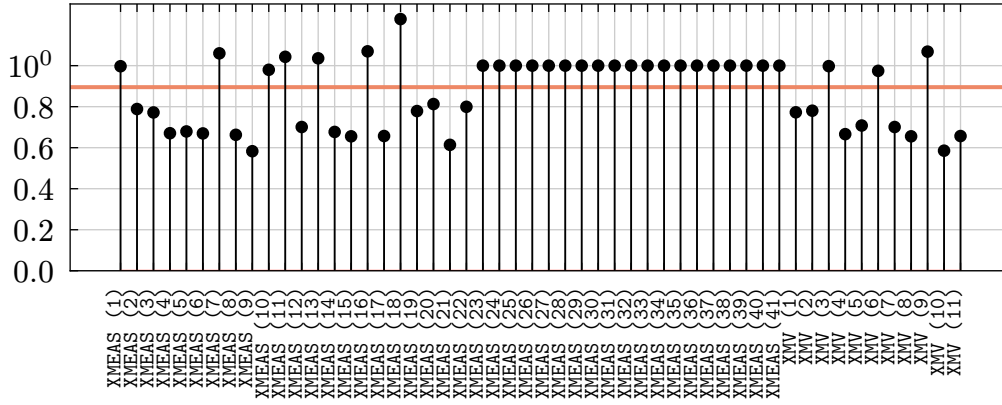


Figure D.14: SNR values for the TEP case study. The horizontal line indicates the mean value of 0.9.

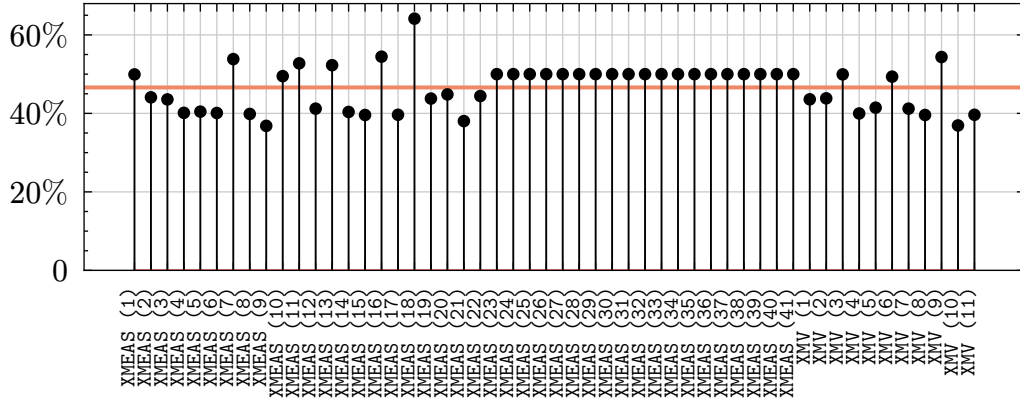


Figure D.15: R^2 values for the TEP case study. The horizontal line indicates the mean value of 46.61%.

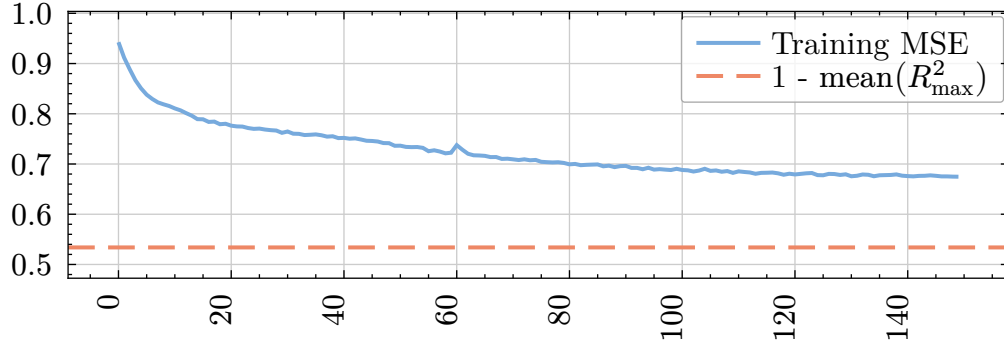


Figure D.16: Training MSE for TEP through early stopping or max epochs. The dashed line indicates the lower bound given by explainable variance.

D.2 Fluid Catalytic Cracking Unit

D.2.1 Overview

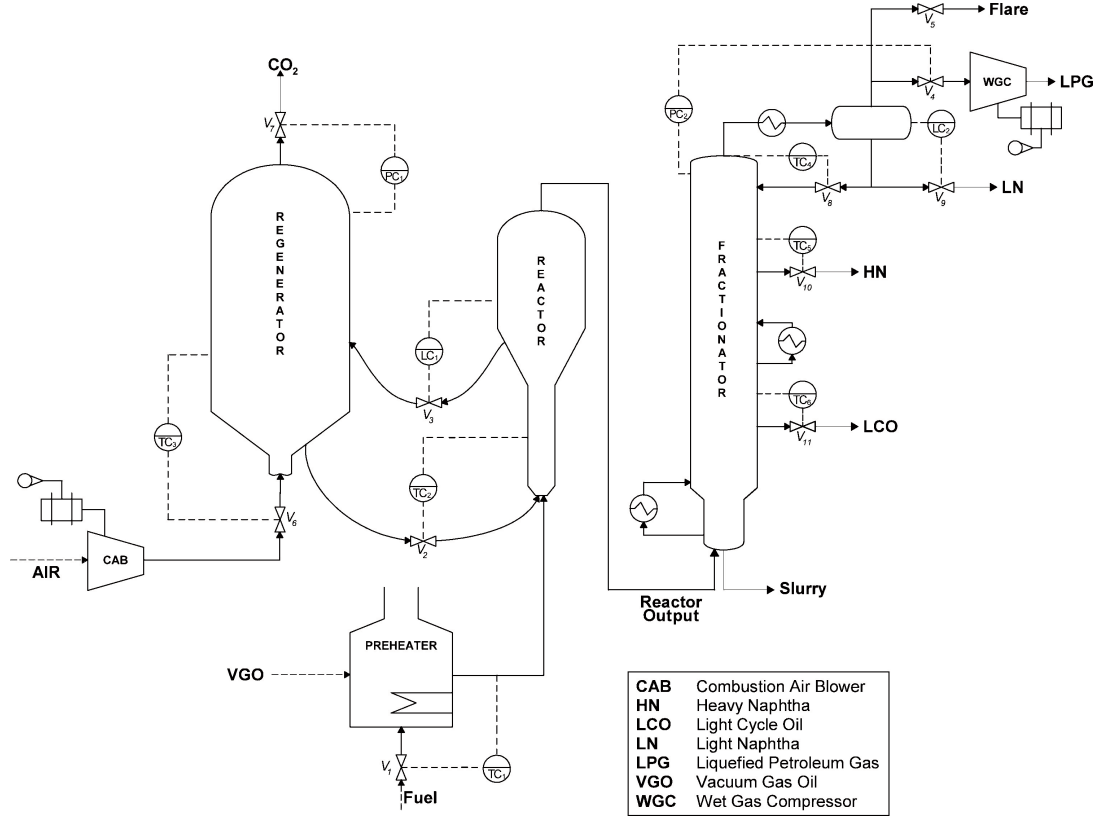


Figure D.17: P&ID diagram of the FCCU case study (Santander et al. 2022).

Controller	CV	Valve	MV
TC_1	T_{pre}	V_1	F_f
TC_2	T_{rea}	V_2	F_{rgc}
LC_1	L_{rea}	V_3	F_{sc}
TC_3	T_{reg}	V_6	F_a
PC_1	P_{reg}	V_7	F_{ig}
PC_2	P_{fra}	V_4	F_{LPG}
LC_2	L_{fra}	V_9	F_{LN}
TC_4	T_{fra}	V_8	F_r
TC_5	T_{hnt}	V_{10}	F_{HN}
TC_6	T_{lco}	V_{11}	F_{LCO}

Table D.6: Closed-loop controllers of the FCCU case study.

Fault ID	Description
deltaP_increase	The differential pressure between reactor and fractionator goes up from a nominal value of 9.5 to 9.9. Increased reactor pressure, among other impacts, leads to increased catalyst circulation rate and deviations in catalyst level.
UAf_decrease	The heat transfer coefficient of the preheater goes down from a nominal value of 25 B.t.u./s to 22.5.
CAB_valveLeak	The variable k_{comb} goes down from a nominal value of 40 to 35. k_{comb} is CAB discharge pipe flow resistance factor. Variations in k_{comb} affects the combustion air flow into the regenerator and therefore regenerator process parameters will show deviations.
condEff_decrease	The fractionator condenser's efficiency goes down from a nominal value of 0.9 to 0.855.
Fhn_sensorDrift	The Fhn sensor shows a drift of 35 lb/min.

Table D.7: Faulty scenarios of the FCCU case study.

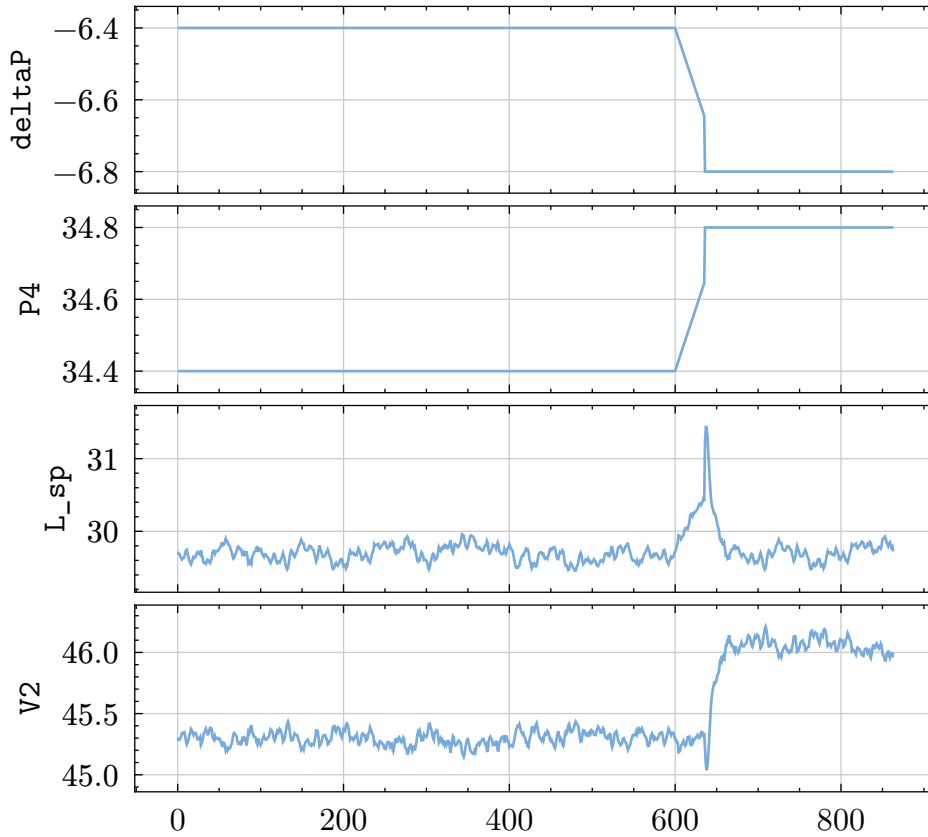


Figure D.18: Some of the anomalous variables impacted by the `deltaP_increase` fault within FCCU case study. Plots arranged hierarchically from root cause to effect.

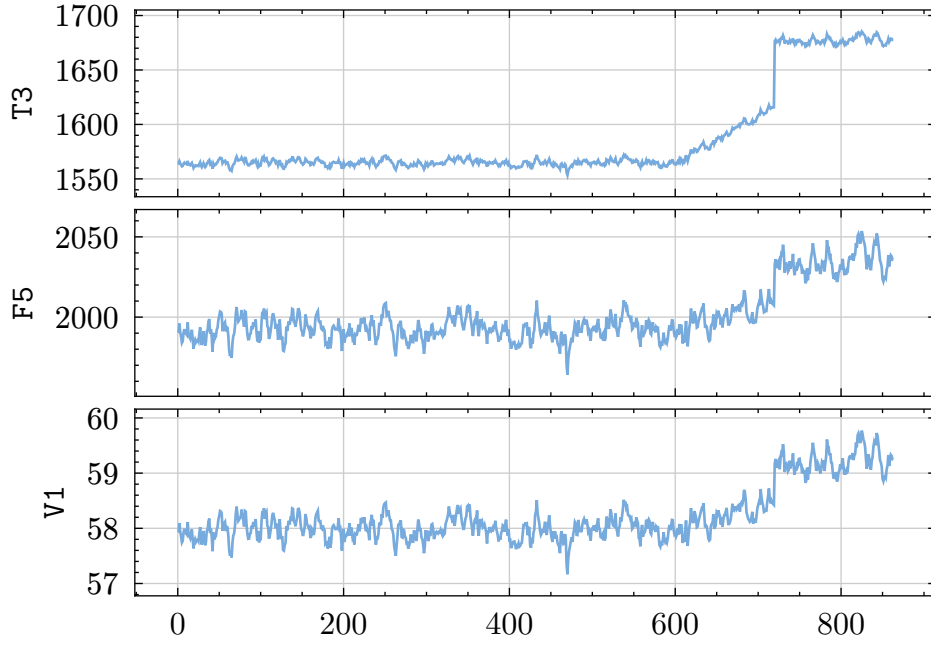


Figure D.19: Some of the anomalous variables impacted by the UAf_decrease fault within FCCU case study. Plots arranged hierarchically from root cause to effect.

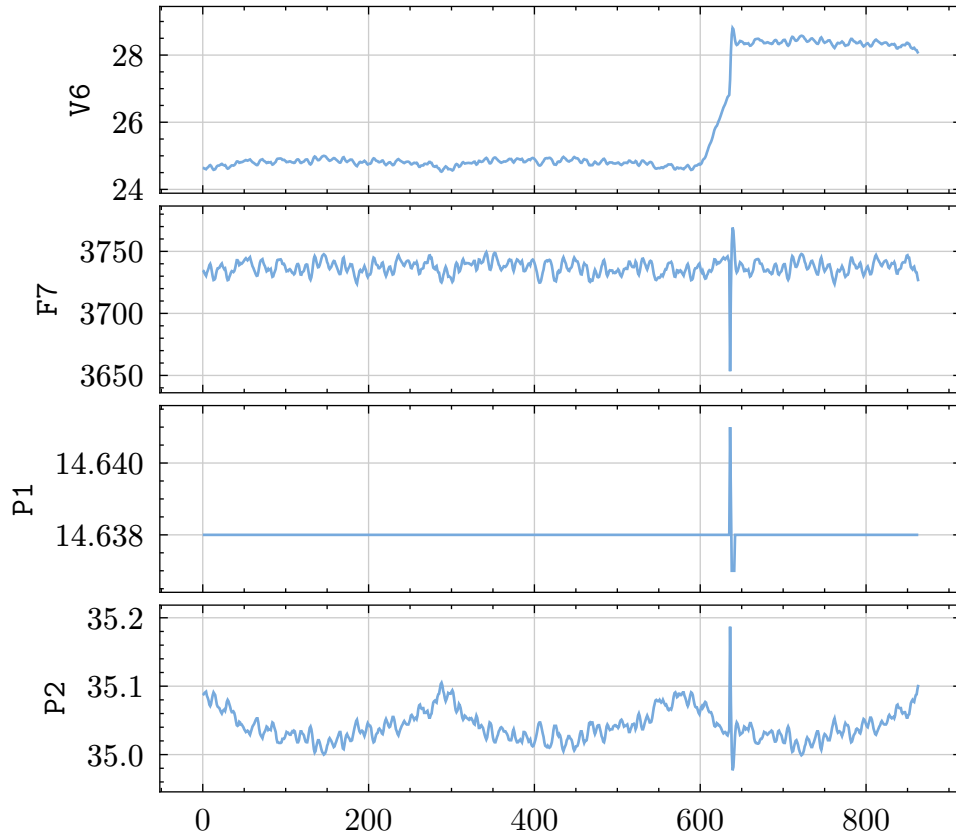


Figure D.20: Some of the anomalous variables impacted by the CAB_valveLeak fault within FCCU case study. Plots arranged hierarchically from root cause to effect.

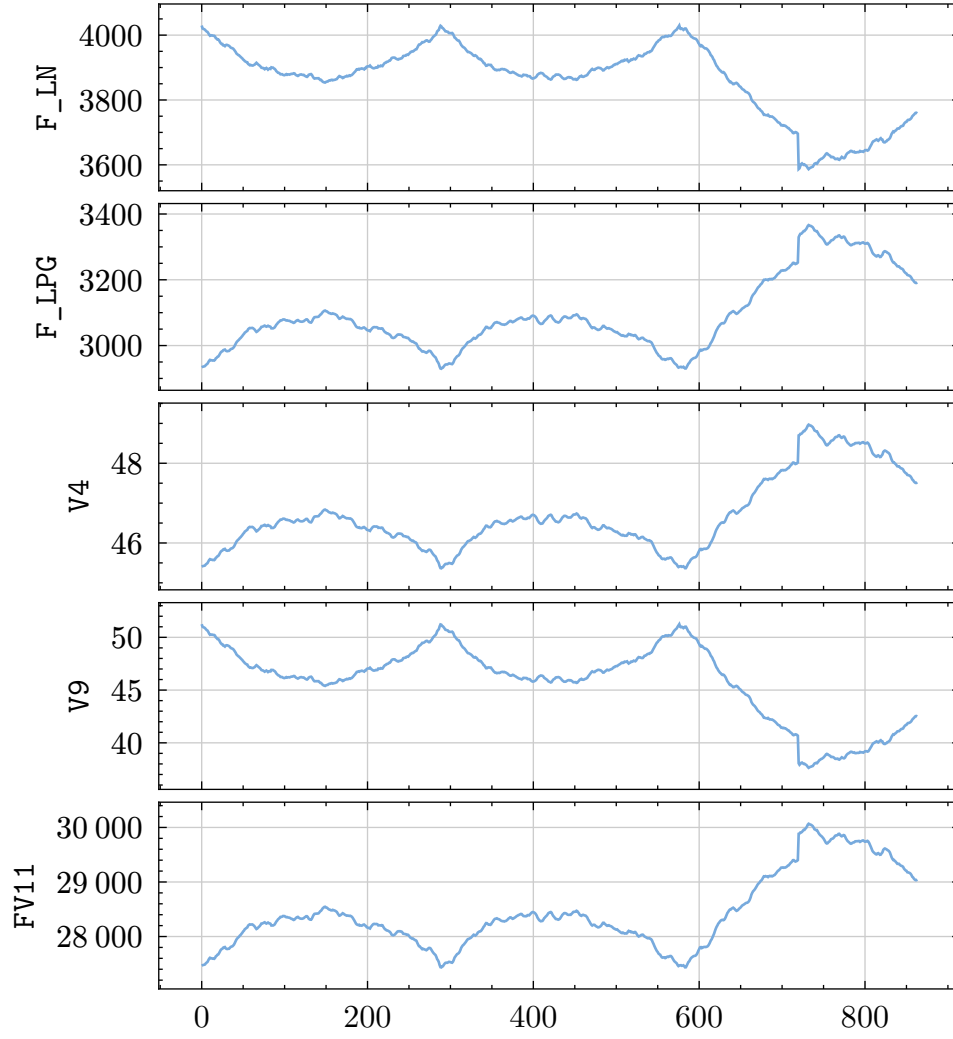


Figure D.21: Some of the anomalous variables impacted by the `condEff_decrease` fault within FCCU case study. Plots arranged hierarchically from root cause to effect.

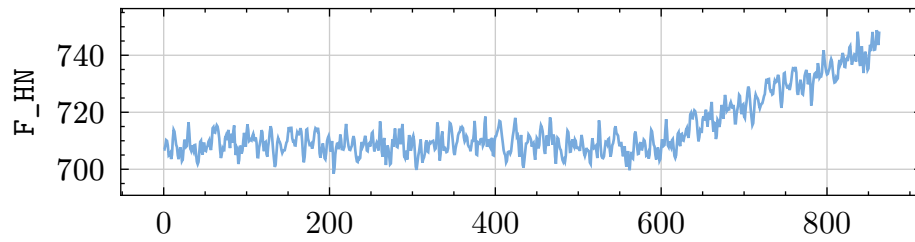


Figure D.22: Some of the anomalous variables impacted by the `Fhn_sensorDrift` fault within FCCU case study. Plots arranged hierarchically from root cause to effect.

D.2.2 System identification

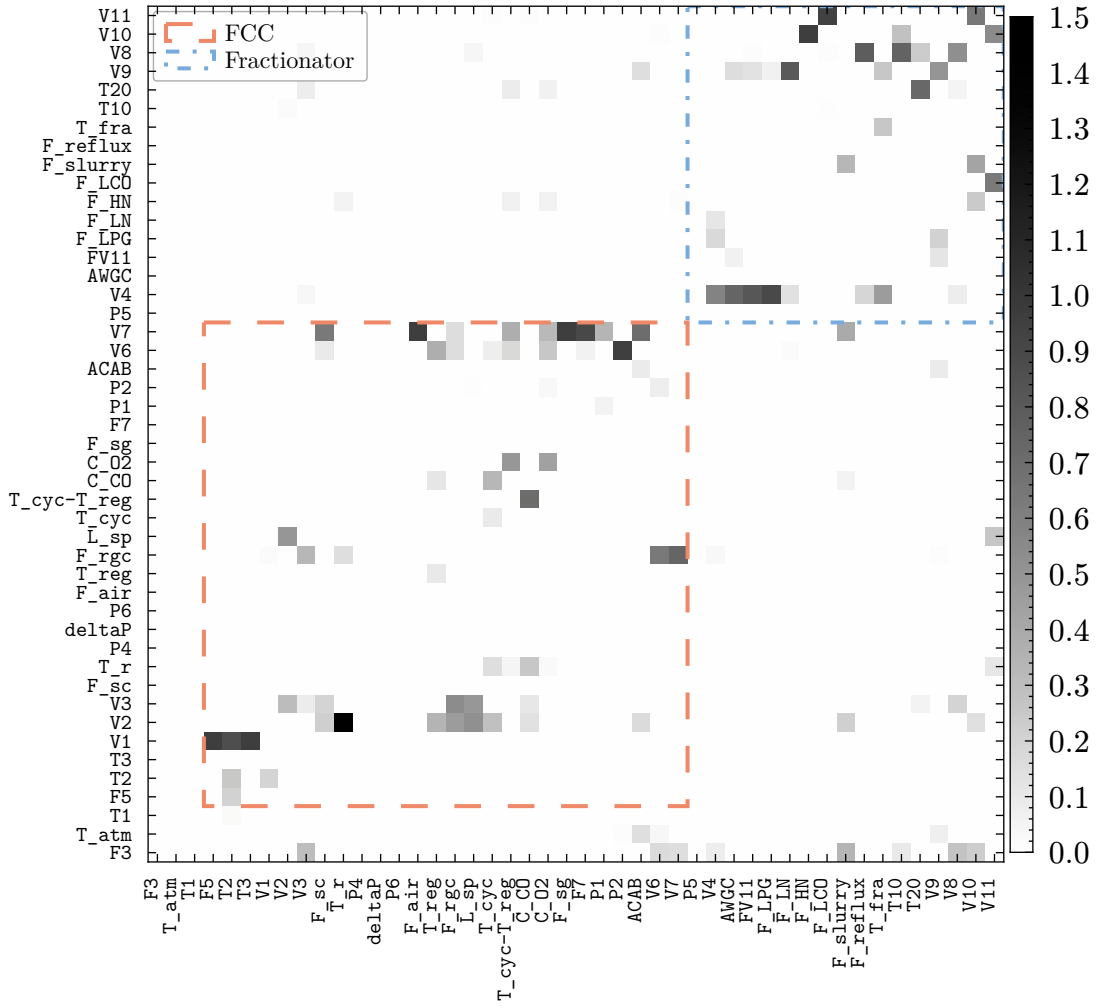


Figure D.23: System identification of the FCCU case study via Lasso regularized SVAR. The two highlighted squares indicate the FCC unit and the fractionator, respectively, showing how well the model captures the physical structure of the plant.

D.2.3 Maximum explainable variance

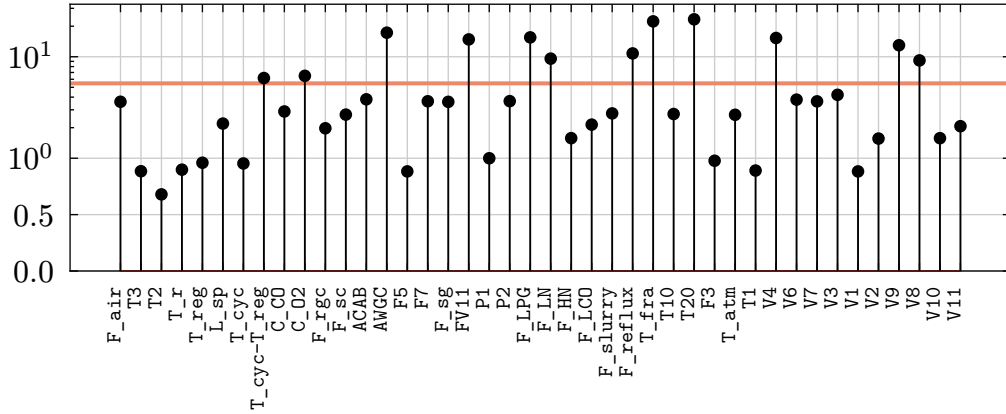


Figure D.24: SNR values for the FCCU case study. The horizontal line indicates the mean value of 5.47.

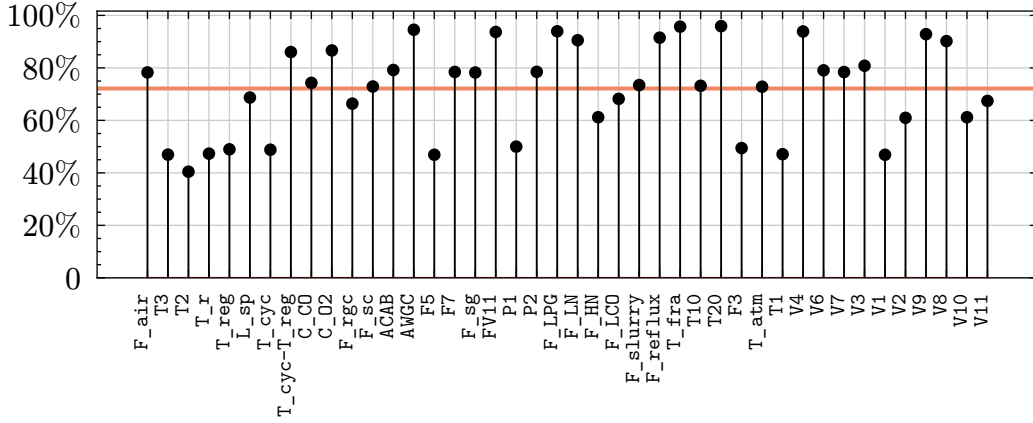


Figure D.25: R^2 values for the FCCU case study. The horizontal line indicates the mean value of 72.14%.

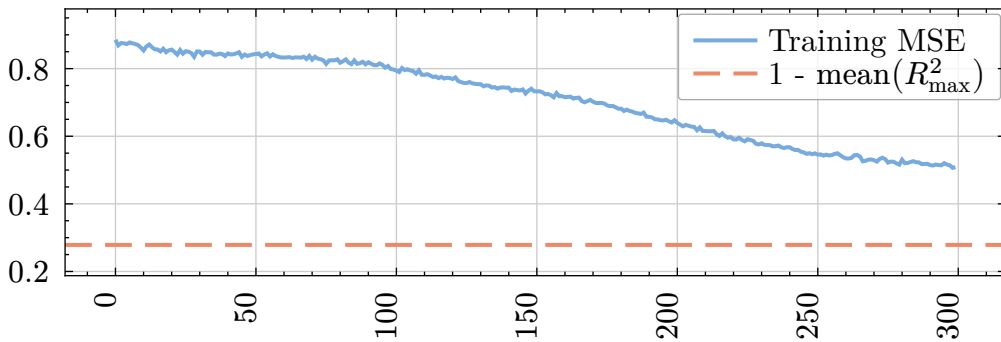


Figure D.26: Training MSE for FCCU through early stopping or max epochs. The dashed line indicates the lower bound given by explainable variance.

Appendix E

Experiments

E.1 Hyperparameter Tuning

Model	Hyperparameter	Values	TEP	FCCU
VAR	lag order	1, 2, 3, 4, 5	2	2
PCA	explained variance	0.9, 0.95, 0.99	0.99	0.99
LSTM	latent dim.	10, 20, 30, 40, 50	40	50
	n. of layers	1, 2	2	2
LSTM-AE	latent dim.	10, 20, 30, 40, 50	40	50
	n. of layers	1, 2	1	1
MTAD-GAT	attention latent dim.	10, 20, 30, 40	10	20
	reconstruction latent dim.	10, 20, 30, 40	10	20
	forecasting latent dim.	10, 20, 30, 40	10	20
	forecasting latent dim.	10, 20, 30, 40	10	10
GDN	latent dim.	10, 50, 100, 200	200	200
	embedding dim.	10, 20, 30, 40, 50	50	20
DyEdgeGAT	latent dim.	10, 20, 30	10	20
	edge dim.	10, 20, 50, 100	20	100
	GNN dim.	10, 20, 50, 100	20	40
	time dim.	2, 5, 10	5	5
IA-STGNN	latent dim.	5, 10, 20	10	20
	embedding dim.	2, 5, 10	5	5
	embedding reg. type	Variational, Clustering	V	C
	embedding reg. strength	1e-3, 1e-2, 1e-1	1e-2	1e-2
	FDI L_1 reg. strength	1e-3, 5e-3, 1e-2	1e-3	1e-2

Table E.8: Hyperparameter tuning for all models, with best values reported for each dataset. For IA-STGNN, the regularization strength of the FDI stage was calibrated on the validation partition, such that deviations remained within a reasonably small range from nominal values.

E.2 Tennessee Eastman Process

E.2.1 Anomaly Detection Results

	Accuracy	Precision	Recall	F1	AUC
VAR	0.84 ± 0.00	<u>0.84 ± 0.00</u>	1.00 ± 0.00	<u>0.91 ± 0.00</u>	0.53 ± 0.00
PCA	<u>0.95</u>	0.72	1.00	0.83	1.00
LSTM	0.93 ± 0.01	0.65 ± 0.04	1.00 ± 0.00	0.79 ± 0.03	1.00 ± 0.00
LSTM-AE	0.92 ± 0.02	0.61 ± 0.06	1.00 ± 0.00	0.75 ± 0.05	1.00 ± 0.00
MTAD-GAT	0.89 ± 0.01	0.53 ± 0.03	1.00 ± 0.00	0.69 ± 0.03	<u>1.00 ± 0.00</u>
GDN	0.87 ± 0.00	0.48 ± 0.01	1.00 ± 0.00	0.65 ± 0.01	1.00 ± 0.00
DyEdgeGAT	0.89 ± 0.02	0.78 ± 0.05	0.84 ± 0.01	0.81 ± 0.03	0.92 ± 0.01
IA-STGNN	0.97 ± 0.01	0.98 ± 0.02	0.98 ± 0.00	0.98 ± 0.01	0.94 ± 0.05

Table E.9: TEP AD metrics for fault 1. Best values in bold, second best underlined.

	Accuracy	Precision	Recall	F1	AUC
VAR	0.84 ± 0.00	<u>0.84 ± 0.00</u>	1.00 ± 0.00	<u>0.91 ± 0.00</u>	0.04 ± 0.00
PCA	<u>0.95</u>	0.71	0.99	0.83	0.99
LSTM	0.93 ± 0.01	0.65 ± 0.04	0.99 ± 0.00	0.78 ± 0.03	<u>1.00 ± 0.00</u>
LSTM-AE	0.92 ± 0.02	0.60 ± 0.06	0.99 ± 0.00	0.75 ± 0.05	0.99 ± 0.00
MTAD-GAT	0.89 ± 0.01	0.53 ± 0.03	<u>0.99 ± 0.00</u>	0.69 ± 0.03	1.00 ± 0.00
GDN	0.87 ± 0.00	0.48 ± 0.01	0.99 ± 0.00	0.65 ± 0.01	0.99 ± 0.00
DyEdgeGAT	0.88 ± 0.02	0.78 ± 0.06	0.83 ± 0.00	0.80 ± 0.03	0.90 ± 0.01
IA-STGNN	0.97 ± 0.01	0.98 ± 0.02	0.98 ± 0.01	0.98 ± 0.01	0.94 ± 0.05

Table E.10: TEP AD metrics for fault 2. Best values in bold, second best underlined.

	Accuracy	Precision	Recall	F1	AUC
VAR	<u>0.84 ± 0.00</u>	<u>0.84 ± 0.00</u>	1.00 ± 0.00	0.91 ± 0.00	0.43 ± 0.00
PCA	0.84	0.17	0.08	0.11	<u>0.51</u>
LSTM	0.82 ± 0.01	0.11 ± 0.00	0.07 ± 0.01	0.08 ± 0.01	0.45 ± 0.01
LSTM-AE	0.80 ± 0.02	0.10 ± 0.01	0.08 ± 0.02	0.09 ± 0.01	0.47 ± 0.00
MTAD-GAT	0.78 ± 0.01	0.12 ± 0.01	0.12 ± 0.02	0.12 ± 0.01	0.49 ± 0.00
GDN	0.76 ± 0.01	0.09 ± 0.01	0.10 ± 0.01	0.09 ± 0.01	0.45 ± 0.02
DyEdgeGAT	0.67 ± 0.02	0.16 ± 0.05	0.05 ± 0.02	0.07 ± 0.03	0.48 ± 0.01
IA-STGNN	0.69 ± 0.03	0.97 ± 0.03	<u>0.65 ± 0.04</u>	<u>0.78 ± 0.03</u>	0.78 ± 0.05

Table E.11: TEP AD metrics for fault 3. Best values in bold, second best underlined.

	Accuracy	Precision	Recall	F1	AUC
VAR	0.84 ± 0.00	<u>0.84 ± 0.00</u>	1.00 ± 0.00	0.91 ± 0.00	0.36 ± 0.00
PCA	0.94	0.70	0.91	<u>0.79</u>	0.98
LSTM	<u>0.93 ± 0.01</u>	0.65 ± 0.03	<u>0.99 ± 0.00</u>	0.78 ± 0.02	0.99 ± 0.00
LSTM-AE	0.90 ± 0.02	0.58 ± 0.06	0.88 ± 0.02	0.69 ± 0.04	0.96 ± 0.00
MTAD-GAT	0.88 ± 0.01	0.52 ± 0.03	0.95 ± 0.03	0.67 ± 0.03	<u>0.98 ± 0.02</u>
GDN	0.81 ± 0.02	0.32 ± 0.07	0.52 ± 0.16	0.40 ± 0.10	0.81 ± 0.06
DyEdgeGAT	0.75 ± 0.10	0.46 ± 0.25	0.36 ± 0.34	0.38 ± 0.32	0.74 ± 0.13
IA-STGNN	0.69 ± 0.02	0.97 ± 0.03	0.65 ± 0.02	0.78 ± 0.01	0.78 ± 0.05

Table E.12: TEP AD metrics for fault 4. Best values in bold, second best underlined.

	Accuracy	Precision	Recall	F1	AUC
VAR	0.84 ± 0.00	<u>0.84 ± 0.00</u>	1.00 ± 0.00	0.91 ± 0.00	<u>1.00 ± 0.00</u>
PCA	0.95	0.72	<u>1.00</u>	0.83	1.00
LSTM	<u>0.87 ± 0.01</u>	0.47 ± 0.03	0.47 ± 0.02	0.47 ± 0.01	0.81 ± 0.01
LSTM-AE	0.84 ± 0.02	0.39 ± 0.05	0.41 ± 0.03	0.40 ± 0.02	0.72 ± 0.03
MTAD-GAT	0.82 ± 0.02	0.32 ± 0.04	0.42 ± 0.02	0.36 ± 0.03	0.69 ± 0.04
GDN	0.79 ± 0.01	0.24 ± 0.01	0.34 ± 0.02	0.29 ± 0.01	0.62 ± 0.02
DyEdgeGAT	0.74 ± 0.03	0.58 ± 0.11	0.32 ± 0.03	0.41 ± 0.06	0.70 ± 0.07
IA-STGNN	0.79 ± 0.01	0.98 ± 0.02	0.77 ± 0.02	<u>0.86 ± 0.01</u>	0.84 ± 0.04

Table E.13: TEP AD metrics for fault 5. Best values in bold, second best underlined.

	Accuracy	Precision	Recall	F1	AUC
VAR	0.84 ± 0.00	<u>0.84 ± 0.00</u>	1.00 ± 0.00	<u>0.91 ± 0.00</u>	0.98 ± 0.00
PCA	<u>0.95</u>	0.72	1.00	0.83	1.00
LSTM	0.93 ± 0.01	0.65 ± 0.03	1.00 ± 0.00	0.79 ± 0.03	1.00 ± 0.00
LSTM-AE	0.92 ± 0.02	0.61 ± 0.06	1.00 ± 0.00	0.75 ± 0.05	1.00 ± 0.00
MTAD-GAT	0.89 ± 0.01	0.53 ± 0.03	1.00 ± 0.00	0.69 ± 0.03	1.00 ± 0.00
GDN	0.87 ± 0.00	0.48 ± 0.01	1.00 ± 0.00	0.65 ± 0.01	1.00 ± 0.00
DyEdgeGAT	0.89 ± 0.02	0.78 ± 0.05	0.84 ± 0.01	0.81 ± 0.03	0.91 ± 0.01
IA-STGNN	0.97 ± 0.01	0.98 ± 0.02	0.98 ± 0.00	0.98 ± 0.01	0.95 ± 0.05

Table E.14: TEP AD metrics for fault 6. Best values in bold, second best underlined.

	Accuracy	Precision	Recall	F1	AUC
VAR	0.84 ± 0.00	<u>0.84 ± 0.00</u>	1.00 ± 0.00	<u>0.91 ± 0.00</u>	0.89 ± 0.00
PCA	0.87	0.43	0.30	0.36	0.68
LSTM	0.93 ± 0.01	0.65 ± 0.04	1.00 ± 0.00	0.78 ± 0.03	<u>1.00 ± 0.00</u>
LSTM-AE	<u>0.92 ± 0.02</u>	0.61 ± 0.06	1.00 ± 0.00	0.75 ± 0.05	1.00 ± 0.00
MTAD-GAT	0.89 ± 0.01	0.53 ± 0.03	1.00 ± 0.00	0.69 ± 0.03	1.00 ± 0.00
GDN	0.85 ± 0.01	0.45 ± 0.03	0.88 ± 0.09	0.60 ± 0.04	0.95 ± 0.03
DyEdgeGAT	0.78 ± 0.01	0.66 ± 0.02	0.46 ± 0.12	0.53 ± 0.08	0.80 ± 0.04
IA-STGNN	0.87 ± 0.01	0.98 ± 0.02	0.86 ± 0.02	0.91 ± 0.01	0.88 ± 0.04

Table E.15: TEP AD metrics for fault 7. Best values in bold, second best underlined.

	Accuracy	Precision	Recall	F1	AUC
VAR	0.84 ± 0.00	<u>0.84 ± 0.00</u>	1.00 ± 0.00	<u>0.91 ± 0.00</u>	0.61 ± 0.00
PCA	<u>0.94</u>	0.70	0.93	0.80	0.98
LSTM	0.93 ± 0.01	0.65 ± 0.04	<u>0.98 ± 0.00</u>	0.78 ± 0.03	0.99 ± 0.00
LSTM-AE	0.91 ± 0.02	0.60 ± 0.06	0.98 ± 0.00	0.75 ± 0.05	0.99 ± 0.00
MTAD-GAT	0.89 ± 0.01	0.53 ± 0.03	0.98 ± 0.00	0.69 ± 0.03	<u>0.99 ± 0.00</u>
GDN	0.87 ± 0.00	0.48 ± 0.01	0.98 ± 0.00	0.64 ± 0.01	0.99 ± 0.00
DyEdgeGAT	0.88 ± 0.02	0.78 ± 0.06	0.83 ± 0.01	0.80 ± 0.03	0.91 ± 0.01
IA-STGNN	0.96 ± 0.01	0.98 ± 0.02	0.97 ± 0.01	0.98 ± 0.00	0.94 ± 0.04

Table E.16: TEP AD metrics for fault 8. Best values in bold, second best underlined.

	Accuracy	Precision	Recall	F1	AUC
VAR	<u>0.84 ± 0.00</u>	<u>0.84 ± 0.00</u>	1.00 ± 0.00	0.91 ± 0.00	0.44 ± 0.00
PCA	0.84	0.18	0.09	0.11	<u>0.51</u>
LSTM	0.82 ± 0.01	0.11 ± 0.01	0.07 ± 0.01	0.09 ± 0.00	0.46 ± 0.01
LSTM-AE	0.80 ± 0.02	0.11 ± 0.01	0.08 ± 0.02	0.09 ± 0.01	0.48 ± 0.00
MTAD-GAT	0.78 ± 0.01	0.12 ± 0.01	0.12 ± 0.03	0.12 ± 0.02	0.50 ± 0.01
GDN	0.76 ± 0.01	0.09 ± 0.01	0.10 ± 0.01	0.10 ± 0.01	0.46 ± 0.02
DyEdgeGAT	0.67 ± 0.02	0.16 ± 0.03	0.05 ± 0.02	0.07 ± 0.03	0.49 ± 0.01
IA-STGNN	0.70 ± 0.02	0.98 ± 0.02	<u>0.66 ± 0.03</u>	<u>0.78 ± 0.02</u>	0.78 ± 0.05

Table E.17: TEP AD metrics for fault 9. Best values in bold, second best underlined.

	Accuracy	Precision	Recall	F1	AUC
VAR	0.84 ± 0.00	<u>0.84 ± 0.00</u>	1.00 ± 0.00	<u>0.91 ± 0.00</u>	0.49 ± 0.00
PCA	0.88	0.52	0.43	0.47	0.83
LSTM	<u>0.89 ± 0.01</u>	0.54 ± 0.03	0.64 ± 0.03	0.59 ± 0.01	0.86 ± 0.00
LSTM-AE	0.88 ± 0.02	0.52 ± 0.06	0.69 ± 0.02	0.59 ± 0.03	<u>0.87 ± 0.00</u>
MTAD-GAT	0.84 ± 0.01	0.40 ± 0.02	0.59 ± 0.03	0.48 ± 0.01	0.81 ± 0.01
GDN	0.81 ± 0.00	0.35 ± 0.01	0.56 ± 0.00	0.43 ± 0.00	0.79 ± 0.00
DyEdgeGAT	0.73 ± 0.01	0.53 ± 0.04	0.27 ± 0.07	0.35 ± 0.07	0.67 ± 0.03
IA-STGNN	0.96 ± 0.01	0.98 ± 0.02	<u>0.96 ± 0.00</u>	0.97 ± 0.01	0.94 ± 0.05

Table E.18: TEP AD metrics for fault 10. Best values in bold, second best underlined.

	Accuracy	Precision	Recall	F1	AUC
VAR	0.84 ± 0.00	<u>0.84 ± 0.00</u>	1.00 ± 0.00	<u>0.91 ± 0.00</u>	0.46 ± 0.00
PCA	0.90	0.57	0.53	0.55	0.79
LSTM	<u>0.91 ± 0.01</u>	0.60 ± 0.03	0.79 ± 0.03	0.68 ± 0.01	<u>0.93 ± 0.01</u>
LSTM-AE	0.88 ± 0.02	0.52 ± 0.06	0.69 ± 0.02	0.59 ± 0.03	0.88 ± 0.00
MTAD-GAT	0.86 ± 0.01	0.48 ± 0.02	0.80 ± 0.07	0.59 ± 0.01	0.91 ± 0.02
GDN	0.81 ± 0.01	0.33 ± 0.01	0.52 ± 0.03	0.40 ± 0.02	0.77 ± 0.01
DyEdgeGAT	0.72 ± 0.01	0.49 ± 0.04	0.24 ± 0.07	0.32 ± 0.07	0.68 ± 0.04
IA-STGNN	0.97 ± 0.01	0.98 ± 0.02	<u>0.98 ± 0.00</u>	0.98 ± 0.01	0.94 ± 0.05

Table E.19: TEP AD metrics for fault 11. Best values in bold, second best underlined.

	Accuracy	Precision	Recall	F1	AUC
VAR	0.84 ± 0.00	<u>0.84 ± 0.00</u>	1.00 ± 0.00	<u>0.91 ± 0.00</u>	0.76 ± 0.00
PCA	<u>0.95</u>	0.71	0.98	0.82	0.99
LSTM	0.93 ± 0.01	0.65 ± 0.04	0.99 ± 0.00	0.79 ± 0.03	<u>1.00 ± 0.00</u>
LSTM-AE	0.92 ± 0.02	0.61 ± 0.06	0.99 ± 0.00	0.75 ± 0.05	1.00 ± 0.00
MTAD-GAT	0.89 ± 0.01	0.53 ± 0.03	<u>1.00 ± 0.00</u>	0.69 ± 0.03	1.00 ± 0.00
GDN	0.87 ± 0.00	0.48 ± 0.01	1.00 ± 0.00	0.65 ± 0.01	1.00 ± 0.00
DyEdgeGAT	0.89 ± 0.02	0.78 ± 0.05	0.83 ± 0.01	0.80 ± 0.03	0.92 ± 0.01
IA-STGNN	0.97 ± 0.02	0.98 ± 0.02	0.98 ± 0.00	0.98 ± 0.01	0.94 ± 0.05

Table E.20: TEP AD metrics for fault 12. Best values in bold, second best underlined.

E Experiments

	Accuracy	Precision	Recall	F1	AUC
VAR	0.84 ± 0.00	<u>0.84 ± 0.00</u>	1.00 ± 0.00	<u>0.91 ± 0.00</u>	0.60 ± 0.00
PCA	<u>0.94</u>	0.70	0.94	0.80	0.97
LSTM	0.93 ± 0.01	0.64 ± 0.04	0.94 ± 0.00	0.76 ± 0.02	0.97 ± 0.00
LSTM-AE	0.91 ± 0.02	0.59 ± 0.06	0.95 ± 0.01	0.73 ± 0.05	<u>0.97 ± 0.00</u>
MTAD-GAT	0.88 ± 0.01	0.52 ± 0.03	0.96 ± 0.00	0.67 ± 0.03	0.98 ± 0.00
GDN	0.86 ± 0.01	0.47 ± 0.01	0.94 ± 0.01	0.62 ± 0.01	0.97 ± 0.00
DyEdgeGAT	0.88 ± 0.02	0.77 ± 0.06	0.80 ± 0.01	0.78 ± 0.03	0.89 ± 0.01
IA-STGNN	0.95 ± 0.02	0.98 ± 0.02	<u>0.96 ± 0.01</u>	0.97 ± 0.01	0.93 ± 0.05

Table E.21: TEP AD metrics for fault 13. Best values in bold, second best underlined.

	Accuracy	Precision	Recall	F1	AUC
VAR	0.84 ± 0.00	<u>0.84 ± 0.00</u>	1.00 ± 0.00	<u>0.91 ± 0.00</u>	0.44 ± 0.00
PCA	<u>0.94</u>	0.70	0.92	0.79	0.97
LSTM	0.93 ± 0.01	0.64 ± 0.04	1.00 ± 0.00	0.78 ± 0.03	1.00 ± 0.00
LSTM-AE	0.92 ± 0.02	0.61 ± 0.06	1.00 ± 0.00	0.75 ± 0.05	<u>1.00 ± 0.00</u>
MTAD-GAT	0.89 ± 0.01	0.53 ± 0.03	1.00 ± 0.00	0.69 ± 0.03	1.00 ± 0.00
GDN	0.87 ± 0.00	0.48 ± 0.01	1.00 ± 0.00	0.65 ± 0.01	1.00 ± 0.00
DyEdgeGAT	0.83 ± 0.06	0.71 ± 0.07	0.65 ± 0.27	0.65 ± 0.20	0.88 ± 0.04
IA-STGNN	0.97 ± 0.02	0.98 ± 0.02	0.99 ± 0.00	0.98 ± 0.01	0.95 ± 0.05

Table E.22: TEP AD metrics for fault 14. Best values in bold, second best underlined.

	Accuracy	Precision	Recall	F1	AUC
VAR	<u>0.84 ± 0.00</u>	<u>0.84 ± 0.00</u>	1.00 ± 0.00	0.91 ± 0.00	0.45 ± 0.00
PCA	0.84	0.16	0.08	0.10	<u>0.54</u>
LSTM	0.82 ± 0.01	0.12 ± 0.00	0.07 ± 0.01	0.09 ± 0.01	0.46 ± 0.01
LSTM-AE	0.80 ± 0.02	0.12 ± 0.02	0.09 ± 0.00	0.10 ± 0.01	0.49 ± 0.00
MTAD-GAT	0.78 ± 0.01	0.12 ± 0.01	0.12 ± 0.03	0.12 ± 0.02	0.50 ± 0.01
GDN	0.76 ± 0.00	0.09 ± 0.01	0.11 ± 0.01	0.10 ± 0.01	0.46 ± 0.02
DyEdgeGAT	0.67 ± 0.02	0.17 ± 0.03	0.05 ± 0.02	0.08 ± 0.03	0.50 ± 0.00
IA-STGNN	0.71 ± 0.03	0.98 ± 0.02	<u>0.68 ± 0.03</u>	<u>0.80 ± 0.02</u>	0.79 ± 0.05

Table E.23: TEP AD metrics for fault 15. Best values in bold, second best underlined.

E.2.2 Fault Isolation Results

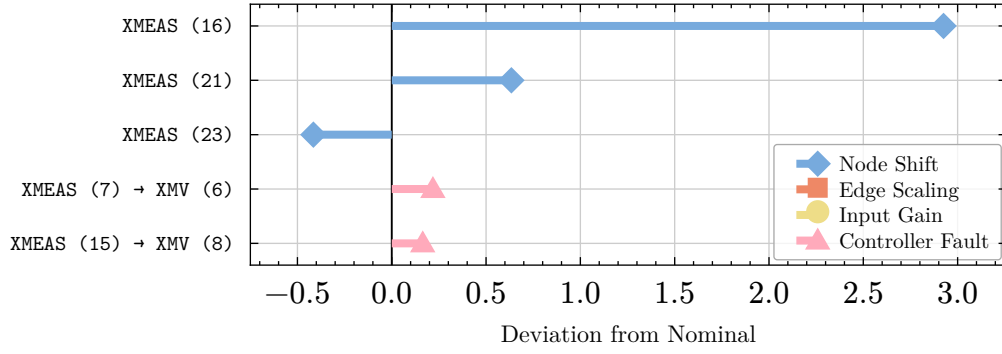


Figure E.27: TEP diagnosis for fault 1.

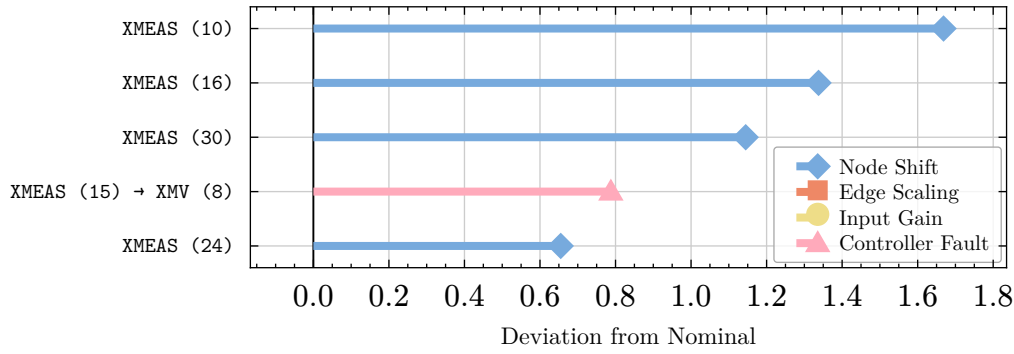


Figure E.28: TEP diagnosis for fault 2.

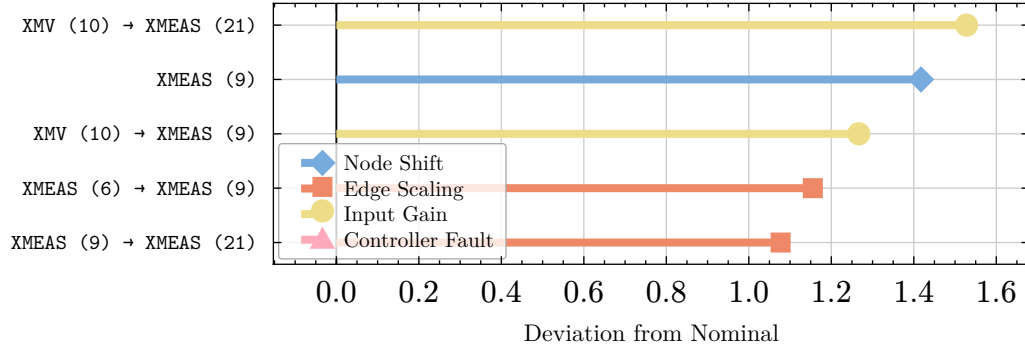


Figure E.29: TEP diagnosis for fault 4.

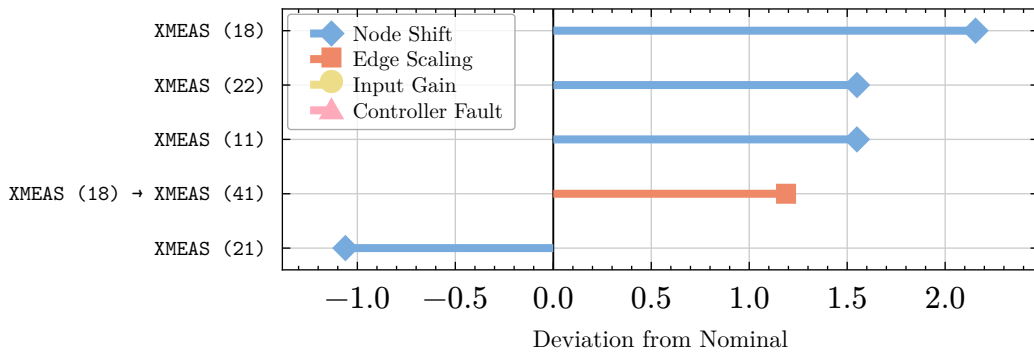


Figure E.30: TEP diagnosis for fault 5.

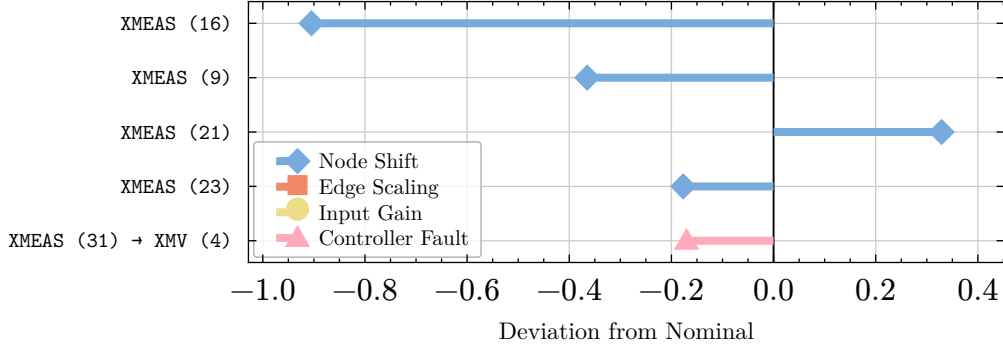


Figure E.31: TEP diagnosis for fault 6.

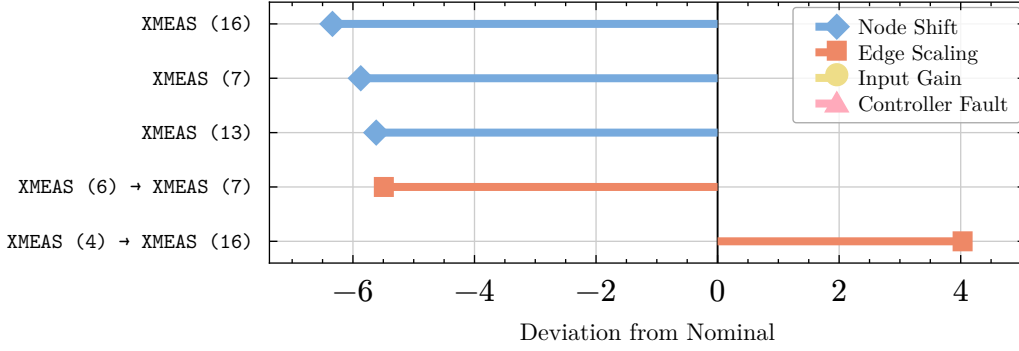


Figure E.32: TEP diagnosis for fault 7.

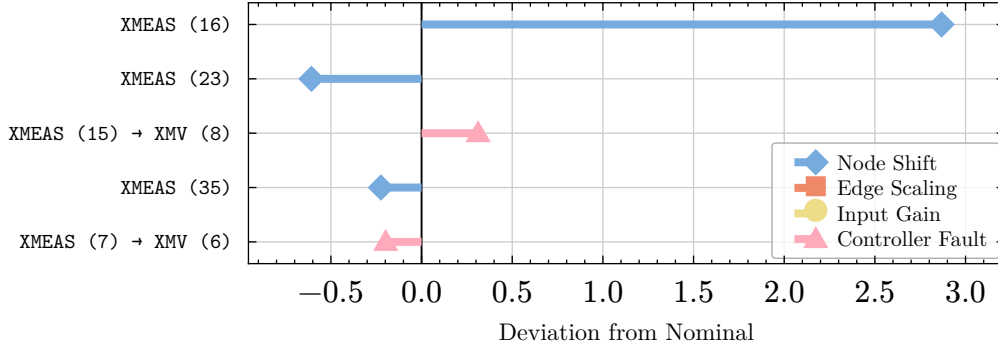


Figure E.33: TEP diagnosis for fault 8.

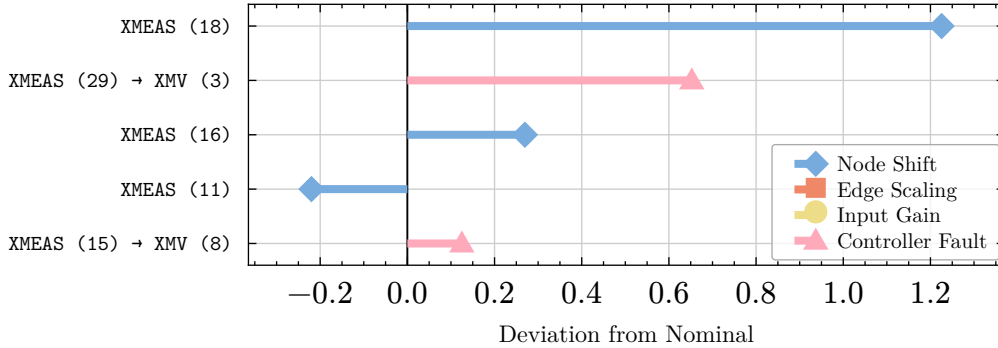


Figure E.34: TEP diagnosis for fault 10.

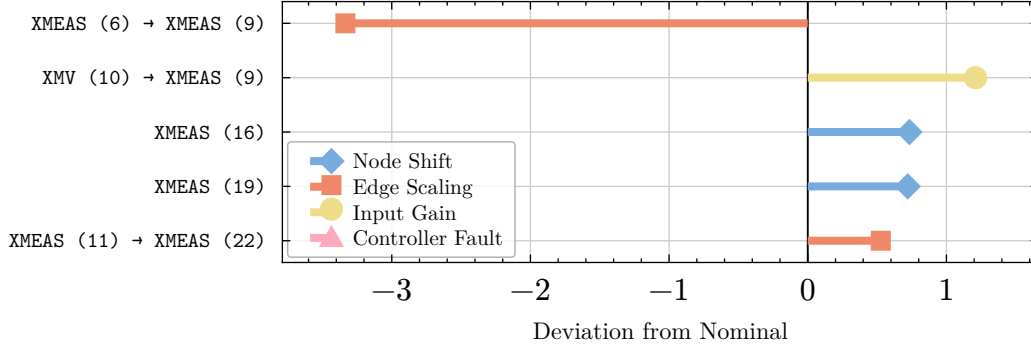


Figure E.35: TEP diagnosis for fault 11.

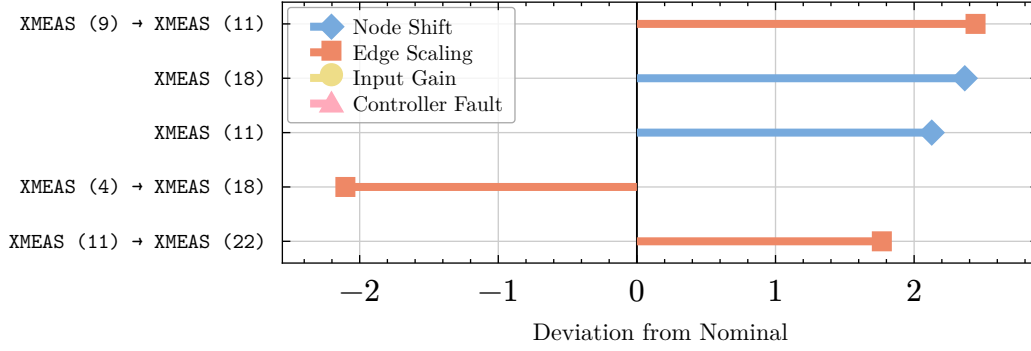


Figure E.36: TEP diagnosis for fault 12.

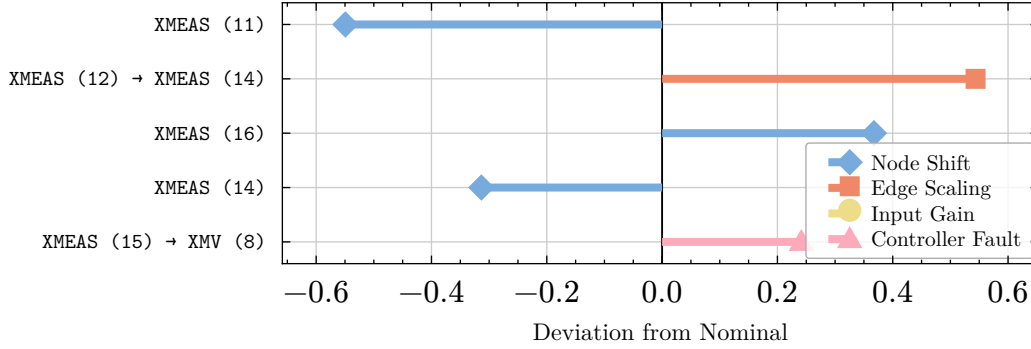


Figure E.37: TEP diagnosis for fault 13.

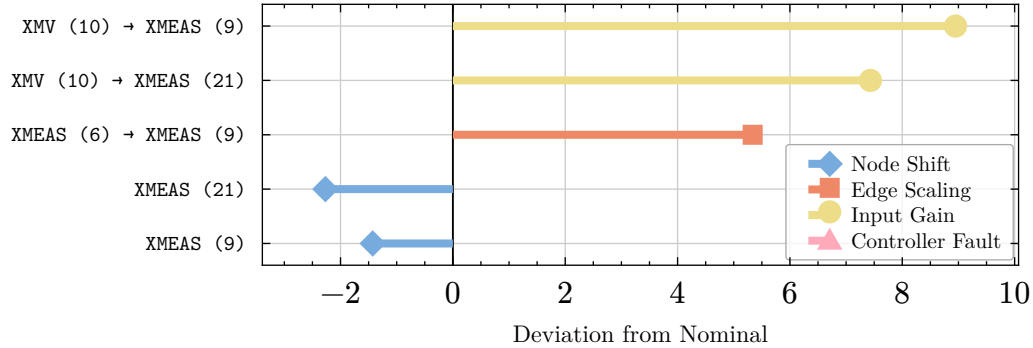


Figure E.38: TEP diagnosis for fault 14.

E.3 Fluid Catalytic Craking Unit

E.3.1 Anomaly Detection Results

	Accuracy	Precision	Recall	F1	AUC
VAR	0.65 ± 0.00	0.87 ± 0.00	<u>0.52 ± 0.00</u>	0.65 ± 0.00	0.72 ± 0.00
PCA	0.81	0.71	0.49	0.58	0.73
LSTM	0.84 ± 0.01	0.88 ± 0.08	0.46 ± 0.00	0.60 ± 0.02	0.67 ± 0.04
LSTM-AE	<u>0.86 ± 0.00</u>	<u>0.97 ± 0.03</u>	0.49 ± 0.00	<u>0.65 ± 0.01</u>	<u>0.76 ± 0.00</u>
MTAD-GAT	0.86 ± 0.00	1.00 ± 0.00	0.47 ± 0.01	0.63 ± 0.01	0.74 ± 0.00
GDN	0.85 ± 0.00	0.96 ± 0.03	0.47 ± 0.01	0.63 ± 0.00	0.75 ± 0.01
DyEdgeGAT	0.70 ± 0.04	0.37 ± 0.11	0.24 ± 0.16	0.28 ± 0.15	0.49 ± 0.12
IA-STGNN	0.86 ± 0.01	0.87 ± 0.01	0.92 ± 0.01	0.90 ± 0.01	0.84 ± 0.02

Table E.24: FCCU AD metrics for fault CAB_valveLeak. Best values in bold, second best underlined.

	Accuracy	Precision	Recall	F1	AUC
VAR	0.63 ± 0.00	0.91 ± 0.00	0.48 ± 0.00	0.63 ± 0.00	0.71 ± 0.00
PCA	0.73	0.45	0.17	0.24	0.62
LSTM	<u>0.80 ± 0.02</u>	0.83 ± 0.10	0.32 ± 0.03	0.46 ± 0.05	<u>0.71 ± 0.05</u>
LSTM-AE	0.83 ± 0.00	<u>0.96 ± 0.03</u>	<u>0.37 ± 0.01</u>	<u>0.53 ± 0.01</u>	0.72 ± 0.01
MTAD-GAT	0.79 ± 0.04	1.00 ± 0.00	0.22 ± 0.16	0.33 ± 0.23	0.69 ± 0.01
GDN	0.75 ± 0.02	0.85 ± 0.13	0.10 ± 0.10	0.16 ± 0.16	0.59 ± 0.06
DyEdgeGAT	0.69 ± 0.03	0.30 ± 0.13	0.19 ± 0.15	0.22 ± 0.15	0.50 ± 0.09
IA-STGNN	0.50 ± 0.05	0.72 ± 0.05	0.36 ± 0.06	0.48 ± 0.06	0.56 ± 0.04

Table E.25: FCCU AD metrics for fault Fhn_sensorDrift. Best values in bold, second best underlined.

	Accuracy	Precision	Recall	F1	AUC
VAR	0.65 ± 0.00	0.91 ± 0.00	<u>0.50 ± 0.00</u>	<u>0.64 ± 0.00</u>	0.72 ± 0.00
PCA	0.79	0.67	0.41	0.51	0.70
LSTM	0.82 ± 0.01	0.86 ± 0.08	0.40 ± 0.02	0.54 ± 0.03	0.73 ± 0.03
LSTM-AE	<u>0.85 ± 0.00</u>	<u>0.97 ± 0.03</u>	0.45 ± 0.00	0.61 ± 0.01	<u>0.76 ± 0.00</u>
MTAD-GAT	0.84 ± 0.01	1.00 ± 0.00	0.39 ± 0.04	0.56 ± 0.04	0.73 ± 0.00
GDN	0.84 ± 0.01	0.95 ± 0.04	0.41 ± 0.03	0.57 ± 0.03	0.73 ± 0.01
DyEdgeGAT	0.68 ± 0.04	0.25 ± 0.16	0.17 ± 0.17	0.19 ± 0.18	0.50 ± 0.12
IA-STGNN	0.88 ± 0.02	0.88 ± 0.01	0.95 ± 0.02	0.91 ± 0.01	0.85 ± 0.02

Table E.26: FCCU AD metrics for fault UAf_decrease. Best values in bold, second best underlined.

E Experiments

	Accuracy	Precision	Recall	F1	AUC
VAR	0.65 ± 0.00	0.91 ± 0.00	<u>0.50 ± 0.00</u>	<u>0.65 ± 0.00</u>	0.72 ± 0.00
PCA	0.79	0.66	0.39	0.49	0.69
LSTM	0.80 ± 0.02	0.84 ± 0.09	0.33 ± 0.05	0.47 ± 0.05	0.61 ± 0.09
LSTM-AE	0.85 ± 0.00	<u>0.97 ± 0.03</u>	0.46 ± 0.01	0.63 ± 0.01	<u>0.76 ± 0.00</u>
MTAD-GAT	<u>0.83 ± 0.01</u>	1.00 ± 0.00	0.35 ± 0.02	0.51 ± 0.03	0.71 ± 0.01
GDN	0.82 ± 0.00	0.95 ± 0.04	0.35 ± 0.02	0.51 ± 0.02	0.71 ± 0.01
DyEdgeGAT	0.66 ± 0.03	0.22 ± 0.07	0.09 ± 0.00	0.12 ± 0.01	0.38 ± 0.06
IA-STGNN	0.81 ± 0.01	0.86 ± 0.01	0.84 ± 0.01	0.85 ± 0.01	0.80 ± 0.02

Table E.27: FCCU AD metrics for fault `condEff_decrease`. Best values in bold, second best underlined.

	Accuracy	Precision	Recall	F1	AUC
VAR	0.65 ± 0.00	0.91 ± 0.00	<u>0.51 ± 0.00</u>	<u>0.65 ± 0.00</u>	0.72 ± 0.00
PCA	0.82	0.71	0.50	0.59	0.73
LSTM	0.84 ± 0.01	0.88 ± 0.07	0.45 ± 0.02	0.59 ± 0.01	0.72 ± 0.04
LSTM-AE	<u>0.86 ± 0.00</u>	<u>0.97 ± 0.03</u>	0.48 ± 0.00	0.64 ± 0.01	<u>0.76 ± 0.00</u>
MTAD-GAT	0.85 ± 0.00	1.00 ± 0.00	0.45 ± 0.01	0.62 ± 0.01	0.74 ± 0.00
GDN	0.85 ± 0.00	0.96 ± 0.03	0.47 ± 0.01	0.63 ± 0.01	0.74 ± 0.01
DyEdgeGAT	0.67 ± 0.03	0.29 ± 0.10	0.13 ± 0.04	0.18 ± 0.05	0.51 ± 0.08
IA-STGNN	0.91 ± 0.01	0.88 ± 0.01	1.00 ± 0.00	0.94 ± 0.01	0.88 ± 0.01

Table E.28: FCCU AD metrics for fault `deltaP_increase`. Best values in bold, second best underlined.